claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-af746cfb1321afe8b.jsonl`
- SHA-256: `603d7c921fe279f69015187d2c9c32005d0e739fa8aedd2bdb7956754747e363`
- Source modified: `2026-06-10T05:44:50+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_2a64bc88-2a8`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-10T05:33:14.932Z)

CONTEXT ? two sibling repos on a Windows machine:
- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).
- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).
Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources.
ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.
QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.
TASK: deep ENGINEERING audit of the indexer's runtime pipeline code. Read docs/CODE_MAP.md FIRST (dense navigation map with known gotchas ? do not just re-report what it already flags; confirm and go DEEPER). Then deep-read: backend/main.py, backend/config/ (models.py, loader.py), backend/pipeline/segmenters/ (base + all 5), backend/pipeline/detectors/ (base, tracknet_runner, wasb_runner, wasb_infer, rally_gate), backend/pipeline/validators/ (all), backend/pipeline/stitcher/compiler.py, backend/pipeline/audio/hit_detector.py, backend/infrastructure/database.py, backend/providers/, backend/sports/, backend/annotations/, backend/reporting/ (incl. spec.yaml), backend/utils/.
ASSESS: (1) architecture quality ? registry/ABC plugin design, layering, seams for the planned platform work (StorageBackend/ComputeBackend, async jobs); (2) correctness risks ? real bugs, copy-paste drift between twin segmenters, contract mismatches; (3) code duplication and dead code; (4) API design of the FastAPI surface (validation, error responses, blocking behavior); (5) DB schema + migration ladder quality; (6) config system (shallow merge, extra=allow, .get shim) ? is it a time bomb or fine; (7) readiness for the NEXT committed slice (async job model) ? what will hurt.
Severity-rate findings honestly: 'critical' = would corrupt data / break prod runs; 'high' = real bug or design flaw with concrete consequence; 'medium' = drift/fragility; 'low'/'info' = polish. NEW findings (not already in CODE_MAP gotchas) are worth the most; if you confirm a CODE_MAP gotcha matters more than documented, say so explicitly.

2. user (2026-06-10T05:33:21.873Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-255 of 330 total (27423 tokens, cap 25000). Call Read with offset=256 limit=255 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-

reminder>

```
1       # CODE_MAP.md
2
3       **Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-
hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a
segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.
4
5       **Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this
doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure
(this revision follows #40).
6
7       **LEGEND**
8       - `@path` = file (always repo-relative)
9       - `::sym` = symbol (class/func/const) in the nearest preceding `@path`
10      - `->` = dataflow / call / "produces"
11      - `§` = data contract (canonical shape)
12      - `?` = gotcha / footgun
13      - `?` = registry / plugin extension point
14      - `WSL` = runs inside WSL2 conda env, NOT the Windows Python process
15
16      ---
17
18      ## 1. PIPELINE
19
20      ### 1A. Runtime: video file -> highlight reel
21      ```
22      typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-
defaults, never fatal)
23                                                  @backend/config/loader.py::load_config ->
@backend/config/models.py::AppConfig
24        -> video_id = MD5(whole file)           @backend/utils/hashing.py::compute_video_id
(single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint
copies)
25        -> validate
@backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
26              FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity
-> SceneCut -> Blur -> Relevance   (? ACTUAL registration/exec order ? see
@backend/pipeline/validators/__init__.py)
27              [validation FAIL -> persist status="failed", return early; report STILL emitted
in finally]
28        -> db.add_video(video_id, filepath, status, [r.dict()])
@backend/infrastructure/database.py
29        -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback
'motion')
30        -> seg = segmenter_registry.get(seg_name)(db, global_config)   ?
@backend/pipeline/segmenters/base.py::segmenter_registry  (400 + available list if missing)
31        -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
32              [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here
without stitching]
33        -> finally: emit_indexer_report(...)    @backend/reporting/__init__.py  (? ALWAYS runs
? even on validation failure / exception; NEVER raises; grafts response["reports"])
34        -> select segment ids -> [(start,end)] (+ stitching padding)
35        -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
36              per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
37      ```
38      Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same
shape):
39      ```
40      P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
41       -> P2 candidate "action windows" from motion/trajectory  [STOP: candidates persisted
via db.add_candidate_segment]
42       -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor ->
```

```
provider.analyze_video(clip, prompt)   ? provider_registry
      -> P4 db.add_play_segment + db.link_candidate_to_segment
      ```
      Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate
(clamp/dedup) -> add_play_segment.
      Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground
truth).

      ### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
      ```
      WasbHybridSegmenter.process_video
@backend/pipeline/segmenters/wasb_hybrid.py
       -> WasbRunner.healthcheck()  (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
       -> WasbRunner.run_predict(video, out_dir)
            stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
            -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU
detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
       -> TrackNetRunner.parse_trajectory_csv(csv)
@backend/pipeline/detectors/tracknet_runner.py  (shared, re-exported by WasbRunner)
       -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start,
velocity_thresh, merge_gap, min_window_duration, chunk_index)  [the model-agnostic WINDOWING
BRAIN]
      ```
      (`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width`
fallback (30.0, 1280.0).)

      ### 1C. Calibration / eval flow (NO pipeline run unless told)
      ```
      GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts ->
List[Interval]
      DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval]   stage
? {candidates, final}
        -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
        -> harness.evaluate -> EvalReport -> report_to_dict
             -> batch.aggregate (corpus micro/macro)            @backend/eval/batch.py
             -> regression.regress(_with_provider) vs baseline   @backend/eval/regression.py
[CI gate]
      Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,
cross_validate}  @backend/eval/calibration.py
      Distill Gemini->local: calibrate_local (thresholds) OR distill_local (learned
classifier)  @backend/eval/{calibrate_local,distill_local}.py
      Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
      Gemini refinement driver (tuning, standalone):
gemini_refine.{refine_window,full_video_rallies}  @backend/eval/gemini_refine.py
      Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run
@backend/eval/audio/e1_probe.py
      ```
      Entrypoints: `@run_eval.py` (single video score), `@run_phase3_eval.py` (manifest batch,
can segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning
drivers).

      ## 2. SUBSYSTEMS

      ### 2.0 Orchestration & config
      - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS).
Static UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig`
(INFO, timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces
stray prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI
summaries may still use `print` for direct output.
```
```

82 - `::app` FastAPI; `::db_instance`,`::global_config` (`Optional[AppConfig]`) module globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` -> every endpoint NPEs.
83 - `::process_video` `@POST /api/process` -> validate -> add_video -> segment -> `finally:` always `emit_indexer_report` (never raises). `::compile_highlights` `@POST /api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET /api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs `indexing.motion_threshold` (dual model/dict read).
84 - `::load_config` ? ? legacy thin wrapper returning a **dict** (`load_app_config(path).model_dump(...)`); kept for transition ? new code imports `load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest` pydantic bodies.
85 - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on `OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`. `compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
86 - ? `--segmenter` argparse `choices` frozen at build time from `segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI default_segmenter fallback `'motion'`.
87 - `@run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()` (blocks automation), skips validation. Config via `backend.config.load_config` (typed); segmenter, padding, and db path all read from the model (no literal fallbacks).
88 - `@run_eval.py::main` ? score one video's DB preds vs GT CSV. `--stage{candidates,final,both}`. Exit 2=arg/IO. ? pure scorer; errors if video never processed. `::_default_db_path` resolves via `backend.config.load_config` (`output.output_dir`/`output.db_name`). `get_file_md5` is an alias for `compute_video_id`.
89 - `@run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate. `load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as label); config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed `AppConfig` (same object the live pipeline feeds segmenters).
90 - `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic: **0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare (snapshots even a regressing run); `::_default_db_path` resolves via `backend.config.load_config`.
91
92 - `@backend/config/__init__.py` ? ? canonical config import surface. Re-exports `load_config`, `save_default_config` (from `.loader`) + `AppConfig`, `Config`, `IndexingConfig`, `ValidationConfig` (from `.models`); `__all__` = exactly those 6. Use `from backend.config import load_config, AppConfig`.
93 - `@backend/config/models.py` ? Pydantic v2 models; **single source of truth for defaults**.
94 - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`, `ai_model='gemini-2.5-flash'`; sections `validation`,`indexing`,`output`,`stitching` (`default_factory`). `model_config = {extra:'allow', validate_assignment:True}`. ? `extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance).
95 - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain dicts** (`model_dump(mode='json')`, NOT the model) so chained `config.get("indexing",{}).get(...)` keeps working; falls back to searching a leaf key in any section (sections discovered dynamically from `model_fields`). ? "bit every validator on real runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises `KeyError` for unknown top-level keys else delegates to `.get`.
96 - `::IndexingConfig` ? segmenter/detector knobs (`motion_threshold=0.08`, `min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`, `chunk_overlap=10.0`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`, `ai_handoff_padding=2.0`, `ai_max_workers=5`, `log/store_yolo_candidates=True`, nested `tracknet`). ? `default_segmenter='yolo_hybrid'` HERE but config.json overrides to `'gemini'`. ? `::IndexingConfig.segmenter_must_be_registered` `@field_validator` is a **no-op pass-through** ? a typo'd segmenter validates fine, only fails later at `segmenter_registry.get(...)`.
97 - `::ValidationConfig` (`min_resolution_width=1280`,`min_resolution_height=720`,`min_fps=29.9`,`min_blur_threshold=20.0`,`max_scene_cuts_per_minute=0.5`, nested `relevance`); `::ValidationRelevanceConfig` (court HSV gates, `court_pixels_min_ratio=0.05`). `::TrackNetConfig` (WSL invocation contract: `wsl_distro='Ubuntu'`, `repo_dir`, `conda_env='TrackNetV4'`, `weights_path=''`, `queue_length=5`, `velocity_threshold=0.02`). `::OutputConfig` (`default_highlights_name`, `db_name='sports_indexer.db'`, `output_dir='output'` ? CLI `--output-dir` overrides it on the typed model in `main()`).

`::StitchingConfig`
(`begin_padding=0.5`,`end_padding=0.5`,`use_cache=False`,`min_shot_count=1`). `::Sport` Literal
of 5 sports. `::Config = AppConfig` alias.
98   - `@backend/config/loader.py` ? load/save with defaults-merge.
99   - `::DEFAULT_CONFIG_PATH = Path("config.json")` (? CWD-relative, NOT repo-root).
`::_get_builtin_defaults` = `AppConfig().model_dump(...)`.
100   - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-
level merge** (`{**defaults, **file_data}`): a section in config.json REPLACES the whole
defaults dict for that section (Pydantic then re-applies field defaults for missing leaves).
Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-
fatal).
101   - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present
(used by setup.py/`--setup`). `::get_default_config` ? ? transitional plain-dict alias.
102   - `@config.json` ? on-disk config; mirrors `AppConfig`. ?
`indexing.default_segmenter='gemini'` diverges from model default `'yolo_hybrid'` ? file
wins at runtime.
103   - `@setup.py` ? `::init_default_config` ? delegates to
`backend.config.save_default_config` (single-source defaults; skips if config.json exists).
`::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python?3.8,
ffmpeg/ffprobe on PATH, optional torch+CUDA).
104
105   ### 2.1 Segmenters (under pipeline/segmenters) ?
106   - `@backend/pipeline/segmenters/base.py`
107   - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract
`name`,`description`,`process_video`. § `process_video(video_id, video_path, player_pool=None,
max_frames=None) -> List[dict{id,start_time,end_time,duration,metadata}]`.
108   - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new
segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module
bottom, AND import the module in `@backend/pipeline/segmenters/__init__.py` (registration is an
IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `.name` ->
name/description/`__init__` MUST NOT touch `self.db`/`self.config`. ? `list_available()` re-
instantiates + swallows exceptions into "No description available." (a throwing `description` is
masked).
109   - `@backend/pipeline/segmenters/__init__.py` ? imports all 5
(`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid`) = the de-facto registration manifest;
re-exports them + `segmenter_registry` via `__all__`.
110   - `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name
`'wasb_hybrid'`) ? WSL WASB substrate (MIT; ships pretrained badminton weights, the LIVE
default). Imports `WasbRunner`/`WasbConfig` from `pipeline/detectors/wasb_runner`. Config
`indexing.wasb.velocity_threshold(0.02)` + shared
`dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`. ? whole-
video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates`; needs
`{PROVIDER}_API_KEY` (missing -> []); `detection_params` has NO `queue_length` (unlike
tracknet). ? divergent shot_count: `int(shot_count)` inside try with no `is None` short-circuit
(copy-paste drift).
111   - `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`'tracknet_hybrid'`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig`
from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`;
`detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably
yolo_hybrid).
112   - `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name
`'yolo_hybrid'`, no YOLO remains ? torchvision detector + supervision ByteTrack; ADR-005).
`::_PERSON_CLASS_ID=1` ? (ultralytics used 0; wrong value -> zero detections).
`::_lazy_load_model` (lazy `torchvision` import so module/tests load without it),
`::_extract_action_windows_from_chunk`, `::_make_tracker` (per-chunk ByteTrack ? track IDs
chunk-local), `::_DETECTOR_FACTORIES`/`::_DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. ?
velocity at `effective_fps = fps/frame_skip` -> `yolo_velocity_threshold` coupled to
`frame_skip`; pipelined-vs-sequential P2 (prefetch ffmpeg overlaps sequential GPU);
`yolo_tracker`/`bytetrack.yaml` is dead/back-compat. ? `gemini_*` config keys are deprecated
aliases still honored.
113   - `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `'gemini'`) ?
pure-AI, no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded;
`max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts
colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning
table).

114   - `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `'motion'`) ? pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM** (random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing server/receiver; uses `print()`; honors `max_frames`.
115
116   ### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)
117   - `@backend/pipeline/detectors/base.py` ? DetectorRunner ABC (healthcheck() -> Tuple[bool,str] never-raises; run_predict(...) -> Optional[str] CSV path). Contract deliberately matches the pre-existing tuple returns and call sites in the hybrids. Rich module docstring includes "how to add a new runner", Linux-native path notes (the de-risk goal), and guidance on threading health into reports. Re-exported from `detectors/__init__.py`. This finishes the WSL de-risk seam (low-regret 6 / review item 1).
118   - `@backend/pipeline/detectors/tracknet_runner.py`
119   - `::TrackNetConfig`(+`.from_indexing_cfg`; ? key drift `wsl_distro` JSON -> `distro` field), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
120   - `::to_wsl_mnt_path` (`C:\x`->`/mnt/c/x`; ? POSIX/`~` passthrough, UNC unhandled), `::TrajectoryPoint`(frame,visible,x,y).
121   - **`::parse_trajectory_csv` @staticmethod** + **`::trajectory_to_action_windows` @staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path` defaults `''` -> `run_predict` returns None; predict.py only `.mp4/.avi` + hard-names `<stem>_predict.csv`; `visible==False` resets prev (occlusion gap breaks track); `fps<=0->30`, `frame_width<=0->1280` silent; `track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout (hangs forever). `::_stage_video` "OK" sentinel.
122   - `@backend/pipeline/detectors/wasb_runner.py::WasbRunner` (inherits DetectorRunner) (+`::WasbConfig`, from `indexing.wasb`; ? `weights_path` HAS a real default `wasb_badminton_best.pth.tar` ? no required guard) ? stages `wasb_infer.py` (`::_INFER_WIN`) + video into WSL, runs in `wasb` env. `::parse_trajectory_csv`/`::trajectory_to_action_windows` = `staticmethod(TrackNetRunner.*)` (explicit rebind = the plugin-equivalence point). Output hard-named `<stem>_wasb.csv`. ? `wasb_infer.py` re-staged each run (editing Windows copy inert until staged); `_wsl_bash` duplicated (drift risk). Both runners now satisfy the DetectorRunner ABC for future swappability.
123   - `@backend/pipeline/detectors/wasb_infer.py` (WSL) ? `::run` core. ? imports WASB-SBDT repo modules + torch ? NOT importable on Windows. Reboot-resumable detector cache: `::_DET_FILE='det_raw.jsonl'`,`::_MANIFEST_FILE='manifest.json'`, `::CACHE_VERSION=1`, `::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line index`, rewrites to heal crash-truncated tail), `::manifest_compatible` (? `frames_dir` abspath'd but `weights` compared raw), `::_atomic_write_text/_atomic_write_trajectory`, `::_dets_to_tracker` (xy MUST be np.array), `::_build_cfg` (`runner.gpus=[0]` hardcoded), `::extract_frames` (cv2-PNG SLOW), `::default_cache_dir` (`<stem>_wasbcache` next to out). ? GPU pass resumable; CPU tracker STATEFUL -> always fully re-run; `--fresh` ignores cache.
124   - `@backend/pipeline/detectors/rally_gate.py` ? Gate A net-crossing post-filter (pure, no I/O/model). `::estimate_play_axis` (larger-spread axis; ? ties -> 'x'), `::count_net_crossings` (sign changes vs median net; deadband jitter zone), `::gate_windows`/`::apply_gate` (`min_crossings=2` default; kills single-toss service feeds). `::GatedWindow`(start,end,crossings,visible_points,kept). ? window contract here is `(start,end)` tuples NOT the rich window dict ? caller must adapt. Consumed by calibrate_local / distill_local / export_wasb_segments.
125
126   ### 2.3 Eval & calibration (all pure core; I/O in CLI drivers)
127   - `@backend/eval/metrics.py` ? scoring primitive (THE spine). `::interval_iou`, `::match_intervals` (greedy, deterministic), `::score`->`::Score`(`.as_dict()` = wire shape), `::pr_curve`, `::Match`, `::MatchResult`. ? greedy not Hungarian; empty matches -> 0.0 (not NaN) for mean_iou/boundary errors.
128   - `@backend/eval/annotations.py` ? generic GT loader. `::load_annotations` (`start,end` CSV; RAISES on bad rows/`start>=end`), `::parse_timestamp` (decimal or MM:SS/HH:MM:SS), `::write_scaffold`.
129   - `@backend/eval/calibration.py` ? overfit guard. `::cross_validate` (within-video k-fold; ? defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video; needs ?2 labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT), `::select_best`, `::kfold_indices`, `::pct_improvement` (? from-zero -> `inf`), `::evaluate`. § `PredictFn = Callable[[Config], List[Interval]]` = the plug-in seam. ? `generalization_gap >~0.1` = overfit alarm.
130   - `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`, `::default_grid` (45 cfgs), `::make_predict_fn` (parses trajectory once, closes over `trajectory_to_action_windows`), `::load_golden_gts` (§`start_time,end_time` cols; ? SILENTLY

skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts,
make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local,
export_wasb_segments, gemini_refine, audio/e1_probe).
131   - `@backend/eval/calibrate_local.py` ? distill Gemini->local thresholds (windowing +
net-crossing gate, no cloud at runtime). `::local_grid` (180 cfgs), `::make_local_predict_fn`
(adds `rally_gate.apply_gate` when `min_crossings>0`), `::build_videos` (? `SystemExit` if a
labels file yields no rallies), `::run`/`::main`. §
`LocalConfig=(velocity,merge,min_dur,min_crossings)`, `BASELINE_LOCAL=(0.02,2.0,2.0,0)`;
manifest JSON shared with distill_local.
132   - `@backend/eval/export_wasb_segments.py` ? tuned cfg -> golden/slicer CSV + comparison
+ optional clip cut. `::TUNED=(0.05,1.0,1.0)` (? from ONE video),
`::SEG_FIELDS`,`::COMP_FIELDS`, `::build_comparison`, `::seconds_to_mmss`, `::_maybe_slice`
(lazy-imports rally_slicer). `--slice` requires `--video`. ? `write_segments_csv` output loads
straight into `rally_slicer.load_rally_labels` AND re-imported by gemini_refine ? schema is
load-bearing.
133   - `@backend/eval/batch.py` ? corpus aggregation (pure; orchestration is in
run_phase3_eval). `::aggregate` (micro pool TP/FP/FN, TP-weighted mean_iou, macro F1),
`::default_stages_for` (`auto`; `::_FINAL_ONLY_SEGMENTERS={motion,gemini}`),
`::resolve_video_path` (normalizes collector `./data\..` Windows paths), `::format_aggregate`.
134   - `@backend/eval/harness.py` ? DB-pred scorer (no re-run).
`::STAGES=('candidates','final')`, `::get_predictions`
(candidates->`query_candidate_segments(detector=)`; final->`query_play_segments({})`; ? unknown
stage raises ValueError; reused by regression.py),
`::evaluate`->`::EvalReport`(`::StageReport`)->`::report_to_dict` (input contract for
batch.aggregate), `::format_report_text`.
135   - `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES`
(5-d), `::extract_yolo_features` (? reads `row['stats']` dict from DB; missing fields default
0.0), `::label_candidates` (same IoU `match_intervals` as evaluator),
`::build_training_set`->(X,y,intervals); `feature_fn=` is the substrate plug-in seam.
136   - `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn).
`fit/predict_proba/predict/to_dict/from_dict/save/load`. ? `class_weight='balanced'` default;
stores feature standardization (mean/std) ? inference MUST use same feature ORDER; `_sigmoid`
clips z to [-30,30]; `lr=0.1,epochs=1000,l2=1e-3`; JSON `"model":"logreg"` tag.
137   - `@backend/eval/eval_partial_labels.py` ? score pipeline vs **PARTIAL human labels**
(PR #60). `::restrict_to_window` (drop preds outside `[0, last GT end]` so un-labeled-tail
detections aren't FPs ? THE partial-label methodology), `::run` (baseline/+gate/tuned/+gate ops
table + per-rally diff via `export_wasb_segments.build_comparison`), `::sweep`/`--sweep` (grid
vs labels; ? fit-on-self = optimistic), `::DEFAULT_GRID`, `::TUNED=(0.05,1.0,1.0)`. ?
`window_end` inferred from `max(GT end)` is WRONG if a video is labelled PAST its last rally ?
P1 fix = collector persists `labeled_window` in the manifest. Reuses calibrate_wasb + rally_gate
+ metrics.
138   - `@backend/eval/rally_seq_proto.py` ? **PROTOTYPE** learned per-FRAME rally segmenter =
owned-model **Gen-0 seed** (PR #61). 6 features over the WASB trajectory (`vis_rate, spd_mean,
spd_max, cross_rate, x_std, y_std`) -> `classifier.LogisticRegression`, honest LOVO.
`::build_frame_arrays`, `::features` (scipy `uniform/maximum_filter1d` windowed; ? speed NOT
fps-normalized yet ? 30 vs 60fps), `::labels_for`, `::probs_to_segments`
(smooth->threshold->merge/min-dur), `::roc_auc` (rank-based, no sklearn), `::run` (per-video AUC
+ LOVO-threshold F1 + oracle-threshold F1). ? DIRECTIONAL only (n?5, linear, threshold-transfer
unsolved); held-out per-frame AUC 0.83?0.92, **beats the heuristic on multi-court footage**
(Kushagra/gBaaddy). Manifest `output/human_lovo_manifest.json` (the n=5 human corpus) shared w/
calibrate_local.
139   - ? **MULTI-COURT / background-game confound (2026-06-08 finding):** WASB tracks EVERY
shuttle in frame incl. other courts; videos with background games (Kushagra, gBaaddy) have high
dead-time shuttle visibility (41?57% vs 18?22% clean) ? the velocity-windowing heuristic can't
find clean rally boundaries (F1 0.16/0.28 vs ~0.75 on single-court). Diagnostic = in-rally÷dead-
time visibility "contrast" (?3.6× heuristic works, ?1.9× fails). Lever = **foreground-court
isolation** (spatial gate to the prominent court, needs real court-region detection). Academy =
multi-court = the target env. See `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.
140   - `@backend/eval/distill_local.py` ? distill Gemini -> a LEARNED classifier (beyond
calibrate_local's threshold ceiling). `::FEATURE_NAMES` (5 fps/res-INVARIANT:
duration,peak,mean,active_ratio,net_crossings), `::build_candidates` (recall-first proposal),
`::predict_rallies` (proba filter then `gemini_refine.merge_overlaps(gap_tol=1.0)`),
`::run`/`::main`. consts `PROPOSAL=(0.005,1.0,0.5)`, `BASELINE_LOCAL=(0.02,2.0,2.0)`. ? leave-
one-video-out needs ?2 videos; final model trained on all.
141   - `@backend/eval/gemini_refine.py` ? refine WASB candidates with Gemini over SHORT

padded clips (precision lever; eval/tuning, NOT the live pipeline).
`::merge_overlaps(gap_tol=0.0)` (reused by distill_local with gap_tol=1.0), `::refine_window` (?
FRESH genai client per worker ? shared throws socket errors), `::full_video_rallies`,
`::run`/`::main` (`--full-video` skips `--trajectory`). ? requires `GEMINI_API_KEY` (raises
SystemExit); imports `BASELINE` from calibrate_wasb +
`TUNED`/`write_segments_csv`/`seconds_to_mmss` from export_wasb_segments.
142   - `@backend/eval/regression.py` ? CI gate. `::DEFAULT_TOLERANCE=0.05`,
`::DEFAULT_BASELINE_PATH=output/regression_baseline.json`, `::regress`,
`::regress_with_provider` (GT via ?`annotation_registry.get(tier)`;
"file"->FileProvider/"vendor"->HumanVendor/"auto"->oracle),
`::save_baseline`/`::load_baselines`, `::RegressionResult`. ? imports `annotation_registry` from
`backend.annotations` NOT `backend.eval.annotations`; gate fires on precision OR recall drop
(F1 reported, not gated); no baseline -> `regressed=False` (silent pass).
143
144   ### 2.3a Audio (E1 probe; NOT adopted for fusion ? ADR-007)
145   - `@backend/pipeline/audio/hit_detector.py` ? classical-onset shuttle-hit detector. Pure
numpy + stdlib `wave` (NO scipy); ffmpeg only for extraction. `::HitEvent`(t,strength),
`::extract_audio` (ffmpeg -> mono 16k PCM WAV), `::load_wav`, `::onset_envelope` (pure-numpy
STFT spectral flux; `band=(lo,hi)` band-limit), `::detect_hits` (adaptive `mean+k·std`;
`k`=sensitivity knob), `::detect_hits_in_video`. ? separate signal ? WASB never modified by
this; per audio-e1-findings recall 100% but discrimination ~2.2×, band-pass null -> NOT adopted
(PR #35).
146   - `@backend/eval/audio/e1_probe.py` ? E1 GO/NO-GO diagnostic (no training/fusion).
`::cluster_hits` (groups thwacks; ? `min_hits>=2` so a lone service-feed hit isn't a rally),
`::run` (discrimination ratio + recall recovery + audio-only P/R/F1), `::main`. reuses
`hit_detector` + `calibrate_wasb.{load_golden_gts,make_predict_fn,BASELINE}` + `metrics`.
147
148   ### 2.4 Stitcher / tools / utils
149   - `@backend/pipeline/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw,
segments:[(start,end)], out_name)`. Per-clip re-encode -> `-f concat -c copy`. `::_parse_time`
(mirrors gemini.parse_time; ? unparseable -> 0.0), `::_get_video_duration`. ? if duration probe
fails (0.0) NO boundary validation; `begin_padding`(0.5)/`end_padding`(1.0) defaults; temp clips
in a TemporaryDirectory under output_dir; raises ValueError/FileNotFoundError/RuntimeError; uses
`print()`. ? two-stage codec contract: clips re-encoded with IDENTICAL settings precisely so
final concat can stream-copy ? change a preset and concat may break.
150   - `@backend/tools/rally_slicer.py` ? standalone CLI. `::compute_clip_windows` (pure,
unit-tested), `::load_rally_labels` (golden schema; ? silently skips degenerate rows),
`::slice_rallies`, `::ClipWindow`(rally:str), `::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (?
duplicate compiler's by copy). `_read_time -> backend/eval/annotations.parse_timestamp`.
151   - `@backend/utils/video.py` ? `::get_video_duration` (ffprobe; ? 0.0 on failure =
"unknown"), `::extract_video_chunk(...,reencode=False,scale_width=None)`. ? copy mode cuts at
nearest keyframe (drift) ? pass `reencode=True` for short/accurate clips; `scale_width` requires
`reencode=True`.
152   - `@backend/utils/geometry.py` ? `::get_velocity_normalised` (fraction of frame-width/s;
? raw-px fallback if width<=0; used by yolo_hybrid), `::calculate_iou` (? +1 px convention
inflates IoU), `::get_center/get_distance/get_velocity`.
153
154   ### 2.5 Validators ?
155   - `@backend/pipeline/validators/base.py::VideoValidator` ABC, `::ValidationResult`
(pydantic: validator_name,passed,message,details), `::ValidationRegistry` (ordered LIST ?
registration order = exec order; ? no dedup), `::registry` (singleton). § `validate(video_path,
config, context) -> ValidationResult`. `::run_all_with_context -> (results, context)` (wraps
each validate in try/except -> failed result not abort); `::run_all` discards context. ? **add a
validator**: subclass + `registry.register(MyValidator())`.
156   - `@backend/pipeline/validators/__init__.py` ? registers in EXEC order: FormatValidator,
ResolutionFPSValidator, PTSContinuityValidator, SceneCutDensityValidator, BlurValidator,
RelevanceValidator (? producers before consumers ? format_check first; ? import mutates global
registry as side effect).
157   - `@backend/pipeline/validators/format.py::FormatValidator` (`format_check`) ?
PRODUCER of `context['ffprobe_meta']`; ? MUST run before resolution_fps; container check is
substring `'mp4'` (so MOV-family passes); external `ffprobe`.
158   - `@backend/pipeline/validators/resolution_fps.py::ResolutionFPSValidator`
(`resolution_fps_check`) ? CONSUMES `ffprobe_meta` (? hard-fails "Run format_check first" if
absent). `min_fps` 29.9; min res 1280x720.
159   - `@backend/pipeline/validators/pts_continuity.py::PTSContinuityValidator`

(`pts_continuity_check`) ? re-probes (no context). ? 10.0s keyframe-gap threshold hardcoded (comment says 8.0s ? drift, NOT config-driven); any backward jump fails; `<2` keyframes PASSES (too short).
160 - `@backend/pipeline/validators/scene_cut.py::SceneCutDensityValidator` (`scene_cut_check`) ? OpenCV. ? `cut_threshold=0.6` hardcoded (NOT config); samples ~2fps, caps at 100 SAMPLED frames (~first 50s); `max_scene_cuts_per_minute` (0.5) is the config fail threshold.
161 - `@backend/pipeline/validators/blur.py::BlurValidator` (`blur_check`) ? Laplacian var < `min_blur_threshold`(default 100.0 in-file, 20.0 in models/config) = fail; samples 15 frames. ? magic absolute, not resolution-normalized.
162 - `@backend/pipeline/validators/relevance.py::RelevanceValidator` (`relevance_check`) ? HSV court-green ratio (? purely color, no geometry despite name); lazy `import backend.sports`; 3-tier config precedence (`validation.relevance` -> sport profile -> hardcoded); ? unknown sport -> empty cfg, falls back to defaults, does NOT fail.
163
164 ### 2.6 Sports / Providers / Annotations ?
165 - `@backend/sports/base.py::SportProfile` ABC (`name`,`get_prompt`,`get_relevance_config`); `@backend/sports/badminton.py::BadmintonSportProfile` (the Gemini prompt + HSV relevance cfg); `@backend/sports/__init__.py::sport_registry` (+`::SportRegistry`, auto-registers badminton; ? `register` instantiates profile to read `.name`). ? **add a sport**: subclass `SportProfile`, `sport_registry.register(MyProfile)`. § prompt emits JSON array `{start_time(float DECIMAL SEC), end_time, shuttle_exchange_count(int), shot_count_confidence('Low'|'Medium'|'High')}`. ? prompt actively defends against MM:SS; `get()` returns a fresh instance each call.
166 - `@backend/providers/base.py::AIVideoProvider` ABC (`__init__(api_key, model_name)`); `@backend/providers/gemini.py::GeminiProvider` (lazy `::client` property; `::MAX_UPLOAD_MB=500`; upload->poll->generate JSON temp 0->delete); `@backend/providers/__init__.py::provider_registry` (+`::ProviderRegistry`, auto-registers `'gemini'`). ? **add a provider**: subclass, `provider_registry.register('name', MyCls)` (key passed EXPLICITLY); `get(name, api_key, model_name)`. § `analyze_video(path, prompt, chunk_duration=0.0, label='') -> (segments:list, metrics:dict{upload_time,process_time,gen_time})`. ? blocking 10s poll; returns `([], metrics)` on ANY failure (empty != error); oversize clip silently skipped; orphaned remote file possible if process dies mid-run.
167 - `@backend/annotations/base.py::AnnotationProvider` ABC (`submit/poll/fetch/annotate`), `::Interval` (= metrics.Interval), `::QUEUED/RUNNING/DONE/FAILED`/`::STATUSES`, `::AnnotationProviderRegistry` (decorator-friendly `register` returns the class; ? keys off `provider_cls().name`, falls back to `__name__` if no-arg ctor throws; `list_available()` -> LIST not dict), `::annotation_registry`. ? **add a tier**: subclass (define `name`), `annotation_registry.register(MyCls)` at module bottom + import in `__init__.py`. § `fetch -> List[Interval]` sorted; contract says RAISE (not `[]`) on unavailable.
168 - `@backend/annotations/file_provider.py::FileProvider` (`'file'`; job_id IS the resolved CSV path; `_resolve` -> `<csv_dir>/<video_ref>.csv`; -> `backend.eval.annotations.load_annotations`; ? `guideline`/`sport` ignored).
169 - `@backend/annotations/automated_provider.py::AutomatedProvider` (`'auto'`; GS-4 stub; injectable `::Labeler`, in-process `_jobs` store, `::warn_if_same_lineage` ORACLE RULE ? only warns, does not block; default `_not_configured` RAISES NotImplementedError). ? get via `annotation_registry.get('auto', labeler=fn, lineage='...')`; registered as `"auto"` only because no-arg ctor succeeds.
170 - `@backend/annotations/__init__.py` ? ? `noqa F401` imports of file_provider/automated_provider are load-bearing (removing de-registers providers).
171
172 ### 2.7 Infrastructure (DB) ?
173 - `@backend/infrastructure/database.py::Database` ? SQLite, 4 tables (`videos/play_segments/events/candidate_segments`), `PRAGMA user_version` migration ladder in `::_init_db` (currently =1, `version<1` creates `candidate_segments`). `::_get_connection` (? re-issues `PRAGMA foreign_keys=ON` per connection ? else CASCADE/SET NULL silently skipped). Methods: `add_video`,`add_play_segment`,`add_event`,`add_candidate_segment`,`link_candidate_to_segment`,`query_candidate_segments(detector=,only_unlabeled=)`,`query_play_segments`,`get_video`, `clear_video_data`. ? candidate_segments is the detector-agnostic fine-tuning extension point (common fields columns, detector-specifics in `stats`/`detection_params` JSON; index `idx_candidates_video_detector`). ? `add_video` is `INSERT OR REPLACE` -> cascade-DELETES dependent segments on re-ingest; events column is `type` not `event_type`; `query_play_segments` filters use truthiness so `duration_min=0`/empty-string = "no filter"; write helpers swallow exceptions.
174

2.8 Reporting / observability ? (per-video, soft-dep `obsreport`)
- `@backend/reporting/__init__.py` ? `::emit_indexer_report(*, db, video_id, video_path, config, val_results, val_context=None, segmenter_requested=, segmenter_actual=, was_reingest=False, segmentation_ran=True)` ? single entrypoint, keyword-only. Normalizes typed config (`config.model_dump(...)` if `hasattr model_dump`); short-circuits to None if unavailable/disabled; pulls play_segments/candidates from `db` (read-only `query_play_segments(vid,{})`/`query_candidate_segments(vid)`); delegates to `harvest.record_run`; writes via `rec.write(output_dir=config["report"]["output_dir"])`. ? wraps everything in bare `except Exception: return None` ? **NEVER raises** (designed for `main.py`'s `finally:`); writes `<video>.report.json`/`.report.md`/`.summary.md` next to video. `::OBSREPORT_AVAILABLE` ? soft-dep gate (True only if `import obsreport` AND `from . import harvest` both succeed). `::report_enabled(config)` reads `config["report"]["enabled"]` default True.
- `@backend/reporting/harvest.py` ? records raw FACTS into `obsreport.RunRecorder` (the footgun RULES live in spec.yaml, not here). `::record_run(...)` ? top-level assembler (does NOT write); builds RunRecorder, reads `config_default = config["indexing"]["default_segmenter"]`, runs 4 stage builders + `_highlights`, `rec.finalize()`. `::_AI_SEGMENTERS={yolo_hybrid,tracknet_hybrid,wasb_hybrid,gemini}` ? (add new AI segmenters here). `::video_meta_from_context` (derives VideoMeta from `val_context['ffprobe_meta']`; now also `audio_present` for Input Quality reports extension; § `fps_source ? {unknown,probed,fallback}`), `::_validation_stage`, `::_segmentation_stage` (§ `details.{segmenter_requested,segmenter_actual,config_default,matches_config_default,segmenter_explicitly_requested,was_reingest,detector,metadata_source}`; ? marks `motion` output "synthetic/heuristic"), `::_ai_stage` (§ `details.{ai_based,provider,model,api_key_present}`; metric `confirmed_segments`), `::_highlights` (coverage_pct). ? `record_run` fps fallback: `fps_source=='unknown'` + ran -> promotes to "fallback", forces fps=30.0 (mirrors runners). `::_SPEC_PATH`/`::load_spec` resolve `spec.yaml` next to module. (Soft import of obsreport so pure helpers like video_meta are importable for tests/coverage even without the dep.)
- `@backend/reporting/spec.yaml` ? § declarative report layout + footgun engine consumed by obsreport. `::customer_verdict` (pass/pass_with_warnings/fail messages). `::sections` (ordered `{id,title,kind,audience?{both,customer,technical}}`). ? `::rules` ? each `{code,severity,when[{path,op,value}],message,faq_ref,customer_message?}`; `when` clauses AND'd; `path` dotted-navigates the recorder JSON. ? `path` strings are a HARD coupling to harvest's stage/metric/detail names ? renaming a detail in harvest.py silently breaks the matching rule. Rules: `silent_provider_noop`(error: no api key + 0 segs), `ai_empty_vs_no_rallies`(warn), `fps_fallback_used`(warn), `synthetic_motion_metadata`(warn), `segmenter_divergence`(info), `reingest_replaced_prior`(info), `validation_failed`(error: halts indexing).

---

3. DATA CONTRACTS (canonical shapes ? one place)

§ **Trajectory CSV** (produced by tracknet/wasb predict, consumed by `parse_trajectory_csv`):
`Frame,Visibility,X,Y` ? header+column order fixed; `Visibility==1`=visible; X,Y pixel coords.
-> `TrajectoryPoint{frame:int, visible:bool, x:float, y:float}` (frame-sorted).

§ **Candidate window dict** (produced by `trajectory_to_action_windows` AND yolo `_extract_action_windows_from_chunk`):
`{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float, mean_velocity:float, track_id_count:int(==1 for trajectory), chunk_index:Optional[int]}`.

§ **candidate_segments DB row** (`db.add_candidate_segment` / `query_candidate_segments`):
`{id, video_id, detector:str, chunk_index, start_time:REAL, end_time:REAL, duration:REAL, confidence:REAL(min(1.0,peak_velocity)), stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count}, detection_params:JSON{...}, confirmed_segment_id, human_label, notes, created_at}` (stats/detection_params deserialized on read).
`detection_params` keys by detector: wasb=`{detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path}`; tracknet adds `queue_length`; yolo=`{velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector_score_threshold,min_action_window_duration}`.

§ **play_segment dict** (`db.add_play_segment` / `query_play_segments`):
`{id:int, video_id, start_time:float, end_time:float, duration:float(end-start), server, receiver, metadata:dict, events:[...]}`. Hybrid P4 metadata `{shot_count, shot_count_confidence:'Unknown', detector:f'{family}-hybrid-{model}'}`; gemini detector = bare `model_name`; motion adds random `intensity/avg_speed_kmh`.

§ **AI segment dict** (provider output, consumed by hybrid P3 / gemini): `{start_time, end_time, shuttle_exchange_count?, shot_count_confidence?}` (offset by chunk/window start; `_candidate_id` stamped internally).

§ **GT / golden CSVs** (TWO conventions ? do not conflate):
- generic (`annotations.load_annotations`): `start,end` 2-col; optional header; decimal or MM:SS/HH:MM:SS; RAISES on bad row.
- golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`, `export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`; SILENTLY skips bad rows.

§ **det_raw.jsonl** (one window/line): `{"w":<window_index>, "f":[[frame_id,[[x,y,score,scale],...]], ...]}`. Lines MUST be contiguous (line N has `w==N`); first violation/JSONDecodeError -> truncate-heal.

§ **manifest.json** (cache): `{version, frames_dir(abspath), weights, sport, frames_in, frames_total, windows_total, windows_done}`. Reuse only if ALL match + `version==CACHE_VERSION`.

§ **collector manifest.json** (eval): `{eval_sets:[{video_id(label), local_path, csv}, ...]}`; ? DB key = file MD5, manifest `video_id` is a label.

§ **eval primitives**: `Interval=Tuple[float,float]` (start<end). `Match{pred_index,gt_index,iou}`. `MatchResult{matches, unmatched_pred_indices(=FP), unmatched_gt_indices(=FN)}`. `Score{iou_threshold,num_pred,num_gt,true_positives,false_positives,false_negatives,precision,recall,f1,mean_iou,mean_start_error,mean_end_error}.as_dict()`.

§ **WSL detector candidate** (in-WSL, model<->tracker): `{'xy':np.array([x,y]), 'score':float, 'scale':int}`; plain JSON form `[x,y,score,scale]`. ? `_dets_to_tracker` MUST rebuild `xy` as np.array (tracker does `np.linalg.norm`).

§ **HitEvent** (audio, `pipeline/audio/hit_detector`): `{t:float(sec), strength:float}` ? onset-envelope peak.

§ **observability report** (`reporting/harvest.record_run` -> `RunRecorder`): stages `validation/segmentation/ai_handoff` + highlights + verdict ? {pass,pass_with_warnings,fail}; flags `{code,faq_ref}`; `video_meta.fps_source ? {probed,fallback}`. ? stage/metric/detail names are coupled to `spec.yaml::rules[].when[].path` ? pairs that MUST stay in sync: `ai_handoff.details.{api_key_present,ai_based}`, `segmentation.metrics.segment_count`, `segmentation.details.{segmenter_actual,matches_config_default,segmenter_explicitly_requested,was_reinget}`, `validation.status`, `video_meta.fps_source`.

---

4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in this repo ? claims below are unverified against external source, only the repo-side caller wasb_infer.py was checked)

- Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra `compose(config_name='eval', overrides=[dataset=<sport>, model=wasb, detector.model_path=<weights>, runner.device=cuda, runner.gpus=[0]])`. `gpus=[0]` because box is single-GPU. (overrides VERIFIED in @backend/pipeline/detectors/wasb_infer.py::_build_cfg.)
- `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH), ImageNet normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read from cfg["model"] at runtime in wasb_infer.run.)
- `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats) -> (results, hms_vis)`. `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`. (caller uses `detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
- `trackers/online.py::OnlineTracker` ? STATEFUL, sequential per `update()` (assumes every frame fed once, in order, no gaps). `update(frame_dets)->{x,y,visi,score}`; `refresh()` resets. `det['xy']` MUST be np.array. Full ordered re-run is deterministic. (caller calls `build_tracker(cfg)`, `tracker.refresh()`, `tracker.update(...)` returning dict with `visi/x/y` ? consistent.)

223   - `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh, transform, seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path, center:Center(is_visible,x,y)}]}]`. (caller constructs exactly this ? VERIFIED in wasb_infer.run.)
224   - Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed, academic-data-trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial, avoided.
225
226   ---
227
228   ## 5. CROSS-REPO (sibling private repo `sports-data-collector` ? external, unverified here)
229   - Indexer keys video by **FILE MD5**; collector keys by human `video_id`. ? re-encoding/re-downloading changes bytes -> MD5; acquisition MUST precede processing; manifest tracks the exact file used.
230   - `python -m src.cli.curate export` (collector) -> `<out>/<sport>/<video_id>.csv` (indexer `start,end` format) + `manifest.json` -> consumed by `@backend/eval/batch.py` + `@run_phase3_eval.py`.
231   - Collector also emits golden rally labels (`rally_number,start_time,end_time,...`) -> consumed by `@backend/eval/calibrate_wasb.py` / `export_wasb_segments.py` / `@backend/tools/rally_slicer.py`.
232
233   ---
234
235   ## 6. GOTCHAS (cross-cutting footguns)
236   - **MD5 keying:** `video_id` = MD5 of WHOLE file via the single `@backend/utils/hashing.py::compute_video_id` helper (64KB chunks), imported by main + all run_*.py. Manifest `video_id` is a label, NOT the DB key.
237   - **Config access:** `backend/main.py` and all four `run_*.py` load via `backend.config.load_config` (typed `AppConfig`). The legacy `.get()` dict-compat shim remains for validators/segmenters. Remaining literal: a defensive `getattr(..., 'motion')` fallback in `main.py` that only fires if `global_config`/`.indexing` is `None` (the ASGI-import edge case).
238   - **Segmenter default:** single source of truth = model `IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships `gemini`). run_*.py read the model value (no literals); `main.py` keeps the defensive `'motion'` getattr fallback noted above.
239   - **`AppConfig.get` must return dict sections, not models:** the legacy `.get("indexing",{}).get(...)` chains everywhere depend on it; the `extra="allow"` model also silently keeps typo'd config keys (no rejection).
240   - **`output_dir`:** a real field on `OutputConfig` (default `"output"`); the CLI `--output-dir` overrides it on the typed model in `main()`, and `compile_highlights` reads the same `global_config.output.output_dir` ? honored end-to-end.
241   - **Gemini free-tier / API-key dead path:** hybrids + gemini return `[]` (not error) if `{PROVIDER}_API_KEY` missing OR WSL healthcheck fails ? a silent no-op. `analyze_video` also returns `[]` on real failure -> empty != "no rallies". The reporting `silent_provider_noop`/`ai_empty_vs_no_rallies` rules exist precisely to disambiguate this in the report.
242   - **Reporting never breaks a run:** `emit_indexer_report` swallows all exceptions (designed for `main.py` `finally:`) and is emitted even on validation failure. `obsreport` is a SOFT dep ? absent -> no-op.
243   - **spec.yaml <-> harvest.py coupling:** rule `path` strings dotted-navigate harvest's stage/metric/detail names; rename a detail in harvest.py and the matching rule silently stops firing. Add rules in spec.yaml, not code.
244   - **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` / runner fallbacks (30.0, 1280.0) silently mis-scale velocity thresholds; calibrate_wasb CLI bakes `59.94/1920`. The report's `fps_fallback_used` flag surfaces this.
245   - **WSL `/mnt` can't see Google Drive / network mounts;** stage video into native WSL fs (`~/clips`) ? `/mnt/c` reads are slow and Drive-backed paths invisible.
246   - **ffmpeg NOT in WSL** by design: all ffmpeg/ffprobe run Windows-side (must be on PATH); WSL only runs the GPU model. `extract_frames` uses cv2 PNG (slow) ? prefer host ffmpeg + `--frames_dir`. ? Don't import `wasb_infer` from Windows code (it imports torch + WASB repo modules).
247   - **GPU float nondeterminism:** detector pass is checkpointed/resumable but not bit-reproducible; the CPU tracker re-run over the ordered cache IS deterministic.
248   - **`output/` is gitignored:** DB, baselines, cache, reels live there. Predictions MUST

exist in `output/<db_name>` before any eval/regression.
249   - **Validator ordering coupling:** exec order is Format -> ResolutionFPS -> PTSContinuity -> SceneCut -> Blur -> Relevance; `resolution_fps_check` hard-fails unless `format_check` ran earlier (shared `context['ffprobe_meta']`). PTS 10.0s gap + scene-cut 0.6 correlation are NOT config-overridable.
250   - **`add_video` INSERT OR REPLACE + FK CASCADE** -> re-ingesting a `video_id` cascade-deletes its segments. Use `clear_video_data` deliberately. `_get_connection` MUST re-issue `PRAGMA foreign_keys=ON` per connection.
251   - **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-paste; fixes to P3/P4 must land in both. ? subtle drift: wasb's shot_count `int(None)`-via-TypeError fallback vs tracknet's `is None` check.
252   - **Two "annotations" modules:** `backend.eval.annotations` (GT CSV loader) vs `backend.annotations` (provider registry). regression.py imports the latter.
253   - **`calibrate_wasb.{load_golden_gts, make_predict_fn, BASELINE}` is a de-facto shared API** for calibrate_local, export_wasb_segments, gemini_refine, e1_probe ? changing its golden-CSV schema or `BASELINE` ripples widely. `gemini_refine.merge_overlaps` reused by `distill_local`; `harness.get_predictions` reused by `regression.regress`.
254   - **Hardcoded tuned constants:** `calibrate_wasb.BASELINE`, `export.TUNED`, slicer/compiler padding ? duplicated by copy, not import; re-tuning won't propagate.
255   - **`timeout_sec=0` = no timeout:** a hung WSL/GPU call blocks forever.

3. user (2026-06-10T05:33:29.870Z)

256   - **Stitch concat assumes identical encode:** per-clip re-encode then `-c copy` concat; if probe fails (0.0) no boundary validation.
257   - **Audio NOT in the live pipeline:** `pipeline/audio/hit_detector` + `eval/audio/e1_probe` are a diagnostic probe (ADR-007); recall 100% but discrimination ~2.2× -> not adopted for fusion.
258
259   ---
260
261   ## 7. RAMP PATHS (to do X -> open Y::symbol)
262 | Task | Open |
263 |---|---|
264 | Change a config default / add a config field | `@backend/config/models.py` (single source of truth) -> regenerate via `setup.py`/`save_default_config`; ? config.json may override (default_segmenter) |
265 | Read config in new code | `from backend.config import load_config, AppConfig` (`@backend/config/__init__.py`); attribute access, NOT raw `json.load` |
266 | Tune candidate windowing (velocity/merge/min-dur) | `@backend/pipeline/detectors/tracknet_runner.py::trajectory_to_action_windows`; cfg `config.json indexing.{wasb,tracknet}.velocity_threshold`, `dead_time_merge_gap`, `min_action_window_duration` |
267 | Add a segmenter | subclass `@backend/pipeline/segmenters/base.py::BasePlaySegmenter`; `segmenter_registry.register(...)` at module bottom; import in `@backend/pipeline/segmenters/__init__.py`; if AI-based add to `@backend/reporting/harvest.py::_AI_SEGMENTERS` |
268 | Change stitching padding | `config.json stitching.{begin_padding,end_padding}`; impl `@backend/pipeline/stitcher/compiler.py::VideoCompiler.compile_highlights`; mirror `@backend/tools/rally_slicer.py::BEGIN_PADDING/END_PADDING` |
269 | Add a sport | subclass `@backend/sports/base.py::SportProfile`; `sport_registry.register(...)` in `@backend/sports/__init__.py`; define prompt+HSV in new file |
270 | Add an AI provider | subclass `@backend/providers/base.py::AIVideoProvider`; `provider_registry.register('name', Cls)` |
271 | Add an annotation/GT tier | subclass `@backend/annotations/base.py::AnnotationProvider`; register + import in `@backend/annotations/__init__.py` |
272 | Add / change a report footgun rule | `@backend/reporting/spec.yaml::rules` (declarative; ? `when.path` must match harvest's stage/metric/detail names); facts in `@backend/reporting/harvest.py` |
273 | Tweak what the report records | `@backend/reporting/harvest.py::record_run` + stage builders; emit point is `@backend/main.py` `finally:` -> `@backend/reporting/__init__.py::emit_indexer_report` |
274 | Run calibration (WASB thresholds) | `@backend/eval/calibrate_wasb.py::main` (`--trajectory --labels --fps --frame-width`); trust `cv` recall not `fit` |

```
275     | Distill Gemini -> local (thresholds / learned) |
`@backend/eval/calibrate_local.py::main` (thresholds) OR `@backend/eval/distill_local.py::main`
(learned classifier); both `--manifest` |
276     | Refine WASB candidates with Gemini (tuning) | `@backend/eval/gemini_refine.py::main`
(`--full-video` skips `--trajectory`; needs `GEMINI_API_KEY`) |
277     | Export tuned rally segments / clips | `@backend/eval/export_wasb_segments.py::main`
(`--slice` needs `--video`) |
278     | Export timelines (EDL/JSON/CSV) + basic stats (Value Layer slice) |
`backend/tools/export.py` (segments_to_edl/json/csv, basic_stats); called from anywhere that has
play_segments (CLI, notebook, future API) |
279     | Run a video end-to-end | `@backend/main.py::main` (`--video-path` or `--server`) or
`@run_process_and_stitch.py::main` (edit hardcoded paths) |
280     | Score DB preds vs GT (one video) | `@run_eval.py::main` |
281     | Batch score a collector manifest | `@run_phase3_eval.py::main` (`--manifest`) |
282     | Guard a regression (CI) | `@run_regression.py::main` (`--video-id`); exit 1=regress,
2=missing DB; snapshot via `--update-baseline` |
283     | Run the whole test suite | `@run_all_tests.py` (repo root); or `pytest tests/`
directly |
284     | Resume a crashed WASB pass | nothing to do ?
`@backend/pipeline/detectors/wasb_infer.py::run` auto-resumes from
`det_raw.jsonl`+`manifest.json` next to `--out`; `--fresh` forces restart |
285     | Add a validator | subclass `@backend/pipeline/validators/base.py::VideoValidator`;
`registry.register(instance)` in `@backend/pipeline/validators/__init__.py` (mind order;
producers before consumers) |
286     | Add a DB column/table | `@backend/infrastructure/database.py::Database._init_db`
migration ladder + bump `PRAGMA user_version` (else version-assert tests fail) |
287     | Train the learned segmenter | `@backend/eval/training.py::build_training_set` ->
`@backend/eval/classifier.py::LogisticRegression.fit/save` |
288     | Probe an audio signal (diagnostic) | `@backend/eval/audio/e1_probe.py::main`; detector
in `@backend/pipeline/audio/hit_detector.py` |
289
290     ---
291
292     ## 8. TEST MAP (`tests/test_*.py` -> subsystem)
293     All pure-logic / mocked-boundary unit tests; no WSL/GPU/Gemini/ffmpeg actually invoked.
`@run_all_tests.py` (repo root) is the suite runner.
294     | Test | Covers |
295     |---|---|
296     | `test_base.py` | `pipeline/segmenters/base.SegmenterRegistry` (register/get/list via
DummySegmenter) |
297     | `test_yolo_hybrid.py` | `pipeline/segmenters/yolo_hybrid.HybridYoloSegmenter`
(torchvision+ByteTrack; `_PERSON_CLASS_ID` filter; candidate link; `store_yolo_candidates=False`
skip) |
298     | `test_tracknet_hybrid.py` |
`pipeline/segmenters/tracknet_hybrid.TrackNetHybridSegmenter` (4-phase; healthcheck-abort;
`_probe_fps_width` fallback) |
299     | `test_gemini.py` | `pipeline/segmenters/gemini.GeminiPlaySegmenter` (single vs chunked
path, per-chunk offset, `shot_count`<-`shuttle_exchange_count`) |
300     | `test_tracknet_runner.py` | `pipeline/detectors/tracknet_runner` (`to_wsl_mnt_path`,
`from_indexing_cfg`, parse + `trajectory_to_action_windows` shared brain, healthcheck,
run_predict, stage) |
301     | `test_rally_gate.py` | `pipeline/detectors/rally_gate` (axis/net estimate, crossing
count, gate kills single-toss) |
302     | `test_wasb_cache.py` | `pipeline/detectors/wasb_infer` resumable cache (serialize,
contiguity heal, manifest invalidation, atomic writes, `default_cache_dir`) |
303     | `test_eval_metrics.py` | `eval.metrics` IoU/matching/score/pr_curve |
304     | `test_eval_annotations.py` | `eval.annotations` GT CSV parse/validate,
`write_scaffold` |
305     | `test_eval_harness.py` | `eval.harness` DB-pred fetch + evaluate + report_to_dict;
also `@run_eval.py::main`/`get_file_md5` (exit codes, `--scaffold`/`--json`) |
306     | `test_eval_batch.py` | `eval.batch` path resolve / stage select / aggregate
(micro/macro, zero-div) |
307     | `test_calibration.py` | `eval.calibration`
pct_improvement/kfold/select_best/cross_validate/LOVO/improvement_report |
308     | `test_calibrate_wasb.py` | `eval.calibrate_wasb`
```

```
load_golden_gts/make_predict_fn/grid/run, BASELINE |
309     | `test_calibrate_local.py` | `eval.calibrate_local` local grid + crossing gate, leave-
one-video-out |
310     | `test_export_wasb_segments.py` | `eval.export_wasb_segments` segment/comparison CSV
schemas, mmss; export->slicer reuse |
311     | `test_eval_training.py` | `eval.training` label_candidates / feature extraction /
build_training_set |
312     | `test_eval_classifier.py` | `eval.classifier.LogisticRegression`
fit/predict_proba/balanced/JSON round-trip |
313     | `test_eval_regression.py` | `eval.regression` baseline store/compare/gate; also
`@run_regression.py::main` (exit-code CI gate) |
314     | `test_distill_local.py` | `eval.distill_local` build_candidates / predict_rallies /
learned-vs-baseline |
315     | `test_gemini_refine.py` | `eval.gemini_refine` merge_overlaps / _offset_segments (pure
helpers, no API) |
316     | `test_hit_detector.py` | `pipeline/audio/hit_detector` (detect_hits, onset_envelope,
_moving_stats) + `eval/audio/e1_probe.cluster_hits` |
317     | `test_rally_slicer.py` | `tools.rally_slicer.compute_clip_windows` (pure) + label
parse |
318     | `test_video.py` | `utils.video` duration/chunk (subprocess mocked) |
319     | `test_geometry.py` | `utils.geometry` center/distance/velocity/normalised-velocity/IoU
|
320     | `test_database.py` | `infrastructure.database.Database`
schema/CRUD/migrations/`user_version`/`EXPECTED_CANDIDATE_COLUMNS` guard |
321     | `test_config.py` | `config` (load_config defaults, save_default_config, AppConfig
defaults pinned, `extra` tolerance, `.get()` dict-compat) |
322     | `test_reporting_harvest.py` | `reporting` (`emit_indexer_report`,
`harvest.record_run`); ? `importorskip("obsreport")`; all 7 footgun rules + plain-dict-OR-
AppConfig + `report.enabled=False` |
323     | `test_reporting_spec_faq.py` | `reporting/spec.yaml` <-> `README.md` cross-ref
integrity (every `faq_ref` resolves; actionable rules cite a faq_ref); ?
`importorskip("obsreport")` |
324     | `test_validators_base.py` | `pipeline/validators/base.ValidationRegistry`
(register/run_all/`run_all_with_context` -> `(results,context)`; exception captured not raised)
|
325     | `test_sport_profiles.py` | `sports` registry + BadmintonSportProfile (prompt contains
"decimal seconds"/chunk value; relevance config keys) |
326     | `test_providers.py` / `test_provider_gemini.py` | `providers` registry +
GeminiProvider lifecycle (`MAX_UPLOAD_MB` guard, non-list JSON -> [], cleanup delete) |
327     | `test_annotation_providers.py` / `test_automated_provider.py` | `annotations`
registry, FileProvider, AutomatedProvider (oracle/lineage); keystone one-scorer contract ->
`eval.metrics.score` |
328     | (no dedicated test) | individual `pipeline/validators/*` impls,
`pipeline/stitcher.compiler`, `main.py`, `run_process_and_stitch.py`,
`pipeline/detectors/wasb_runner`, `pipeline/segmenters/{wasb_hybrid,motion}`,
`eval/audio/e1_probe` (diagnostic) ? ? exercised only indirectly |
329     | ? stale `.pyc` only | `tests/__pycache__/test_gemini_api.*` ? no live source exists |
330
```

4. user (2026-06-10T05:33:36.212Z)

```
1       import os
2       import sys
3       import argparse
4       import json
5       import logging
6       import uvicorn
7       from dotenv import load_dotenv
8
9       load_dotenv()
10
11      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
12      logging.basicConfig(
13          level=logging.INFO,
```

```
        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
        datefmt='%H:%M:%S'
    )
    logger = logging.getLogger(__name__)
    from fastapi import FastAPI, HTTPException
    from fastapi.middleware.cors import CORSMiddleware
    from pydantic import BaseModel
    from typing import List, Dict, Any, Optional

    from fastapi.staticfiles import StaticFiles
    from fastapi.responses import HTMLResponse
    from backend.pipeline.validators.base import registry
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
    from backend.pipeline.stitcher.compiler import VideoCompiler
    from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
    from backend.config import load_config as load_app_config, AppConfig  # new typed config
    from backend.results import Result, Failure  # standardized error/result contract
    from backend.reporting import emit_indexer_report

    app = FastAPI(title="Sports Highlight Indexer API")

    # Enable CORS for local web interface
    app.add_middleware(
        CORSMiddleware,
        allow_origins=["*"],
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )

    # Serves static frontend files directly (now under api/ per approved structure)
    os.makedirs("backend/api/static", exist_ok=True)
    app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")

    @app.get("/", response_class=HTMLResponse)
    def serve_index():
        index_path = "backend/api/static/index.html"
        if os.path.exists(index_path):
            with open(index_path, "r", encoding="utf-8") as f:
                return f.read()
        return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"

    # Global variables initialized at startup
    db_instance: Optional[Database] = None
    global_config: Optional[AppConfig] = None  # now typed (Pydantic model)

    # Legacy thin wrapper for any remaining direct calls during transition.
    # New code should import load_app_config / AppConfig from backend.config directly.
    def load_config(config_path: str = "config.json") -> Dict[str, Any]:
        cfg = load_app_config(config_path)
        return cfg.model_dump(mode="json")

    # --- API Models ---
    class ProcessRequest(BaseModel):
        video_path: str
        player_pool: Optional[List[str]] = None
        max_frames: Optional[int] = None
        segmenter: Optional[str] = None

    class QueryRequest(BaseModel):
        video_id: str
        server: Optional[str] = None
        receiver: Optional[str] = None
```

```
78          duration_min: Optional[float] = None
79          event_type: Optional[str] = None
80
81      class CompileRequest(BaseModel):
82          video_id: str
83          segment_ids: List[int]
84          output_filename: str
85
86      # --- API Endpoints ---
87      @app.get("/api/config")
88      def get_config():
89          return global_config
90
91      @app.post("/api/process")
92      def process_video(req: ProcessRequest):
93          if not os.path.exists(req.video_path):
94              raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
95
96          video_id = compute_video_id(req.video_path)
97          was_reingest = db_instance.get_video(video_id) is not None
98
99          # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
100         logger.info(f"Running validations for {req.video_path}...")
101         val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
102         failures = [r for r in val_results if not r.passed]
103
104         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
105         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
106         seg_name = req.segmenter or default_seg
107         segmenter_actual = None
108         segmentation_ran = False
109         response: Optional[Dict[str, Any]] = None
110         try:
111             if failures:
112                 db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in
val_results])
113                 response = {
114                     "video_id": video_id,
115                     "status": "failed",
116                     "message": "Video failed validation checks.",
117                     "results": [r.dict() for r in val_results],
118                 }
119                 return response
120
121             # 2. Run segmenter & indexer
122             print(f"[API] Video passed checks. Segmenting and indexing...")
123             db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
val_results])
124
125             segmenter_cls = segmenter_registry.get(seg_name)
126             if not segmenter_cls:
127                 available = list(segmenter_registry.list_available().keys())
128                 raise HTTPException(
129                     status_code=400,
130                     detail=f"Segmenter '{seg_name}' not found. Available segmenters:
{available}"
131                 )
132
133             print(f"[API] Using segmenter: {seg_name}")
134             segmenter_actual = seg_name
135             segmenter = segmenter_cls(db_instance, global_config)
```

```
            result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
            segmentation_ran = True

            if isinstance(result, Failure):
                print(f"[API] Segmenter failed: {result.message}")
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.dict() for r in val_results],
                    "play_segments": [],
                }
                return response

            segments = result.value if hasattr(result, "value") else result

            response = {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
                "results": [r.dict() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response.
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=was_reingest, segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths

    @app.post("/api/query")
    def query_play_segments(req: QueryRequest):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type
        }
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    @app.post("/api/compile")
    def compile_highlights(req: CompileRequest):
        video = db_instance.get_video(req.video_id)
        if not video:
            raise HTTPException(status_code=404, detail="Indexed video record not found.")

        # Retrieve matching play segments from db
        all_segments = db_instance.query_play_segments(req.video_id, {})
        matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]

        if not matched_segments:
            raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")

        # Extract start/end time pairs
```

```
        time_pairs = [(s["start_time"], s["end_time"]) for s in matched_segments]

        # Run compiler
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        compiler = VideoCompiler(output_dir)
        try:
            out_path = compiler.compile_highlights(video["filepath"], time_pairs,
req.output_filename)
            return {"status": "success", "filepath": out_path}
        except Exception as e:
            raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}")

    # --- CLI Debug Utility & Orchestration ---
    def run_cli_diagnose_clip(video_path: str, timestamp: float):
        """CLI debugging utility to examine segment properties around a timestamp."""
        print("=" * 60)
        print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
        print("=" * 60)

        cfg = load_config()  # legacy wrapper still works, returns dict for now

        # 1. Validation check diagnostics
        print("[DIAGNOSTIC] Running validation framework...")
        results = registry.run_all(video_path, cfg)
        for r in results:
            status_symbol = "[PASS]" if r.passed else "[FAIL]"
            print(f"  {status_symbol} {r.validator_name}: {r.message}")
            if r.details:
                print(f"         Details: {r.details}")

        # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
        print("-" * 60)
        print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
        import cv2
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            print("[FAIL] OpenCV could not open video.")
            return

        fps = cap.get(cv2.CAP_PROP_FPS)
        total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)

        start_frame = max(0, int((timestamp - 3.0) * fps))
        end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))

        # Measure frame-by-frame changes in sample region
        cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
        prev_gray = None
        motions = []

        for f in range(start_frame, end_frame):
            ret, frame = cap.read()
            if not ret:
                break
            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            gray = cv2.GaussianBlur(gray, (21, 21), 0)

            if prev_gray is not None:
                diff = cv2.absdiff(prev_gray, gray)
                _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
                motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
                motions.append(motion_ratio)
```

```
            prev_gray = gray

        cap.release()

        if motions:
            avg_motion = sum(motions) / len(motions)
            # Support both legacy dict (from wrapper) and future direct model
            if hasattr(cfg, "indexing"):
                threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
            else:
                threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)  # type:
ignore[attr-defined]
            print(f"  Sample Frame Count: {len(motions)}")
            print(f"  Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
            if avg_motion > threshold:
                print("  Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
            else:
                print("  Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
        else:
            print("  [ERROR] No visual frames were extracted in the sample range.")

        print("=" * 60)

    def main():
        parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
        parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
        parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
        parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
        parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
        parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
        parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
        parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")

        available_segmenters = list(segmenter_registry.list_available().keys())
        parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
        parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
        parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
        parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")

        args = parser.parse_args()

        if args.setup:
            # Invoke setup diagnostics
            import subprocess
            print("[INFO] Invoking setup.py...")
            subprocess.run([sys.executable, "setup.py"])
            return

        if args.trim_start is not None and args.trim_end is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to extract a segment.")
```

```
                return
            # Determine output path
            out_filename = args.trim_output or "trimmed_segment.mp4"
            if os.path.isabs(out_filename):
                out_path = out_filename
                out_dir = os.path.dirname(out_path)
                out_name = os.path.basename(out_path)
            else:
                out_dir = args.output_dir
                out_path = os.path.join(out_dir, out_filename)
                out_name = out_filename

            os.makedirs(out_dir, exist_ok=True)
            print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")

            compiler = VideoCompiler(out_dir)
            try:
                res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
                print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
            except Exception as e:
                print(f"[ERROR] Failed to compile trimmed segment: {e}")
            return

        if args.debug_clip is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to debug a specific clip.")
                return
            run_cli_diagnose_clip(args.video_path, args.debug_clip)
            return

        # Initialize Global Config and DB
        global global_config, db_instance
        global_config = load_app_config()  # returns typed AppConfig

        # The CLI --output-dir overrides the configured output directory. It now lives
        # on the typed model (OutputConfig.output_dir), so every consumer ? including
        # /api/compile ? reads the same value instead of a write-only runtime hack.
        global_config.output.output_dir = args.output_dir
        os.makedirs(global_config.output.output_dir, exist_ok=True)

        db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
        db_instance = Database(db_path)

        # CLI processing mode (no web UI needed)
        if args.video_path and not args.server:
            print(f"[CLI] Processing video file: {args.video_path}")
            if not os.path.exists(args.video_path):
                print(f"[FAIL] Video file not found: {args.video_path}")
                return

            video_id = compute_video_id(args.video_path)
            was_reingest = db_instance.get_video(video_id) is not None

            # Validate (with-context variant surfaces ffprobe_meta for the report)
            print("[CLI] Validating...")
            val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
            failures = [r for r in val_results if not r.passed]

            # Save video status
            status = "failed" if failures else "processed"
```

```
            db_instance.add_video(video_id, args.video_path, status, [r.dict() for r in
val_results])

            seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
            segmenter_actual = None
            segmentation_ran = False
            try:
                print("\n" + "="*40)
                print("VALIDATION SUMMARY")
                print("="*40)
                for r in val_results:
                    status_symbol = "[PASS]" if r.passed else "[FAIL]"
                    print(f"{status_symbol} {r.validator_name}: {r.message}")
                print("="*40 + "\n")

                if failures:
                    print("[FAIL] Video failed checks. Indexing halted.")
                    return

                # Index
                segmenter_cls = segmenter_registry.get(seg_name)
                if not segmenter_cls:
                    print(f"[FAIL] Segmenter '{seg_name}' not found.")
                    return

                print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
                segmenter_actual = seg_name
                segmenter = segmenter_cls(db_instance, global_config)
                result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
                segmentation_ran = True

                if isinstance(result, Failure):
                    print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
                    print(f"[FAIL] Indexing halted for video.")
                    segments = []
                else:
                    segments = result.value if hasattr(result, "value") else result
                    print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
            finally:
                # Always emit the per-video observability report. Never raises.
                paths = emit_indexer_report(
                    db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
                    val_results=val_results, val_context=val_context,
                    segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
                    was_reingest=was_reingest, segmentation_ran=segmentation_ran,
                )
                if paths:
                    print(f"[REPORT] Technical report : {paths.get('technical_md')}")
                    print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
                    print(f"[REPORT] Machine JSON     : {paths.get('json')}")
            return

        # Start Local Server Mode
        if args.server or not args.video_path:
            print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
            uvicorn.run(app, host="127.0.0.1", port=args.port)

    if __name__ == "__main__":
        main()

```

5. user (2026-06-10T05:33:36.711Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, Literal
12
13      from pydantic import BaseModel, Field, field_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33      class TrackNetConfig(BaseModel):
34          wsl_distro: str = "Ubuntu"
35          repo_dir: str = "~/models/TrackNetV4"
36          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37          conda_env: str = "TrackNetV4"
38          weights_path: str = ""
39          queue_length: int = 5
40          stage_in_wsl: bool = True
41          wsl_stage_dir: str = "~/clips"
42          timeout_sec: int = 0
43          velocity_threshold: float = 0.02
44
45
46      class IndexingConfig(BaseModel):
47          default_segmenter: str = "yolo_hybrid"
48          use_gpu: bool = True
49          motion_threshold: float = 0.08
50          min_segment_duration: float = 3.0
51          dead_time_merge_gap: float = 2.0
52          chunk_duration: float = 90.0
53          chunk_overlap: float = 10.0
54          detector_model: str = "fasterrcnn_resnet50_fpn"
55          detector_score_threshold: float = 0.5
56          yolo_velocity_threshold: float = 0.15
57          yolo_frame_skip: int = 3
58          yolo_tracker: str = "bytetrack.yaml"
59          yolo_parallel_chunks: bool = False
60          yolo_chunk_workers: int = 2
61          min_action_window_duration: float = 2.0
62          log_yolo_candidates: bool = True
```

```
63          store_yolo_candidates: bool = True
64          ai_handoff_padding: float = 2.0
65          ai_max_workers: int = 5
66
67          tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
68
69          @field_validator("default_segmenter")
70          @classmethod
71          def segmenter_must_be_registered(cls, v: str) -> str:
72              # Registry check happens at runtime in main/segmenter selection.
73              # Here we just allow any string for forward compatibility.
74              return v
75
76
77      class OutputConfig(BaseModel):
78          default_highlights_name: str = "highlights.mp4"
79          db_name: str = "sports_indexer.db"
80          # Directory where the DB, highlight reels, and processed clips are written.
81          # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
82          output_dir: str = "output"
83
84
85      class StitchingConfig(BaseModel):
86          begin_padding: float = 0.5
87          end_padding: float = 0.5
88          use_cache: bool = False
89          min_shot_count: int = 1
90
91
92      class AppConfig(BaseModel):
93          """Root configuration object.
94
95          All runtime configuration should be accessed via an instance of this class
96          (or the loader) rather than raw dict lookups.
97          """
98
99          sport: Sport = "badminton"
100         ai_provider: str = "gemini"
101         ai_model: str = "gemini-2.5-flash"
102
103         validation: ValidationConfig = Field(default_factory=ValidationConfig)
104         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
105         output: OutputConfig = Field(default_factory=OutputConfig)
106         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
107
108         # Allow extra keys during transition so existing config.json files do not break.
109         model_config = {
110             "extra": "allow",
111             "validate_assignment": True,
112         }
113
114         # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
115         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
116         # attribute access / the typed config. A nested *section* is returned as a plain
dict
117         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
118         # section model instead breaks it (a model has no .get), which bit every validator
on
119         # real runs. Scalar leaves are returned as-is.
120         def get(self, key: str, default: Any = None) -> Any:
121             _MISSING = object()
122             val: Any = _MISSING
123             if key in type(self).model_fields or hasattr(self, key):
124                 val = getattr(self, key, _MISSING)
```

```
125             if val is _MISSING:
126                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
127                 # discovered dynamically from the model fields, so a future top-level
section
128                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
129                 for fname in type(self).model_fields:
130                     section = getattr(self, fname, None)
131                     if isinstance(section, BaseModel) and hasattr(section, key):
132                         val = getattr(section, key)
133                         break
134             if val is _MISSING:
135                 return default
136             if isinstance(val, BaseModel):
137                 return val.model_dump(mode="json")
138             return val
139 
140         def __getitem__(self, key: str) -> Any:
141             val = self.get(key)
142             if val is None and key not in type(self).model_fields:
143                 # Re-raise like a real dict for unknown top-level
144                 raise KeyError(key)
145             return val
146 
147 
148     # Convenience type alias for callers that want to annotate
149     Config = AppConfig
```

6. user (2026-06-10T05:34:16.033Z)

```
1       """Configuration loader and saver.
2 
3       Provides a single place to load validated AppConfig with built-in defaults
4       merged with optional file overrides (config.json).
5 
6       This replaces scattered json.load + .get() patterns throughout the codebase.
7       """
8 
9       from __future__ import annotations
10 
11      import json
12      import logging
13      from pathlib import Path
14      from typing import Any
15 
16      from .models import AppConfig
17 
18      logger = logging.getLogger(__name__)
19 
20      DEFAULT_CONFIG_PATH = Path("config.json")
21 
22 
23      def _get_builtin_defaults() -> dict[str, Any]:
24          """Return a plain dict of the current built-in defaults.
25 
26          The AppConfig model is the single source of truth for defaults.
27          """
28          return AppConfig().model_dump(mode="json")
29 
30 
31      def load_config(path: str | Path | None = None) -> AppConfig:
32          """Load configuration with the following precedence:
33 
34          1. Built-in defaults defined in the Pydantic models.
```

```
35          2. Values from the JSON file (if present and readable).
36          3. (Future) environment / CLI overrides.
37
38          Missing file or unreadable file ? returns a fully defaulted AppConfig.
39          Unknown keys in the file are tolerated (extra="allow" on the model).
40          """
41          cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
42
43          file_data: dict[str, Any] = {}
44          if cfg_path.exists():
45              try:
46                  with open(cfg_path, "r", encoding="utf-8") as f:
47                      file_data = json.load(f)
48              except Exception as exc:
49                  # Non-fatal: continue with defaults + whatever we could parse.
50                  # In production we might log here.
51                  logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
52
53          # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
54          merged = {**_get_builtin_defaults(), **file_data}
55
56          return AppConfig.model_validate(merged)
57
58
59      def save_default_config(path: str | Path | None = None) -> Path:
60          """Write a fresh default config.json.
61
62          Used by setup.py diagnostics and the --setup CLI path.
63          Overwrites if the file exists.
64          """
65          cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
66          cfg = AppConfig()
67          cfg_path.parent.mkdir(parents=True, exist_ok=True)
68          cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
69          return cfg_path
70
71
72      # Backwards-compatible alias used by some older call sites during transition.
73      def get_default_config() -> dict[str, Any]:
74          """Return defaults as a plain dict (for transitional use only)."""
75          return _get_builtin_defaults()
76
```

7. user (2026-06-10T05:34:16.478Z)

```
1       """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3       This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5       The goal is to make it impossible to confuse "no data / no rallies" with "something
actually failed"
6       (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
7
8       We use simple dataclasses so callers can use isinstance checks or structural pattern
matching.
9       This is intentionally small and incremental ? we start by applying it to the segmenter
10      and provider layers (the biggest sources of silent [] returns today).
11
12      The types are generic where it makes sense. Callers can still access .value on Success
13      or .message on Failure.
14
15      This module has no dependencies on the rest of the app.
16      """
17
```

```
    from __future__ import annotations

    from dataclasses import dataclass, field
    from typing import Generic, TypeVar, List, Optional, Any, Union

    T = TypeVar("T")


    @dataclass
    class Success(Generic[T]):
        """Represents a successful outcome with an optional list of non-fatal warnings."""

        value: T
        warnings: List[str] = field(default_factory=list)

        def is_success(self) -> bool:
            return True

        def unwrap(self) -> T:
            """Return the value. Only safe to call if you already know it's a Success."""
            return self.value


    @dataclass
    class Failure:
        """Represents a failure with a human-readable message and optional details."""

        message: str
        details: Optional[dict[str, Any]] = None
        is_recoverable: bool = False  # e.g. missing API key (recoverable) vs internal crash

        def is_success(self) -> bool:
            return False

        def unwrap(self) -> Any:
            """Raise to make misuse obvious in code that assumes success."""
            raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")


    # Convenient alias for use in type hints: Result[list[dict], Failure]
    # Result[ValueType] means Success[ValueType] | Failure
    # (the Failure "error" type is fixed for this contract)
    Result = Union[Success[T], Failure]

```

8. user (2026-06-10T05:34:16.825Z)

```
    {
        "sport": "badminton",
        "ai_provider": "gemini",
        "ai_model": "gemini-2.5-flash",
        "validation": {
            "min_resolution_width": 1280,
            "min_resolution_height": 720,
            "min_fps": 29.9,
            "min_blur_threshold": 20.0,
            "max_scene_cuts_per_minute": 0.5,
            "relevance": {
                "court_green_lower": [
                    35,
                    40,
                    40
                ],
                "court_green_upper": [
                    85,
```

```
                    255,
                    255
                ],
                "court_pixels_min_ratio": 0.05
            }
        },
        "indexing": {
            "default_segmenter": "gemini",
            "use_gpu": true,
            "motion_threshold": 0.08,
            "min_segment_duration": 3.0,
            "dead_time_merge_gap": 2.0,
            "chunk_duration": 90,
            "chunk_overlap": 10,
            "detector_model": "fasterrcnn_resnet50_fpn",
            "detector_score_threshold": 0.5,
            "yolo_velocity_threshold": 0.15,
            "yolo_frame_skip": 3,
            "yolo_tracker": "bytetrack.yaml",
            "yolo_parallel_chunks": false,
            "yolo_chunk_workers": 2,
            "min_action_window_duration": 2.0,
            "log_yolo_candidates": true,
            "store_yolo_candidates": true,
            "ai_handoff_padding": 2.0,
            "ai_max_workers": 5,
            "tracknet": {
                "wsl_distro": "Ubuntu",
                "repo_dir": "~/models/TrackNetV4",
                "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
                "conda_env": "TrackNetV4",
                "weights_path": "",
                "queue_length": 5,
                "stage_in_wsl": true,
                "wsl_stage_dir": "~/clips",
                "timeout_sec": 0,
                "velocity_threshold": 0.02
            }
        },
        "output": {
            "default_highlights_name": "highlights.mp4",
            "db_name": "sports_indexer.db"
        },
        "stitching": {
            "begin_padding": 0.5,
            "end_padding": 0.5,
            "use_cache": false,
            "min_shot_count": 1
        }
    }
```

9. assistant (2026-06-10T05:34:22.579Z)

Now the segmenters ? base and all 5.

10. user (2026-06-10T05:34:23.571Z)

```
      from abc import ABC, abstractmethod
      from typing import List, Dict, Any, Type, Optional
      from backend.infrastructure.database import Database
      from backend.results import Result  # new standardized contract (Result[Value] ==
Success[Value] | Failure)

      class BasePlaySegmenter(ABC):
          def __init__(self, db: Database, config: Dict[str, Any]):
```

```
8               self.db = db
9               self.config = config
10
11          @property
12          @abstractmethod
13          def name(self) -> str:
14              """Unique identifier for this segmenter."""
15              pass
16
17          @property
18          @abstractmethod
19          def description(self) -> str:
20              """Human-readable explanation of this segmenter."""
21              pass
22
23          @abstractmethod
24          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25              """
26              Processes a video to detect play segments, populates SQLite database,
27              and returns a Result.
28
29              Use isinstance(result, Failure) (or result.is_success()) to distinguish
30              "no segments found" from "something went wrong" (missing key, healthcheck
31              failure, etc.). This is the start of the standardized error/result contract.
32              """
33              pass
34
35      class SegmenterRegistry:
36          def __init__(self):
37              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40              """Registers a new segmenter class under its name property."""
41              try:
42                  temp_inst = segmenter_cls(None, {})
43                  name = temp_inst.name
44              except Exception as e:
45                  raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}")
46              self._segmenters[name] = segmenter_cls
47
48          def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49              return self._segmenters.get(name)
50
51          def list_available(self) -> Dict[str, str]:
52              """Returns a dict of registered segmenter names and their descriptions."""
53              res = {}
54              for name, cls in self._segmenters.items():
55                  try:
56                      temp_inst = cls(None, {})
57                      desc = temp_inst.description
58                  except Exception:
59                      desc = "No description available."
60                  res[name] = desc
61              return res
62
63      segmenter_registry = SegmenterRegistry()
64
```

11. user (2026-06-10T05:34:23.970Z)

```
1       """WASB-based hybrid play segmenter.
2
3       Identical three-phase shape to TrackNetHybridSegmenter, but Phase 2 uses **WASB**
```

```
4       (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via WSL2
5       (backend/detectors/wasb_runner.py) instead of TrackNetV4. WASB ships pretrained
6       badminton weights and runs today, so this is the live shuttle-motion substrate.
7       Phase 3 (AI semantic boundaries) and Phase 4 (DB) are shared with the other hybrids.
8
9       For the per-video *calibration* goal we mainly use Phase 2 (trajectory -> candidate
10      windows); the windowing thresholds (velocity_thresh / merge_gap / min_window_duration)
11      are what get tuned against golden rally labels.
12      """
13      import os
14      import time
15      import logging
16      import concurrent.futures
17      from typing import List, Dict, Any
18
19      import cv2
20
21      from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
22      from backend.utils.video import get_video_duration, extract_video_chunk
23      from backend.sports import sport_registry
24      from backend.providers import provider_registry
25      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
26      from backend.pipeline.detectors.base import DetectorRunner  # ABC for the de-risked
detector seam
27
28      logger = logging.getLogger(__name__)
29
30
31      def _fmt_ts(seconds: float) -> str:
32          seconds = max(0, int(seconds))
33          h, rem = divmod(seconds, 3600)
34          m, s = divmod(rem, 60)
35          return f"{h:02d}:{m:02d}:{s:02d}"
36
37
38      class WasbHybridSegmenter(BasePlaySegmenter):
39          @property
40          def name(self) -> str:
41              return "wasb_hybrid"
42
43          @property
44          def description(self) -> str:
45              return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
46                      "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
47
48          @staticmethod
49          def _probe_fps_width(video_path: str) -> tuple:
50              cap = cv2.VideoCapture(video_path)
51              fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
52              width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
53              cap.release()
54              return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
55
56          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None,
57                            max_frames: int = None) -> List[Dict[str, Any]]:
58              sport_name = self.config.get("sport", "badminton")
59              try:
60                  sport_profile = sport_registry.get(sport_name)
61              except KeyError as e:
62                  logger.error(str(e))
63                  return []
64
```

```
            idx_cfg = self.config.get("indexing", {})
            provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
            model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))

            api_key_env_var = f"{provider_name.upper()}_API_KEY"
            api_key = os.environ.get(api_key_env_var)
            if not api_key:
                logger.error(f"{api_key_env_var} not set; cannot run the {provider_name}
handoff.")
                return []
            try:
                provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                return []
            fps, frame_width = self._probe_fps_width(video_path)

            wasb_cfg = WasbConfig.from_indexing_cfg(idx_cfg)
            runner: DetectorRunner = WasbRunner(wasb_cfg)

            w = idx_cfg.get("wasb", {})
            velocity_thresh = w.get("velocity_threshold", 0.02)
            merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
            min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
            handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
            ai_max_workers = idx_cfg.get("ai_max_workers", 5)
            log_candidates = idx_cfg.get("log_yolo_candidates", True)
            store_candidates = idx_cfg.get("store_yolo_candidates", True)

            detection_params = {
                "detector": "wasb_hybrid",
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "min_action_window_duration": min_window_duration,
                "weights_path": wasb_cfg.weights_path,
            }

            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("WASB WSL environment not ready: %s", msg)
                return []
            logger.info("WASB WSL env OK: %s", msg)

            # --- Phase 2: shuttle trajectory -> candidate rally windows ---
            t_start = time.time()
            out_dir = os.path.join("output", "wasb")
            csv_path = runner.run_predict(video_path, out_dir)
            if not csv_path:
                logger.error("WASB produced no trajectory; aborting.")
                return []

            points = runner.parse_trajectory_csv(csv_path)
            logger.info("Parsed %d trajectory points (%d visible).",
                        len(points), sum(1 for p in points if p.visible))

            all_candidate_windows = runner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width, chunk_start=0.0,
```

```
126                 velocity_thresh=velocity_thresh, merge_gap=merge_gap,
127                 min_window_duration=min_window_duration, chunk_index=0,
128             )
129             logger.info("Phase 2 (WASB) completed in %.1fs. Candidate windows: %d",
130                         time.time() - t_start, len(all_candidate_windows))
131
132             if log_candidates:
133                 for cw in all_candidate_windows:
134                     logger.info(f"[WASB rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
135                                 f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
136                                 f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")
137
138             candidate_ids = [None] * len(all_candidate_windows)
139             if store_candidates:
140                 for i, cw in enumerate(all_candidate_windows):
141                     stats = {
142                         "peak_velocity": cw["peak_velocity"], "mean_velocity":
cw["mean_velocity"],
143                         "active_frame_count": cw["active_frame_count"], "track_id_count":
cw["track_id_count"],
144                     }
145                     candidate_ids[i] = self.db.add_candidate_segment(
146                         video_id, detector="wasb_hybrid", start_time=cw["start"],
end_time=cw["end"],
147                         chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
148                         stats=stats, detection_params=detection_params,
149                     )
150
151             if not all_candidate_windows:
152                 return []
153
154             # --- Phase 3: AI provider handoff (shared pattern) ---
155             handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
156             os.makedirs(handoff_dir, exist_ok=True)
157             logger.info("Phase 3: %s validation on %d candidate windows...",
158                         provider_name, len(all_candidate_windows))
159
160             def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
161                 w_start = max(0, window["start"] - handoff_padding)
162                 w_end = min(video_duration, window["end"] + handoff_padding)
163                 w_dur = w_end - w_start
164                 w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
165                 if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
166                     return []
167                 prompt = sport_profile.get_prompt(chunk_duration=w_dur)
168                 segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
169                                                      label=f"Handoff {wi+1}")
170                 valid = []
171                 for seg in segments:
172                     try:
173                         seg["start_time"] = float(seg["start_time"]) + w_start
174                         seg["end_time"] = float(seg["end_time"]) + w_start
175                         seg["_candidate_id"] = candidate_ids[wi]
176                         valid.append(seg)
177                     except (ValueError, TypeError, KeyError) as e:
178                         logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
179                 try:
180                     os.remove(w_path)
181                 except OSError:
182                     pass
```

```
183                 return valid
184
185             ai_segments: List[dict] = []
186             with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
187                 futures = [executor.submit(process_candidate_window, wi, cw)
188                            for wi, cw in enumerate(all_candidate_windows)]
189                 for future in futures:
190                     ai_segments.extend(future.result())
191             try:
192                 os.rmdir(handoff_dir)
193             except OSError:
194                 pass
195             logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
196
197             # --- Phase 4: DB population (shared pattern) ---
198             segments_data = []
199             for segment in ai_segments:
200                 try:
201                     start = float(segment["start_time"])
202                     end = float(segment["end_time"])
203                     dur = end - start
204                     shot_count = segment.get("shuttle_exchange_count")
205                     try:
206                         shot_count = int(shot_count)
207                     except (ValueError, TypeError):
208                         shot_count = max(3, int(dur / 0.8))
209                     metadata = {"shot_count": shot_count,
210                                 "shot_count_confidence":
segment.get("shot_count_confidence", "Unknown"),
211                                 "detector": f"wasb-hybrid-{model_name}"}
212                     segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
213                     if segment_id:
214                         candidate_id = segment.get("_candidate_id")
215                         if candidate_id is not None:
216                             self.db.link_candidate_to_segment(candidate_id, segment_id)
217                         segments_data.append({"id": segment_id, "start_time": start,
"end_time": end,
218                                               "duration": dur, "metadata": metadata})
219                 except Exception as e:
220                     logger.warning("Skipping invalid segment struct: %s", e)
221
222             return segments_data
223
224
225     segmenter_registry.register(WasbHybridSegmenter)
226
```

12. user (2026-06-10T05:34:42.285Z)

```
1       """TrackNet-based hybrid play segmenter.
2
3       Same three-phase shape as HybridYoloSegmenter, but Phase 2 (candidate
4       generation) uses a TrackNetV4 *shuttle* trajectory instead of YOLO *person*
5       motion. TrackNet runs in WSL2 (see backend/detectors/tracknet_runner.py and
6       docs/DECISIONS.md ADR-001); the shuttle trajectory is converted into candidate
7       rally windows, which are then handed to the AI provider for semantic boundary
8       confirmation (Phase 3) and written to the DB (Phase 4).
9
10      Rationale: a tiny, fast, motion-blurred shuttle is exactly what general
11      detectors miss; a purpose-built trajectory model localises it far more
12      precisely, giving cleaner candidate rallies than person-velocity heuristics.
13      """
```

```
14
15      import os
16      import time
17      import logging
18      import concurrent.futures
19      from typing import List, Dict, Any
20
21      import cv2
22
23      from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
24      from backend.utils.video import get_video_duration, extract_video_chunk
25      from backend.sports import sport_registry
26      from backend.providers import provider_registry
27      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
28      from backend.pipeline.detectors.base import DetectorRunner  # ABC for the de-risked
detector seam
29
30      logger = logging.getLogger(__name__)
31
32
33      def _fmt_ts(seconds: float) -> str:
34          seconds = max(0, int(seconds))
35          h, rem = divmod(seconds, 3600)
36          m, s = divmod(rem, 60)
37          return f"{h:02d}:{m:02d}:{s:02d}"
38
39
40      class TrackNetHybridSegmenter(BasePlaySegmenter):
41          @property
42          def name(self) -> str:
43              return "tracknet_hybrid"
44
45          @property
46          def description(self) -> str:
47              return ("Two-stage pipeline: TrackNetV4 shuttle-trajectory detection (WSL2) for
precise "
48                      "candidate rallies + AI provider for high-precision semantic
boundaries.")
49
50          @staticmethod
51          def _probe_fps_width(video_path: str) -> tuple:
52              """Return (fps, frame_width) for a video, with sensible fallbacks."""
53              cap = cv2.VideoCapture(video_path)
54              fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
55              width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
56              cap.release()
57              return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
58
59          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None,
60                            max_frames: int = None) -> List[Dict[str, Any]]:
61              # --- Sport profile + AI provider wiring (identical to the other segmenters) ---
62              sport_name = self.config.get("sport", "badminton")
63              try:
64                  sport_profile = sport_registry.get(sport_name)
65              except KeyError as e:
66                  logger.error(str(e))
67                  return []
68
69              idx_cfg = self.config.get("indexing", {})
70              provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
71              model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
72
```

```
            api_key_env_var = f"{provider_name.upper()}_API_KEY"
            api_key = os.environ.get(api_key_env_var)
            if not api_key:
                logger.error(
                    f"{api_key_env_var} environment variable is not set! "
                    f"To run the {provider_name} handoff, set your API key. "
                    f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
                )
                return []
            try:
                provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                return []
            fps, frame_width = self._probe_fps_width(video_path)

            # --- TrackNet runner config + thresholds ---
            tn_cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
            runner: DetectorRunner = TrackNetRunner(tn_cfg)

            tn = idx_cfg.get("tracknet", {})
            velocity_thresh = tn.get("velocity_threshold", 0.02)   # normalised shuttle
displacement/sec
            merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
            min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
            handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
            ai_max_workers = idx_cfg.get("ai_max_workers", 5)
            log_candidates = idx_cfg.get("log_yolo_candidates", True)
            store_candidates = idx_cfg.get("store_yolo_candidates", True)

            detection_params = {
                "detector": "tracknet_hybrid",
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "min_action_window_duration": min_window_duration,
                "weights_path": tn_cfg.weights_path,
                "queue_length": tn_cfg.queue_length,
            }

            # --- Phase 1: env healthcheck (fail fast with a clear message) ---
            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("TrackNet WSL environment not ready: %s", msg)
                return []
            logger.info("TrackNet WSL env OK: %s", msg)

            # --- Phase 2: shuttle trajectory -> candidate rally windows ---
            # TrackNet processes the whole video in one pass (it carries its own
            # frame buffering), so unlike the YOLO segmenter we don't pre-chunk here.
            t_start = time.time()
            tn_out_dir = os.path.join("output", "tracknet")
            csv_path = runner.run_predict(video_path, tn_out_dir)
            if not csv_path:
                logger.error("TrackNet produced no trajectory; aborting.")
                return []

            points = runner.parse_trajectory_csv(csv_path)
            logger.info("Parsed %d trajectory points (%d visible).",
                        len(points), sum(1 for p in points if p.visible))
```

```
136
137             all_candidate_windows = runner.trajectory_to_action_windows(
138                 points, fps=fps, frame_width=frame_width, chunk_start=0.0,
139                 velocity_thresh=velocity_thresh, merge_gap=merge_gap,
140                 min_window_duration=min_window_duration, chunk_index=0,
141             )
142             logger.info("Phase 2 (TrackNet) completed in %.1fs. Candidate windows: %d",
143                         time.time() - t_start, len(all_candidate_windows))
144
145             if log_candidates:
146                 for w in all_candidate_windows:
147                     logger.info(
148                         f"[TrackNet rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
149                         f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']}
"
150                         f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f}"
151                     )
152
153             # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
154             candidate_ids = [None] * len(all_candidate_windows)
155             if store_candidates:
156                 for i, w in enumerate(all_candidate_windows):
157                     stats = {
158                         "peak_velocity": w["peak_velocity"],
159                         "mean_velocity": w["mean_velocity"],
160                         "active_frame_count": w["active_frame_count"],
161                         "track_id_count": w["track_id_count"],
162                     }
163                     confidence = min(1.0, w["peak_velocity"])
164                     candidate_ids[i] = self.db.add_candidate_segment(
165                         video_id, detector="tracknet_hybrid",
166                         start_time=w["start"], end_time=w["end"],
167                         chunk_index=w["chunk_index"], confidence=confidence,
168                         stats=stats, detection_params=detection_params,
169                     )
170
171             if not all_candidate_windows:
172                 return []
173
174             # --- Phase 3: AI provider handoff (identical pattern to HybridYoloSegmenter)
---
175             handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
176             os.makedirs(handoff_dir, exist_ok=True)
177             logger.info("Phase 3: %s validation on %d candidate windows...",
178                         provider_name, len(all_candidate_windows))
179
180             def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
181                 w_start = max(0, window["start"] - handoff_padding)
182                 w_end = min(video_duration, window["end"] + handoff_padding)
183                 w_dur = w_end - w_start
184                 w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
185                 if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
186                     return []
187                 prompt = sport_profile.get_prompt(chunk_duration=w_dur)
188                 segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
189                                                      label=f"Handoff {wi+1}")
190                 valid = []
191                 for seg in segments:
192                     try:
193                         seg["start_time"] = float(seg["start_time"]) + w_start
194                         seg["end_time"] = float(seg["end_time"]) + w_start
195                         seg["_candidate_id"] = candidate_ids[wi]
196                         valid.append(seg)
197                     except (ValueError, TypeError, KeyError) as e:
```

```
                        logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
                try:
                    os.remove(w_path)
                except OSError:
                    pass
                return valid

            ai_segments: List[dict] = []
            with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
                futures = [executor.submit(process_candidate_window, wi, w)
                           for wi, w in enumerate(all_candidate_windows)]
                for future in futures:
                    ai_segments.extend(future.result())

            try:
                os.rmdir(handoff_dir)
            except OSError:
                pass
            logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))

            # --- Phase 4: DB population (identical pattern to HybridYoloSegmenter) ---
            segments_data = []
            for segment in ai_segments:
                try:
                    start = float(segment["start_time"])
                    end = float(segment["end_time"])
                    dur = end - start
                    shot_count = segment.get("shuttle_exchange_count")
                    if shot_count is None:
                        shot_count = max(3, int(dur / 0.8))
                    else:
                        try:
                            shot_count = int(shot_count)
                        except (ValueError, TypeError):
                            shot_count = max(3, int(dur / 0.8))
                    metadata = {
                        "shot_count": shot_count,
                        "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
                        "detector": f"tracknet-hybrid-{model_name}",
                    }
                    segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
                    if segment_id:
                        candidate_id = segment.get("_candidate_id")
                        if candidate_id is not None:
                            self.db.link_candidate_to_segment(candidate_id, segment_id)
                        segments_data.append({
                            "id": segment_id, "start_time": start, "end_time": end,
                            "duration": dur, "metadata": metadata,
                        })
                except Exception as e:
                    logger.warning("Skipping invalid segment struct: %s", e)

            return segments_data


    # Register the segmenter
    segmenter_registry.register(TrackNetHybridSegmenter)

```

13. user (2026-06-10T05:34:42.373Z)

```
import os
import time
import json
import logging
import subprocess
import concurrent.futures
from typing import List, Dict, Any, Tuple
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
from backend.utils.video import get_video_duration, extract_video_chunk
from backend.sports import sport_registry
from backend.providers import provider_registry

logger = logging.getLogger(__name__)

class GeminiPlaySegmenter(BasePlaySegmenter):
    @property
    def name(self) -> str:
        return "gemini"

    @property
    def description(self) -> str:
        return "High-precision AI-powered semantic play segment detection using Google
Gemini Multimodal API."

    def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
        """
        Processes a video to detect play segments using Google Gemini Multimodal Video
API,
        populates SQLite, and indexes detailed events.
        """
        # Get sport profile config
        sport_name = self.config.get("sport", "badminton")
        try:
            sport_profile = sport_registry.get(sport_name)
        except KeyError as e:
            logger.error(str(e))
            return []

        # Get AI provider and model config
        idx_cfg = self.config.get("indexing", {})
        provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
        model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))

        # Get appropriate API key based on the provider
        api_key_env_var = f"{provider_name.upper()}_API_KEY"
        api_key = os.environ.get(api_key_env_var)
        if not api_key:
            logger.error(
                f"{api_key_env_var} environment variable is not set! "
                f"To run the {provider_name} segmenter, set your API key. "
                f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
            )
            return []

        try:
            provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
        except KeyError as e:
            logger.error(str(e))
            return []

```

```
            # A: Check for debug_duration to trim the video first
            debug_duration = self.config.get("indexing", {}).get("debug_duration")
            video_to_upload = video_path
            is_trimmed = False
            temp_trimmed_path = None
            is_compressed = False
            temp_compressed_path = None

            if debug_duration:
                logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
                temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
                os.makedirs("output", exist_ok=True)
                if os.path.exists(temp_trimmed_path):
                    try:
                        os.remove(temp_trimmed_path)
                    except Exception:
                        pass

                cmd = [
                    "ffmpeg", "-y",
                    "-ss", "0",
                    "-t", str(debug_duration),
                    "-i", video_path,
                    "-c", "copy",
                    temp_trimmed_path
                ]
                logger.debug(f"Running command: {' '.join(cmd)}")
                try:
                    subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
                    logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
                    video_to_upload = temp_trimmed_path
                    is_trimmed = True
                except Exception as e:
                    logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to
full video.")



            # Helper to clean up local temp files
            def cleanup_temp_files():
                if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
                    try:
                        os.remove(temp_trimmed_path)
                    except Exception:
                        pass

            # 1. Determine actual video duration
            video_duration = get_video_duration(video_to_upload)
            if video_duration > 0:
                logger.info(f"Detected video duration: {video_duration:.1f}s")
            else:
                logger.warning("Could not detect video duration. Prompt will not include
duration context.")

            # 2. Decide whether to use chunked processing
            idx_cfg = self.config.get("indexing", {})
            chunk_duration = idx_cfg.get("chunk_duration", 300)    # 5 minutes default
            chunk_overlap = idx_cfg.get("chunk_overlap", 30)       # 30 seconds overlap
            use_chunking = video_duration > 0 and video_duration > chunk_duration

            ai_segments = []
            chunks_dir = os.path.join("output", "chunks")
```

```
120
121             if use_chunking:
122                 # --- CHUNKED PROCESSING ---
123                 step = chunk_duration - chunk_overlap
124                 chunk_starts = []
125                 t = 0.0
126                 while t < video_duration:
127                     chunk_starts.append(t)
128                     t += step
129                 num_chunks = len(chunk_starts)
130                 logger.info(
131                     f"Video is {video_duration:.1f}s ? splitting into {num_chunks}
overlapping chunks "
132                     f"({chunk_duration}s each, {chunk_overlap}s overlap)."
133                 )
134
135                 os.makedirs(chunks_dir, exist_ok=True)
136
137                 def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list,
dict]:
138                     chunk_end = min(chunk_start + chunk_duration, video_duration)
139                     actual_chunk_dur = chunk_end - chunk_start
140                     chunk_label = f"Chunk {ci+1}/{num_chunks}"
141                     chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
142
143                     # Extract chunk with ffmpeg
144                     logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
145                     success = extract_video_chunk(video_to_upload, chunk_start,
actual_chunk_dur, chunk_path)
146                     if not success:
147                         logger.error(f"Failed to extract {chunk_label}. Skipping.")
148                         return chunk_label, [], {}
149
150                     # Measure chunk size
151                     try:
152                         chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
153                     except Exception:
154                         chunk_size_mb = 0.0
155
156                     # Get prompt from sport profile
157                     prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
158
159                     # Process this chunk through the provider
160                     chunk_segments, metrics = provider.analyze_video(
161                         chunk_path, prompt, chunk_duration=actual_chunk_dur,
label=chunk_label
162                     )
163                     metrics["file_size_mb"] = chunk_size_mb
164                     metrics["segments_found"] = len(chunk_segments)
165
166                     # Offset timestamps back to full-video coordinates
167                     for segment in chunk_segments:
168                         if "start_time" in segment:
169                             try:
170                                 segment["start_time"] = float(segment["start_time"]) +
chunk_start
171                             except (ValueError, TypeError):
172                                 pass
173                         if "end_time" in segment:
174                             try:
175                                 segment["end_time"] = float(segment["end_time"]) +
chunk_start
176                             except (ValueError, TypeError):
177                                 pass
```

```
178
179                     logger.info(f"{chunk_label} returned {len(chunk_segments)} play
segments.")
180
181                     # Clean up chunk file
182                     try:
183                         os.remove(chunk_path)
184                     except Exception:
185                         pass
186
187                     return chunk_label, chunk_segments, metrics
188
189                 # Run chunks in parallel
190                 all_metrics = {}
191                 logger.info("Starting parallel processing with up to 5 concurrent
workers...")
192                 t_chunking_start = time.time()
193                 with concurrent.futures.ThreadPoolExecutor(max_workers=5) as executor:
194                     futures = [executor.submit(process_single_chunk, ci, start) for ci,
start in enumerate(chunk_starts)]
195                     for future in concurrent.futures.as_completed(futures):
196                         label, segments, metrics = future.result()
197                         ai_segments.extend(segments)
198                         if metrics:
199                             all_metrics[label] = metrics
200
201                 # Print Summary Log
202                 summary_lines = [
203                     "",
204                     "=" * 60,
205                     "               GEMINI CHUNKING SUMMARY",
206                     "=" * 60,
207                     f" Total time: {time.time() - t_chunking_start:.1f}s",
208                     f" Total chunks: {num_chunks}",
209                     f" Total play segments found: {len(ai_segments)}",
210                     "-" * 60,
211                     f" {'Chunk':<12} | {'Size':<8} | {'Upload':<8} | {'Process':<8} |
{'Gen.':<8} | {'Segments'}",
212                     "-" * 60,
213                 ]
214                 for i in range(num_chunks):
215                     lbl = f"Chunk {i+1}/{num_chunks}"
216                     m = all_metrics.get(lbl, {})
217                     sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
218                     upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
219                     prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
220                     gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
221                     seg = m.get('segments_found', 0) if m else 0
222                     summary_lines.append(f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} |
{gen:<8} | {seg}")
223                 summary_lines.append("=" * 60)
224                 logger.info("\n".join(summary_lines))
225
226                 # Clean up local temp files
227                 cleanup_temp_files()
228                 try:
229                     os.rmdir(chunks_dir)
230                 except Exception:
231                     pass
232
233                 logger.info(f"All chunks processed. {len(ai_segments)} total play segments
before deduplication.")
234
235             else:
236                 # --- SINGLE-FILE PROCESSING (short videos) ---
```

```
237                 try:
238                     file_sz = os.path.getsize(video_to_upload) / (1024 * 1024)
239                 except Exception:
240                     file_sz = 0.0
241 
242                 # Get prompt from sport profile
243                 prompt = sport_profile.get_prompt(chunk_duration=video_duration)
244                 ai_segments, metrics = provider.analyze_video(
245                     video_to_upload, prompt, chunk_duration=video_duration,
label="SingleFile"
246                 )
247                 summary_lines = [
248                     "",
249                     "=" * 50,
250                     "        GEMINI SINGLE FILE SUMMARY",
251                     "=" * 50,
252                     f" File Size: {file_sz:.1f} MB",
253                     f" Upload time: {metrics.get('upload_time', 0):.1f}s",
254                     f" API Processing time: {metrics.get('process_time', 0):.1f}s",
255                     f" Prompt Generation time: {metrics.get('gen_time', 0):.1f}s",
256                     f" Play segments found: {len(ai_segments)}",
257                     "=" * 50,
258                 ]
259                 logger.info("\n".join(summary_lines))
260                 cleanup_temp_files()
261 
262             if not ai_segments:
263                 return []
264 
265             # Populate SQLite DB with parsed play segments
266             segments_data = []
267             logger.info(f"Processing {len(ai_segments)} play segments through
validation...")
268 
269             def parse_time(val) -> float:
270                 """Converts a timestamp value to total float seconds.
271                 Handles float/int pass-through, plain numeric strings, and
272                 colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
273                 when unexpected formats are encountered."""
274                 if isinstance(val, (int, float)):
275                     return float(val)
276                 if isinstance(val, str):
277                     val = val.strip()
278                     if ":" in val:
279                         # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal
seconds.
280                         # Convert it, but warn so the issue is visible in logs.
281                         parts = val.split(":")
282                         parts.reverse()
283                         total = 0.0
284                         for i, p in enumerate(parts):
285                             try:
286                                 total += float(p) * (60 ** i)
287                             except ValueError:
288                                 pass
289                         logger.warning(f"Timestamp '{val}' was in colon-separated format
(MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")
290                         return total
291                     try:
292                         return float(val)
293                     except ValueError:
294                         logger.warning(f"Could not parse timestamp string '{val}' as a
number. Defaulting to 0.0.")
295                         return 0.0
296                 logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
```

```
'{val}'. Defaulting to 0.0.")
                return 0.0

            # 7. Parse timestamps and build candidate segment list
            idx_cfg = self.config.get("indexing", {})
            min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
            max_segment_duration = 90.0  # Singles play segments almost never exceed this

            candidates = []
            for idx, segment in enumerate(ai_segments):
                start = parse_time(segment.get("start_time", 0.0))
                end = parse_time(segment.get("end_time", 0.0))
                if start >= end:
                    logger.info(f"Skipping segment #{idx+1}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s)")
                    continue
                candidates.append({
                    "idx": idx,
                    "start": start,
                    "end": end,
                    "raw": segment
                })

            # 8. Post-hoc validation pass
            logger.info(f"Running validation on {len(candidates)} candidate play
segments...")
            validated = []
            rejected = []

            for c in candidates:
                label = f"Segment #{c['idx']+1} [{c['start']:.2f}s - {c['end']:.2f}s]"

                # A: Reject segments that start beyond the video
                if video_duration > 0 and c["start"] >= video_duration:
                    rejected.append((label, f"start_time ({c['start']:.2f}s) is beyond video
duration ({video_duration:.1f}s)"))
                    continue

                # B: Clamp end_time to video duration
                if video_duration > 0 and c["end"] > video_duration:
                    logger.info(f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
({video_duration:.1f}s). Clamping.")
                    c["end"] = video_duration

                dur = c["end"] - c["start"]

                # C: Reject segments shorter than min_segment_duration
                if dur < min_segment_duration:
                    rejected.append((label, f"duration ({dur:.2f}s) is below minimum
({min_segment_duration}s)"))
                    continue

                # D: Flag suspiciously long segments
                if dur > max_segment_duration:
                    logger.warning(
                        f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "
                        f"This may be a detection error. Keeping but flagging."
                    )

                validated.append(c)

            # E: Resolve overlapping segments (keep the longer one)
            if len(validated) > 1:
                # Sort by start time
```

```
                validated.sort(key=lambda r: r["start"])
                non_overlapping = [validated[0]]
                for curr in validated[1:]:
                    prev = non_overlapping[-1]
                    # Check if current segment overlaps with the previous one
                    if curr["start"] < prev["end"]:
                        prev_dur = prev["end"] - prev["start"]
                        curr_dur = curr["end"] - curr["start"]
                        kept, dropped = (prev, curr) if prev_dur >= curr_dur else (curr,
prev)
                        kept_label = f"Segment #{kept['idx']+1} [{kept['start']:.2f}s -
{kept['end']:.2f}s]"
                        dropped_label = f"Segment #{dropped['idx']+1}
[{dropped['start']:.2f}s - {dropped['end']:.2f}s]"
                        rejected.append((dropped_label, f"overlaps with {kept_label} ? kept
the longer segment"))
                        if kept == curr:
                            non_overlapping[-1] = curr  # Replace previous with current
                    else:
                        non_overlapping.append(curr)
                validated = non_overlapping

            # Log validation summary
            if rejected:
                reject_lines = [f"  ? {label}: {reason}" for label, reason in rejected]
                logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
"\n".join(reject_lines))
            logger.info(f"{len(validated)} play segments passed validation (out of
{len(ai_segments)} from Gemini).")

            stitch_cfg = self.config.get("stitching", {})
            begin_pad = stitch_cfg.get("begin_padding", 0.5)
            end_pad = stitch_cfg.get("end_padding", 0.5)

            total_highlight_dur = 0.0
            for c in validated:
                total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad

            mins = int(total_highlight_dur // 60)
            secs = int(total_highlight_dur % 60)
            logger.info(
                f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
                f"({total_highlight_dur:.1f}s total; includes {begin_pad}s begin +
{end_pad}s end padding per segment)"
            )

            # 9. Populate SQLite DB with validated play segments
            for c in validated:
                start = c["start"]
                end = c["end"]
                segment = c["raw"]
                dur = end - start

                # Extract shot_count from Gemini or compute estimate if not present
                shot_count = segment.get("shuttle_exchange_count")
                if shot_count is None:
                    shot_count = max(3, int(dur / 0.8))  # 1 shot every 0.8 seconds as
fallback estimate
                else:
                    try:
                        shot_count = int(shot_count)
                    except Exception:
                        shot_count = max(3, int(dur / 0.8))

                # Only store metrics we actually know ? no fabricated data
```

```
                metadata = {
                    "shot_count": shot_count,
                    "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
                    "detector": model_name
                }

                segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)

                if segment_id:
                    segments_data.append({
                        "id": segment_id,
                        "start_time": start,
                        "end_time": end,
                        "duration": dur,
                        "metadata": metadata
                    })

            return segments_data

    # Register the segmenter
    segmenter_registry.register(GeminiPlaySegmenter)

```

14. user (2026-06-10T05:34:59.233Z)

```
      import cv2
      import time
      import numpy as np
      import random
      from typing import List, Dict, Any, Tuple
      import logging
      from backend.infrastructure.database import Database
      from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
      from backend.results import Result, Success, Failure  # standardized contract

      logger = logging.getLogger(__name__)

      class MotionPlaySegmenter(BasePlaySegmenter):
          @property
          def name(self) -> str:
              return "motion"

          @property
          def description(self) -> str:
              return "Traditional CV pixel-motion heuristics tracking frame differences."

          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
              """
              Processes a video to detect play segments using motion heuristics,
              populates SQLite with detected segments, and indexes detailed events.
              """
              if not player_pool:
                  player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]

              cap = cv2.VideoCapture(video_path)
              if not cap.isOpened():
                  logger.error(f"Segmenter could not open video: {video_path}")
                  return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)

              fps = cap.get(cv2.CAP_PROP_FPS)
              total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
```

```
            duration = total_frames / fps if fps > 0 else 0

            # Sub-sample settings (5 FPS for analysis)
            analysis_fps = 5.0
            frame_interval = max(1, int(fps / analysis_fps))

            idx_cfg = self.config.get("indexing", {})
            motion_threshold = idx_cfg.get("motion_threshold", 0.08)
            min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
            dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)

            prev_gray = None
            motion_signals = []
            timestamps = []

            frame_idx = 0
            processed_count = 0
            t_start = time.time()
            # Emit a progress line roughly every 5% of the video so long runs are
            # observable instead of silent (a full 1080p match is ~tens of minutes).
            progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
            next_progress = progress_step
            logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
                        f"(~{duration/60:.1f} min), analyzing every {frame_interval}th
frame")
            while True:
                # Skip decoding for intermediate frames using cap.grab() for high
performance
                for _ in range(frame_interval - 1):
                    if not cap.grab():
                        break
                    frame_idx += 1

                ret, frame = cap.read()
                if not ret:
                    break

                # Process frame
                gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
                # Apply Gaussian Blur to smooth noise
                gray = cv2.GaussianBlur(gray, (21, 21), 0)

                t = frame_idx / fps
                timestamps.append(t)

                if prev_gray is not None:
                    # Frame absolute difference (highly optimized motion tracking)
                    diff = cv2.absdiff(prev_gray, gray)
                    _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)

                    # Dilate to fill gaps
                    thresh = cv2.dilate(thresh, None, iterations=2)

                    # Motion ratio
                    motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
                    motion_signals.append(motion_ratio)
                else:
                    motion_signals.append(0.0)

                prev_gray = gray
                frame_idx += 1
                processed_count += 1

                if progress_step and frame_idx >= next_progress:
```

```
                    pct = 100.0 * frame_idx / total_frames
                    elapsed = time.time() - t_start
                    eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
                    logger.info(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames,
"
                                f"{processed_count} analyzed, {elapsed:.0f}s elapsed,
~{eta:.0f}s left")
                    next_progress += progress_step

                if max_frames is not None and processed_count >= max_frames:
                    logger.debug(f"Reached max_frames limit of {max_frames}. Halting indexer
frame parsing.")
                    break

                if frame_idx >= total_frames:
                    break

            cap.release()

            # Smooth signal using a moving average
            window_size = 5
            smoothed = []
            for i in range(len(motion_signals)):
                start = max(0, i - window_size // 2)
                end = min(len(motion_signals), i + window_size // 2 + 1)
                smoothed.append(sum(motion_signals[start:end]) / (end - start))

            # Detect active play boundaries
            in_segment = False
            segment_start = None
            raw_segments: List[Tuple[float, float]] = []

            for i, t in enumerate(timestamps):
                signal = smoothed[i]
                if signal > motion_threshold:
                    if not in_segment:
                        in_segment = True
                        segment_start = t
                else:
                    if in_segment:
                        in_segment = False
                        raw_segments.append((segment_start, t))

            # Handle video ending while in active segment
            if in_segment:
                raw_segments.append((segment_start, timestamps[-1]))

            # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
            merged_segments: List[Tuple[float, float]] = []
            if raw_segments:
                curr_start, curr_end = raw_segments[0]
                for next_start, next_end in raw_segments[1:]:
                    if next_start - curr_end < dead_time_merge_gap:
                        # Merge
                        curr_end = next_end
                    else:
                        merged_segments.append((curr_start, curr_end))
                        curr_start, curr_end = next_start, next_end
                merged_segments.append((curr_start, curr_end))

            # Post-Processing: 2. Filter segments shorter than min_segment_duration
            final_segments: List[Tuple[float, float]] = [
                r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
            ]

```

```
            # Populate SQLite DB with segments and metadata events
            segments_data = []
            for start, end in final_segments:
                # Assign Server and Receiver randomly from pool
                server, receiver = random.sample(player_pool, 2)

                # Formulate dynamic rich tags
                dur = end - start
                metadata = {
                    "shot_count": random.randint(4, 25),
                    "intensity": "high" if dur > 12 else "medium",
                    "avg_speed_kmh": round(random.uniform(40, 85), 1)
                }

                segment_id = self.db.add_play_segment(video_id, start, end, server,
receiver, metadata)

                if segment_id:
                    # Add sub-segment events
                    # A: Serve Event (always at start)
                    serve_foul = random.random() < 0.15 # 15% chance of serve foul
                    self.db.add_event(
                        segment_id=segment_id,
                        timestamp=start + 0.5,
                        event_type="serve",
                        player=server,
                        details={"status": "foul" if serve_foul else "valid", "type":
"low_serve" if random.random() > 0.5 else "high_serve"}
                    )

                    # B: Smash Events (1 to 3 random smashes per segment)
                    smashes_count = random.randint(0, 3)
                    for _ in range(smashes_count):
                        smash_time = random.uniform(start + 1.5, end - 1.0)
                        smasher = random.choice([server, receiver])
                        self.db.add_event(
                            segment_id=segment_id,
                            timestamp=round(smash_time, 2),
                            event_type="smash",
                            player=smasher,
                            details={"speed_kmh": round(random.uniform(120, 250), 1)}
                        )

                    # C: Segment Termination Event (foul/winner)
                    foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
                    is_foul = random.random() > 0.4
                    error_player = random.choice([server, receiver])

                    self.db.add_event(
                        segment_id=segment_id,
                        timestamp=end - 0.2,
                        event_type="foul" if is_foul else "winner",
                        player=error_player,
                        details={"type": random.choice(foul_types)} if is_foul else
{"shot_type": "drop" if random.random() > 0.5 else "smash"}
                    )

                    # Store segment representation to return to caller
                    segments_data.append({
                        "id": segment_id,
                        "start_time": start,
                        "end_time": end,
                        "duration": dur,
                        "server": server,
                        "receiver": receiver,
```

```
                        "metadata": metadata
                    })

            return Success(value=segments_data)

    # Register the segmenter
    segmenter_registry.register(MotionPlaySegmenter)

```

15. user (2026-06-10T05:35:00.154Z)

```
    """Two-stage rally segmenter: torchvision person detection + ByteTrack tracking
    (Stage 1, local) feeding an AI provider for semantic boundaries (Stage 2).

    Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted
    service free of copyleft obligations (see docs/MONETIZATION_AUDIT.md and ADR-005),
    detection now uses torchvision's permissively-licensed (BSD) detectors and the
    association/tracking that YOLO.track() used to bundle in is provided by
    supervision's MIT-licensed ByteTrack.

    The registry key stays "yolo_hybrid" for back-compat with existing configs and the
    DB `detector` tags, even though no YOLO is involved anymore.
    """
    import os
    import time
    import logging
    import cv2
    import torch
    import concurrent.futures
    from typing import List, Dict, Any, Tuple

    from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
    from backend.utils.video import get_video_duration, extract_video_chunk
    from backend.utils.geometry import get_velocity, get_velocity_normalised
    from backend.sports import sport_registry
    from backend.providers import provider_registry

    logger = logging.getLogger(__name__)

    # torchvision detection models are trained on COCO where the "person" category is
    # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
    # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
    _PERSON_CLASS_ID = 1

    # Permissively-licensed torchvision detectors selectable via config
    # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
    _DETECTOR_FACTORIES = {
        "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
        "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
        "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
    }
    _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"


    def _fmt_ts(seconds: float) -> str:
        """Format seconds as HH:MM:SS for cross-verifying a timestamp against the video."""
        seconds = max(0, int(seconds))
        h, rem = divmod(seconds, 3600)
        m, s = divmod(rem, 60)
        return f"{h:02d}:{m:02d}:{s:02d}"


    class HybridYoloSegmenter(BasePlaySegmenter):
        def __init__(self, db, config):
            super().__init__(db, config)
```

```
55              self.model = None
56              self.device = None
57
58          @property
59          def name(self) -> str:
60              return "yolo_hybrid"
61
62          @property
63          def description(self) -> str:
64              return ("Two-stage pipeline: torchvision person detection + ByteTrack for fast "
65                      "candidate filtering, then an AI provider for high-precision semantic
boundaries.")
66
67          def _lazy_load_model(self):
68              if self.model is not None:
69                  return
70
71              # Imported lazily so the module imports without torchvision present and so
72              # the test suite (which pre-sets self.model) never pulls in model weights.
73              from torchvision.models import detection as tv_detection
74
75              idx_cfg = self.config.get("indexing", {})
76              use_gpu = idx_cfg.get("use_gpu", True)
77              self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
78              if use_gpu and not torch.cuda.is_available():
79                  logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
80
81              model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
82              if model_name not in _DETECTOR_FACTORIES:
83                  logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
84                  model_name = _DEFAULT_DETECTOR
85
86              builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
87              logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
88              # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
89              self.model = builder(weights="DEFAULT")
90              self.model.eval()
91              self.model.to(self.device)
92
93          def _make_tracker(self, fps: float):
94              """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
95
96              Imported lazily so tests can mock this method without supervision installed.
97              """
98              import supervision as sv
99              frame_rate = max(1, int(round(fps)))
100             return sv.ByteTrack(frame_rate=frame_rate)
101
102         def _detect_and_track(self, frame, tracker, score_thresh: float) -> List[Tuple[int,
Tuple[float, float, float, float]]]:
103             """Detect persons in a frame and assign persistent track IDs.
104
105             Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
106             This replaces the old ultralytics ``model.track(...)`` call, which bundled
107             detection AND association: torchvision does the detection, supervision's
108             ByteTrack does the association.
109             """
110             import numpy as np
111             import supervision as sv
112
113             # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
114             rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
115             tensor = torch.from_numpy(rgb).permute(2, 0,
```

```
1).float().div(255.0).to(self.device)
116             with torch.no_grad():
117                 outputs = self.model([tensor])
118 
119             out = outputs[0]
120             boxes = out["boxes"].detach().cpu().numpy()
121             labels = out["labels"].detach().cpu().numpy()
122             scores = out["scores"].detach().cpu().numpy()
123 
124             # Keep only confident person detections.
125             mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
126             if not mask.any():
127                 return []
128 
129             # ByteTrack requires confidence to be populated (it filters on it internally).
130             dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
131             tracked = tracker.update_with_detections(dets)
132 
133             result: List[Tuple[int, Tuple[float, float, float, float]]] = []
134             for i in range(len(tracked)):
135                 tid = tracked.tracker_id[i]
136                 if tid is None:
137                     continue
138                 x1, y1, x2, y2 = tracked.xyxy[i]
139                 result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
140             return result
141 
142         def _extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
143                                                velocity_thresh: float, merge_gap: float,
144                                                frame_skip: int = 3,
145                                                tracker_config: str = "bytetrack.yaml",
146                                                chunk_index: int = None) -> List[Dict[str,
Any]]:
147             """
148             Runs person detection + tracking on a video chunk and extracts high-motion
149             action windows. Returns a list of dicts in the full video's timeframe, each
with:
150                 start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
151                 track_id_count, chunk_index.
152 
153             Args:
154                 frame_skip: Process every Nth frame (1 = every frame). Higher values
155                             dramatically reduce inference cost with minimal quality loss.
156                 tracker_config: Retained for back-compat (was the YOLO tracker yaml); the
157                             tracker is now supervision ByteTrack and this is unused.
158                 chunk_index: Index of the source chunk (recorded on each window for
traceability).
159             """
160             cap = cv2.VideoCapture(chunk_path)
161             if not cap.isOpened():
162                 logger.error(f"Could not open {chunk_path}")
163                 return []
164 
165             fps = cap.get(cv2.CAP_PROP_FPS)
166             if fps <= 0:
167                 fps = 30.0
168 
169             frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
170             if frame_width <= 0:
171                 frame_width = 1280.0  # Sensible fallback
172 
173             # Effective sampling rate after frame-skipping
174             effective_fps = fps / max(frame_skip, 1)
175 
```

```
            # A fresh tracker per chunk ? track IDs only need to be consistent within a
            # chunk (windows are computed per chunk, then merged across chunks by time).
            tracker = self._make_tracker(effective_fps)
            score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)

            # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
            # so windows can be enriched with motion stats for logging + fine-tuning.
            active_frames = []
            frame_idx = 0
            track_history = {}

            while True:
                ret, frame = cap.read()
                if not ret:
                    break

                # Skip frames for performance ? at 30fps, every 3rd frame still
                # gives ~10 samples/second, plenty for human-scale velocity.
                if frame_idx % max(frame_skip, 1) != 0:
                    frame_idx += 1
                    continue

                # Detect persons + assign persistent track IDs.
                detections = self._detect_and_track(frame, tracker, score_thresh)

                frame_peak_velocity = 0.0
                frame_active_ids = set()
                current_ids = set()

                for track_id, bbox in detections:
                    current_ids.add(track_id)

                    if track_id in track_history:
                        prev_bbox = track_history[track_id]
                        velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
                        if velocity > velocity_thresh:
                            frame_peak_velocity = max(frame_peak_velocity, velocity)
                            frame_active_ids.add(track_id)

                    track_history[track_id] = bbox

                # Prune stale IDs that are no longer visible ? prevents false
                # velocity spikes when the tracker reassigns an old ID.
                track_history = {k: v for k, v in track_history.items() if k in current_ids}

                if frame_active_ids:
                    active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))

                frame_idx += 1

            cap.release()

            # Group active frames into continuous windows
            if not active_frames:
                return []

            max_gap_frames = int(merge_gap * fps)

            def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
                velocities = [v for _, v, _ in group]
                track_ids = set().union(*[ids for _, _, ids in group])
                return {
                    "start": group[0][0] / fps + chunk_start,
```

```
                    "end": group[-1][0] / fps + chunk_start,
                    "active_frame_count": len(group),
                    "peak_velocity": max(velocities),
                    "mean_velocity": sum(velocities) / len(velocities),
                    "track_id_count": len(track_ids),
                    "chunk_index": chunk_index,
                }

            windows = []
            group = [active_frames[0]]
            for af in active_frames[1:]:
                if af[0] - group[-1][0] <= max_gap_frames:
                    group.append(af)
                else:
                    windows.append(_finalize(group))
                    group = [af]
            windows.append(_finalize(group))

            # Filter out extremely short action windows
            min_window_duration = self.config.get("indexing",
{}).get("min_action_window_duration", 2.0)
            windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

            return windows

        def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
            self._lazy_load_model()

            # Get sport profile config
            sport_name = self.config.get("sport", "badminton")
            try:
                sport_profile = sport_registry.get(sport_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            # Get AI provider and model config
            idx_cfg = self.config.get("indexing", {})
            provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
            model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))

            # Get appropriate API key based on the provider
            api_key_env_var = f"{provider_name.upper()}_API_KEY"
            api_key = os.environ.get(api_key_env_var)
            if not api_key:
                logger.error(
                    f"{api_key_env_var} environment variable is not set! "
                    f"To run the {provider_name} segmenter, set your API key. "
                    f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
                )
                return []

            try:
                provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            logger.info(f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}")

```

```
            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                return []

            chunk_duration = idx_cfg.get("chunk_duration", 300)
            chunk_overlap = idx_cfg.get("chunk_overlap", 30)
            velocity_thresh = idx_cfg.get("yolo_velocity_threshold", 0.15)  # Fraction of
frame width per second
            merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
            frame_skip = idx_cfg.get("yolo_frame_skip", 3)
            tracker_config = idx_cfg.get("yolo_tracker", "bytetrack.yaml")
            handoff_padding = idx_cfg.get("ai_handoff_padding",
idx_cfg.get("gemini_handoff_padding", 2.0))
            ai_max_workers = idx_cfg.get("ai_max_workers", idx_cfg.get("gemini_max_workers",
5))
            log_candidates = idx_cfg.get("log_yolo_candidates", True)
            store_candidates = idx_cfg.get("store_yolo_candidates", True)

            # Snapshot of the params that produced these detections ? stored alongside each
            # candidate so a fine-tuning run can reproduce / correlate against them.
            detection_params = {
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "frame_skip": frame_skip,
                "tracker": tracker_config,
                "detector_model": idx_cfg.get("detector_model", _DEFAULT_DETECTOR),
                "detector_score_threshold": idx_cfg.get("detector_score_threshold", 0.5),
                "min_action_window_duration": idx_cfg.get("min_action_window_duration",
2.0),
            }

            chunks_dir = os.path.join("output", "chunks_yolo")
            os.makedirs(chunks_dir, exist_ok=True)

            step = chunk_duration - chunk_overlap
            chunk_starts = []
            t = 0.0
            while t < video_duration:
                chunk_starts.append(t)
                t += step

            num_chunks = len(chunk_starts)
            logger.info(f"Video duration: {video_duration:.1f}s. Splitting into {num_chunks}
chunks.")

            all_candidate_windows = []

            t_start = time.time()
            prefetch_workers = idx_cfg.get("yolo_prefetch_workers", 2)

            if num_chunks > 1 and prefetch_workers > 0:
                # --- PIPELINED PROCESSING ---
                # GPU inference must stay sequential (single GPU), but ffmpeg chunk
                # extraction is pure I/O. We pre-extract upcoming chunks in background
                # threads so extraction of chunk N+1 overlaps with inference on chunk N.
                logger.info(f"Running detection Phase 2 with pipelined extraction
({prefetch_workers} prefetch workers)...")

                extract_executor =
concurrent.futures.ThreadPoolExecutor(max_workers=prefetch_workers)

                def _extract_chunk(ci: int, cs: float) -> Tuple[int, float, str, bool]:
                    chunk_end = min(cs + chunk_duration, video_duration)
                    actual_dur = chunk_end - cs
```

```
                    chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
                    success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
                    return (ci, cs, chunk_path, success)

                # Submit all extractions upfront ? they'll queue and run as workers free up
                extract_futures = [
                    extract_executor.submit(_extract_chunk, ci, cs)
                    for ci, cs in enumerate(chunk_starts)
                ]

                # Process results in order: wait for extraction, then run inference on GPU
                for future in extract_futures:
                    ci, chunk_start, chunk_path, success = future.result()
                    chunk_end = min(chunk_start + chunk_duration, video_duration)
                    logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")

                    if not success:
                        logger.warning(f"Chunk {ci+1} extraction failed. Skipping.")
                        continue

                    windows = self._extract_action_windows_from_chunk(
                        chunk_path, chunk_start, velocity_thresh, merge_gap,
                        frame_skip=frame_skip, tracker_config=tracker_config, chunk_index=ci
                    )
                    logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
                    all_candidate_windows.extend(windows)

                    try:
                        os.remove(chunk_path)
                    except Exception:
                        pass

                extract_executor.shutdown(wait=False)
            else:
                # --- SEQUENTIAL PROCESSING (single chunk or prefetch disabled) ---
                for ci, chunk_start in enumerate(chunk_starts):
                    chunk_end = min(chunk_start + chunk_duration, video_duration)
                    actual_dur = chunk_end - chunk_start
                    chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")

                    logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")
                    success = extract_video_chunk(video_path, chunk_start, actual_dur,
chunk_path)
                    if not success:
                        continue

                    windows = self._extract_action_windows_from_chunk(
                        chunk_path, chunk_start, velocity_thresh, merge_gap,
                        frame_skip=frame_skip, tracker_config=tracker_config, chunk_index=ci
                    )
                    logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
                    all_candidate_windows.extend(windows)

                    try:
                        os.remove(chunk_path)
                    except Exception:
                        pass

            # Deduplicate overlapping candidate windows across chunks (chunks overlap by
            # chunk_overlap, so the same rally can surface in two adjacent chunks).
            def _merge_stats(a: Dict[str, Any], b: Dict[str, Any]) -> Dict[str, Any]:
```

```
                fa, fb = a["active_frame_count"], b["active_frame_count"]
                total = fa + fb or 1
                return {
                    "start": a["start"],
                    "end": max(a["end"], b["end"]),
                    "active_frame_count": fa + fb,
                    "peak_velocity": max(a["peak_velocity"], b["peak_velocity"]),
                    # Frame-weighted mean so longer sub-windows dominate proportionally.
                    "mean_velocity": (a["mean_velocity"] * fa + b["mean_velocity"] * fb) /
total,
                    "track_id_count": max(a["track_id_count"], b["track_id_count"]),
                    "chunk_index": a["chunk_index"],
                }

            if all_candidate_windows:
                all_candidate_windows.sort(key=lambda w: w["start"])
                merged_windows = [all_candidate_windows[0]]
                for current in all_candidate_windows[1:]:
                    last = merged_windows[-1]
                    if current["start"] <= last["end"]:  # Overlap
                        merged_windows[-1] = _merge_stats(last, current)
                    else:
                        merged_windows.append(current)
                all_candidate_windows = merged_windows

            logger.info(f"Phase 2 tracking completed in {time.time() - t_start:.1f}s.")
            logger.info(f"Total candidate windows: {len(all_candidate_windows)}")

            # Per-rally console log (final, full-video timestamps) for video cross-
verification.
            if log_candidates:
                for w in all_candidate_windows:
                    logger.info(
                        f"[rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
                        f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']}
"
                        f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f} "
                        f"tracks={w['track_id_count']}"
                    )

            # Persist raw candidates for the fine-tuning dataset. Map window index ->
candidate_id
            # so confirmed AI segments can be linked back in Phase 4.
            candidate_ids = [None] * len(all_candidate_windows)
            if store_candidates:
                for i, w in enumerate(all_candidate_windows):
                    stats = {
                        "peak_velocity": w["peak_velocity"],
                        "mean_velocity": w["mean_velocity"],
                        "active_frame_count": w["active_frame_count"],
                        "track_id_count": w["track_id_count"],
                    }
                    # Use peak velocity (capped at 1.0) as a coarse generic confidence
score.
                    confidence = min(1.0, w["peak_velocity"])
                    candidate_ids[i] = self.db.add_candidate_segment(
                        video_id, detector="yolo_hybrid",
                        start_time=w["start"], end_time=w["end"],
                        chunk_index=w["chunk_index"], confidence=confidence,
                        stats=stats, detection_params=detection_params,
                    )

            try:
                os.rmdir(chunks_dir)
            except Exception:
```

```
476                 pass
477
478             # Phase 3: AI Provider Handoff
479             gemini_segments = []
480             gemini_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
481             os.makedirs(gemini_dir, exist_ok=True)
482
483             logger.info(f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows...")
484
485             def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
486                 w_start = max(0, window["start"] - handoff_padding)
487                 w_end = min(video_duration, window["end"] + handoff_padding)
488                 w_dur = w_end - w_start
489
490                 w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
491                 # Re-encode short handoff clips to avoid keyframe-boundary drift
492                 success = extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True)
493                 if not success:
494                     return []
495
496                 prompt = sport_profile.get_prompt(chunk_duration=w_dur)
497                 segments, metrics = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur, label=f"Handoff {wi+1}")
498
499                 # Coordinate mapping ? shift local timestamps back to full-video time
500                 valid_segments = []
501                 for segment in segments:
502                     try:
503                         segment["start_time"] = float(segment["start_time"]) + w_start
504                         segment["end_time"] = float(segment["end_time"]) + w_start
505                         # Stamp the source candidate so Phase 4 can link
prediction->confirmation.
506                         segment["_candidate_id"] = candidate_ids[wi]
507                         valid_segments.append(segment)
508                     except (ValueError, TypeError, KeyError) as e:
509                         logger.warning(f"Dropping segment with unparseable timestamps:
{segment} ? {e}")
510
511                 try:
512                     os.remove(w_path)
513                 except OSError:
514                     pass
515
516                 return valid_segments
517
518             with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
519                 futures = [executor.submit(process_candidate_window, wi, w) for wi, w in
enumerate(all_candidate_windows)]
520                 # Iterate in submission order to preserve temporal ordering
521                 for future in futures:
522                     gemini_segments.extend(future.result())
523
524             try:
525                 os.rmdir(gemini_dir)
526             except OSError:
527                 pass
528
529             logger.info(f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic
play segments.")
530
531             # Phase 4: DB Population
532             segments_data = []
```

```
            for segment in gemini_segments:
                try:
                    start = float(segment["start_time"])
                    end = float(segment["end_time"])
                    dur = end - start

                    shot_count = segment.get("shuttle_exchange_count")
                    if shot_count is None: shot_count = max(3, int(dur / 0.8))
                    else:
                        try: shot_count = int(shot_count)
                        except (ValueError, TypeError): shot_count = max(3, int(dur / 0.8))

                    metadata = {
                        "shot_count": shot_count,
                        "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
                        "detector": f"yolo-hybrid-{model_name}"
                    }

                    segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
                    if segment_id:
                        # Link back to the candidate that produced this segment. A candidate
                        # may yield multiple AI segments (one-to-many); confirmed_segment_id
keeps
                        # the last one ? the link table records "this candidate was
confirmed".
                        candidate_id = segment.get("_candidate_id")
                        if candidate_id is not None:
                            self.db.link_candidate_to_segment(candidate_id, segment_id)
                        segments_data.append({
                            "id": segment_id,
                            "start_time": start,
                            "end_time": end,
                            "duration": dur,
                            "metadata": metadata
                        })
                except Exception as e:
                    logger.warning(f"Skipping invalid segment struct: {e}")

            return segments_data

    # Register the segmenter
    segmenter_registry.register(HybridYoloSegmenter)

```

16. assistant (2026-06-10T05:35:18.584Z)

Now the detectors layer.

17. user (2026-06-10T05:35:19.816Z)

```
      """Abstract base for detector runners (TrackNet, WASB, future native, etc.).

      This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
      of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
      the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.

      The ABC lets us:
      - Keep the current WSL adapters working unchanged for Windows users.
      - Swap in (or A/B test) a Linux-native implementation later with zero changes to
        the hybrid segmenters or the model-agnostic windowing brain.
      - Provide a single place to evolve the healthcheck / error contract and thread
        detector readiness into per-video reports.

```

```
    Contract (deliberately matches what the concrete runners + hybrids + tests already did):
    - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
      Run detector, return the Windows-side path to the produced trajectory CSV
      (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
      The caller (hybrid) is responsible for output dir; impls may stage the video
      into WSL fs for perf.

    - healthcheck() -> tuple[bool, str]
      Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
      Message is logged (and can be surfaced to reports or UX in future).
      Intended to be cheap (env existence, import, GPU visibility, weights present).

    Existing behavior is 100% preserved. The only additions are the inheritance and
    the explicit common type for future implementers.

    How to add a new runner (for docs / future contributors):
    1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
    2. Implement run_predict and healthcheck (return (bool, str) tuple).
    3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
       TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
       deliberately static and model-agnostic). If the new detector needs different
       windowing, it can implement its own or we generalize later.
    4. Lazy-import heavy native deps inside the methods (so importing the segmenter
       registry never pulls torch/tf on a machine that doesn't have them).
    5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
       DetectorRunner instances (they already do the right thing via duck-typing + the
       type hints we add here).
    6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
       returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
    7. No change to eval/calibration code paths (they consume the CSV + the static
       windowing helpers directly).

    Optional Linux-native path (the main de-risk goal):
    A LinuxNativeRunner (or ONNXRunner) would:
    - Skip all wsl / _stage / conda activate.
    - Load weights locally (torch or onnx) or call a native binary.
    - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
    - healthcheck can just check "weights file exists and torch importable + CUDA?".
    This removes the WSL requirement for Linux users / future SaaS workers while
    keeping the exact same candidate quality characteristics (or better, once native
    weights are available).

    Error surfacing into reports:
    The healthcheck message is already logged by the hybrids on failure. The reporting
    harvest sees the outcome as "0 candidates" + detector name. When we wire richer
    segmenter Results (or pass detector_health into emit_indexer_report context), the
    (ok, msg) can be recorded explicitly under segmentation.details or a new detector
    stage. The ABC makes that future change local to one place.

    See also:
    - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
    - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
    - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
    """

    from abc import ABC, abstractmethod
    from typing import Optional, Tuple


    class DetectorRunner(ABC):
        """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""

        @abstractmethod
        def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
            """Run inference and return path to trajectory CSV (or None on failure)."""
```

```
79              pass
80
81          @abstractmethod
82          def healthcheck(self) -> Tuple[bool, str]:
83              """Quick check if the runner (env, weights, etc.) is usable.
84
85              Returns (ok, message). Never raises. Cheap to call before a long run.
86              """
87              pass
88
```

18. user (2026-06-10T05:35:20.109Z)

```
1       """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4       inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6           1. Translate the Windows video path to its WSL-visible form (and optionally
7              stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8           2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9           3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10          4. Convert that trajectory into high-motion "action windows" ? the same
11             candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12             pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
20      import logging
21      import subprocess
22      from dataclasses import dataclass
23      from typing import List, Dict, Any, Optional, Tuple
24
25      from backend.pipeline.detectors.base import DetectorRunner
26
27      logger = logging.getLogger(__name__)
28
29
30      @dataclass
31      class TrackNetConfig:
32          """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "TrackNetV4"
37          weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
38          queue_length: int = 5
39          stage_in_wsl: bool = True                       # copy video into WSL fs first
(perf)
40          wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
in WSL
41          timeout_sec: int = 0                            # 0 = no timeout
42
43          @classmethod
44          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
45              tn = (idx_cfg or {}).get("tracknet", {}) or {}
46              return cls(
47                  distro=tn.get("wsl_distro", "Ubuntu"),
48                  repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
```

```
                conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
                conda_env=tn.get("conda_env", "TrackNetV4"),
                weights_path=tn.get("weights_path", ""),
                queue_length=int(tn.get("queue_length", 5)),
                stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
                wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
                timeout_sec=int(tn.get("timeout_sec", 0)),
            )


    def to_wsl_mnt_path(win_path: str) -> str:
        """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).

        Already-POSIX paths (starting with / or ~) are returned unchanged so the
        same helper is safe to call on either kind of path.
        """
        p = str(win_path)
        if p.startswith("/") or p.startswith("~"):
            return p
        p = p.replace("\\", "/")
        if len(p) >= 2 and p[1] == ":":
            drive = p[0].lower()
            return f"/mnt/{drive}{p[2:]}"
        return p


    @dataclass
    class TrajectoryPoint:
        frame: int
        visible: bool
        x: float
        y: float


    class TrackNetRunner(DetectorRunner):  # DetectorRunner ABC (finish item 6 / new item 1)
        """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.

        Implements the common DetectorRunner contract so a future Linux-native or
        alternative trajectory runner can be substituted without touching the hybrids.
        """

        def __init__(self, cfg: TrackNetConfig):
            self.cfg = cfg

        # ------------------------------------------------------------------ WSL exec

        def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
            """Run a bash command string inside the configured WSL distro."""
            full = f"source {self.cfg.conda_sh} && {inner_cmd}"
            argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
            logger.debug("WSL exec: %s", full)
            return subprocess.run(
                argv, capture_output=True, text=True,
                timeout=(self.cfg.timeout_sec or None),
            )

        def healthcheck(self) -> Tuple[bool, str]:
            """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.

            Returns (ok, message). Cheap to call before a long run.
            """
            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"python -c \"import tensorflow as tf; "
                f"print('TF', tf.__version__, 'GPUs',
```

```
len(tf.config.list_physical_devices('GPU')))\""
            )
            try:
                res = self._wsl_bash(cmd)
            except FileNotFoundError:
                return False, "`wsl` not found ? is this running on Windows with WSL2
installed?"
            except subprocess.TimeoutExpired:
                return False, "WSL healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return False, f"WSL env check failed: {(res.stderr or out).strip()}"
            return True, out

        # ------------------------------------------------------------------ inference

        def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
            """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.

            ``output_win_dir`` is a Windows directory (under the repo's output/) that
            WSL writes into via its /mnt view, so the Windows process can read the CSV
            back with no copy.
            """
            if not self.cfg.weights_path:
                logger.error(
                    "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
                    "in config.json to the WSL path of your trained TrackNetV4 weights."
                )
                return None

            os.makedirs(output_win_dir, exist_ok=True)
            # Normalize separators so basename/splitext work when tests (or collector)
            # pass Windows-style paths on a Linux host.
            video_win_path = str(video_win_path).replace("\\", "/")
            base = os.path.basename(video_win_path)
            stem, ext = os.path.splitext(base)

            # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
            if ext.lower() not in (".mp4", ".avi"):
                logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
                return None

            wsl_out = to_wsl_mnt_path(output_win_dir)
            expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")

            # Resolve the video path WSL will read.
            if self.cfg.stage_in_wsl:
                staged = self._stage_video(video_win_path, base)
                if staged is None:
                    return None
                wsl_video = staged
            else:
                wsl_video = to_wsl_mnt_path(video_win_path)

            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"cd {self.cfg.repo_dir}/src && "
                f"python predict.py "
                f"--video_path {shlex.quote(wsl_video)} "
                f"--model_weights {shlex.quote(self.cfg.weights_path)} "
                f"--output_dir {shlex.quote(wsl_out)} "
                f"--queue_length {self.cfg.queue_length}"
            )
            logger.info("Running TrackNet inference on %s ...", base)
```

```
176             try:
177                 res = self._wsl_bash(cmd)
178             except subprocess.TimeoutExpired:
179                 logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
180                 return None
181 
182             if res.returncode != 0:
183                 logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
184                 return None
185 
186             if not os.path.exists(expected_csv_win):
187                 logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
188                 return None
189             logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
190             return expected_csv_win
191 
192         def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
193             """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
194 
195             Returns the WSL path to the staged copy, or None on failure.
196             """
197             src_mnt = to_wsl_mnt_path(video_win_path)
198             dest = f"{self.cfg.wsl_stage_dir}/{base}"
199             cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{shlex.quote(dest)} && echo OK"
200             try:
201                 res = self._wsl_bash(cmd)
202             except subprocess.TimeoutExpired:
203                 logger.error("Staging video into WSL timed out.")
204                 return None
205             if res.returncode != 0 or "OK" not in (res.stdout or ""):
206                 logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
207                 return None
208             return dest
209 
210         # ------------------------------------------------------------ CSV -> windows
211 
212         @staticmethod
213         def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
214             """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
215             points: List[TrajectoryPoint] = []
216             with open(csv_win_path, newline="") as f:
217                 reader = csv.DictReader(f)
218                 for row in reader:
219                     try:
220                         points.append(TrajectoryPoint(
221                             frame=int(row["Frame"]),
222                             visible=int(row["Visibility"]) == 1,
223                             x=float(row["X"]),
224                             y=float(row["Y"]),
225                         ))
226                     except (KeyError, ValueError):
227                         continue
228             points.sort(key=lambda p: p.frame)
229             return points
230 
231         @staticmethod
232         def trajectory_to_action_windows(
233             points: List[TrajectoryPoint],
234             fps: float,
235             frame_width: float,
```

```
            chunk_start: float = 0.0,
            velocity_thresh: float = 0.02,
            merge_gap: float = 2.0,
            min_window_duration: float = 2.0,
            chunk_index: Optional[int] = None,
        ) -> List[Dict[str, Any]]:
            """Convert a shuttle trajectory into candidate rally windows.

            A frame is "active" when the shuttle is visible AND its inter-frame
            displacement (normalised by frame width, per second) exceeds
            ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
            ``merge_gap`` seconds of each other are grouped into one window; short
            windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
            shape feeding the AI-handoff phase is identical.
            """
            if fps <= 0:
                fps = 30.0
            if frame_width <= 0:
                frame_width = 1280.0

            # Compute per-frame normalised velocity from the last *visible* point.
            active = []  # (frame_idx, velocity)
            prev: Optional[TrajectoryPoint] = None
            for p in points:
                if not p.visible:
                    prev = None  # break the trajectory; absent shuttle = not in flight
                    continue
                if prev is not None:
                    dframes = p.frame - prev.frame
                    if dframes > 0:
                        dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
                        velocity = (dist / frame_width) * (fps / dframes)
                        if velocity > velocity_thresh:
                            active.append((p.frame, velocity))
                prev = p

            if not active:
                return []

            max_gap_frames = int(merge_gap * fps)

            def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
                vels = [v for _, v in group]
                return {
                    "start": group[0][0] / fps + chunk_start,
                    "end": group[-1][0] / fps + chunk_start,
                    "active_frame_count": len(group),
                    "peak_velocity": max(vels),
                    "mean_velocity": sum(vels) / len(vels),
                    "track_id_count": 1,  # single shuttle; kept for window-shape parity
                    "chunk_index": chunk_index,
                }

            windows = []
            group = [active[0]]
            for af in active[1:]:
                if af[0] - group[-1][0] <= max_gap_frames:
                    group.append(af)
                else:
                    windows.append(_finalize(group))
                    group = [af]
            windows.append(_finalize(group))

            return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

```

19. user (2026-06-10T05:35:32.291Z)

```
"""WASB shuttle-trajectory runner (WSL2 subprocess adapter).

Windows-side counterpart to backend/detectors/wasb_infer.py. It:
  1. stages the video into the WSL filesystem (fast local reads, not /mnt),
  2. copies wasb_infer.py into the WASB repo's src/ dir,
  3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
  4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.

So the only WASB-specific bits live here; the windowing + CSV parsing are shared
with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
"""
import os
import shlex
import logging
import subprocess
from dataclasses import dataclass
from typing import Dict, Any, Optional, Tuple

# Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
# parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
from backend.pipeline.detectors.base import DetectorRunner

logger = logging.getLogger(__name__)

# Windows path to the inference wrapper we stage into WSL.
_INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")


@dataclass
class WasbConfig:
    """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
    distro: str = "Ubuntu"
    repo_dir: str = "~/models/WASB-SBDT"
    conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
    conda_env: str = "wasb"
    weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
    sport: str = "badminton"
    wsl_stage_dir: str = "~/clips"
    timeout_sec: int = 0  # 0 = no timeout

    @classmethod
    def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
        w = (idx_cfg or {}).get("wasb", {}) or {}
        return cls(
            distro=w.get("wsl_distro", "Ubuntu"),
            repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
            conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
            conda_env=w.get("conda_env", "wasb"),
            weights_path=w.get("weights_path",
                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
            sport=w.get("sport", "badminton"),
            wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
            timeout_sec=int(w.get("timeout_sec", 0)),
        )


class WasbRunner(DetectorRunner):  # DetectorRunner ABC (finish item 6 / new item 1)
    def __init__(self, cfg: WasbConfig):
```

```
60              self.cfg = cfg
61
62          def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
63              full = f"source {self.cfg.conda_sh} && {inner_cmd}"
64              argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
65              logger.debug("WSL exec: %s", full)
66              return subprocess.run(argv, capture_output=True, text=True,
67                                    timeout=(self.cfg.timeout_sec or None))
68
69          def healthcheck(self) -> Tuple[bool, str]:
70              """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
71              cmd = (f"conda activate {self.cfg.conda_env} && "
72                     f"python -c \"import torch; print('torch', torch.__version__, "
73                     f"'cuda', torch.cuda.is_available())\"")
74              try:
75                  res = self._wsl_bash(cmd)
76              except FileNotFoundError:
77                  return False, "`wsl` not found ? Windows + WSL2 required."
78              except subprocess.TimeoutExpired:
79                  return False, "WSL healthcheck timed out."
80              out = (res.stdout or "").strip()
81              if res.returncode != 0:
82                  return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
83              return True, out
84
85          def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
86              src_mnt = to_wsl_mnt_path(video_win_path)
87              dest = f"{self.cfg.wsl_stage_dir}/{base}"
88              cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{shlex.quote(dest)} && echo OK"
89              try:
90                  res = self._wsl_bash(cmd)
91              except subprocess.TimeoutExpired:
92                  logger.error("Staging video into WSL timed out.")
93                  return None
94              if res.returncode != 0 or "OK" not in (res.stdout or ""):
95                  logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
96                  return None
97              return dest
98
99          def _stage_infer_script(self) -> bool:
100             """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
101             src_mnt = to_wsl_mnt_path(_INFER_WIN)
102             cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
103             res = self._wsl_bash(cmd)
104             return res.returncode == 0 and "OK" in (res.stdout or "")
105
106         def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
107             """Run WASB on a video. Returns the Windows path to the trajectory CSV, or
None."""
108             os.makedirs(output_win_dir, exist_ok=True)
109             # Normalize for cross-platform basename (tests use Windows paths on Linux).
110             video_win_path = str(video_win_path).replace("\\", "/")
111             base = os.path.basename(video_win_path)
112             stem = os.path.splitext(base)[0]
113             wsl_out_dir = to_wsl_mnt_path(output_win_dir)
114             expected_csv_win = os.path.join(output_win_dir, f"{stem}_wasb.csv")
115
116             if not self._stage_infer_script():
117                 logger.error("Could not stage wasb_infer.py into WSL.")
118                 return None
119             staged_video = self._stage_video(video_win_path, base)
120             if staged_video is None:
```

```
121                 return None
122             frames_dir = f"{self.cfg.wsl_stage_dir}/{stem}_frames"
123 
124             cmd = (
125                 f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
126                 f"python wasb_infer.py "
127                 f"--video {shlex.quote(staged_video)} "
128                 f"--frames_out_dir {shlex.quote(frames_dir)} "
129                 f"--weights {shlex.quote(self.cfg.weights_path)} "
130                 f"--sport {shlex.quote(self.cfg.sport)} "
131                 f"--out {shlex.quote(wsl_out_dir + '/' + stem + '_wasb.csv')}"
132             )
133             logger.info("Running WASB inference on %s ...", base)
134             try:
135                 res = self._wsl_bash(cmd)
136             except subprocess.TimeoutExpired:
137                 logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
138                 return None
139             if res.returncode != 0:
140                 logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
141                 return None
142             if not os.path.exists(expected_csv_win):
143                 logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
144                 return None
145             logger.info("WASB trajectory CSV: %s", expected_csv_win)
146             return expected_csv_win
147 
148         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
149         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
150         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
151 
```

20. user (2026-06-10T05:35:32.933Z)

```
1       """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2 
3       WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4       the opponent for service after losing a point, that single trajectory looks like a
5       rally. The discriminator is structural, not duration-based: a real rally has the
6       shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7       single service feed crosses ~once.
8 
9       This module is camera-agnostic and needs NO new model ? it works on the trajectory
10      we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11      axis with the larger shuttle spread) and the net position (median shuttle coord on
12      that axis ? the shuttle clusters at the two ends and crosses the net region often,
13      so the median sits near the net), then counts sign changes of (coord - net) across
14      the visible points inside a candidate window, with a deadband to ignore jitter.
15 
16      Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17      Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18      windows from trajectory_to_action_windows, or to fold into it later.
19      """
20      from __future__ import annotations
21 
22      from dataclasses import dataclass
23      from typing import List, Sequence, Tuple
24 
25      Interval = Tuple[float, float]
26 
27 
```

```
    @dataclass
    class GatedWindow:
        start: float
        end: float
        crossings: int
        visible_points: int
        kept: bool


    def _spread(vals: Sequence[float]) -> float:
        """Robust spread = p95 - p5 (ignores a few outlier detections)."""
        if len(vals) < 2:
            return 0.0
        s = sorted(vals)
        lo = s[max(0, int(0.05 * (len(s) - 1)))]
        hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
        return hi - lo


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])


    def estimate_play_axis(points) -> Tuple[str, float, float]:
        """Return (axis 'x'|'y', net_value, span) from visible points.

        Play axis = the image axis along which the shuttle travels most (larger spread).
        For a baseline camera that's Y (players top/bottom); for a side camera, X.
        """
        xs = [p.x for p in points if p.visible]
        ys = [p.y for p in points if p.visible]
        sx, sy = _spread(xs), _spread(ys)
        if sx >= sy:
            return "x", _median(xs), sx
        return "y", _median(ys), sy


    def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
        """Count sign changes of (coord - net) across visible points, ignoring the
        deadband zone near the net (so jitter at the net doesn't inflate the count)."""
        crossings = 0
        last_side = None
        for p in window_points:
            if not p.visible:
                continue
            v = (p.x if axis == "x" else p.y) - net
            if abs(v) < deadband:
                continue
            side = v > 0
            if last_side is not None and side != last_side:
                crossings += 1
            last_side = side
        return crossings


    def _points_in(points, fps: float, start: float, end: float):
        return [p for p in points if start * fps <= p.frame <= end * fps]


    def gate_windows(points, windows: List[Interval], fps: float,
                     min_crossings: int = 2, deadband_frac: float = 0.06) ->
```

```
List[GatedWindow]:
93          """Annotate each (start,end) window with its shuttle net-crossing count and a
94          keep decision (crossings >= min_crossings). Deadband is a fraction of the play
95          axis span. Net/axis are estimated once over the whole trajectory (stable)."""
96          axis, net, span = estimate_play_axis(points)
97          deadband = deadband_frac * span
98          out: List[GatedWindow] = []
99          for (s, e) in windows:
100             wp = _points_in(points, fps, s, e)
101             c = count_net_crossings(wp, axis, net, deadband)
102             vis = sum(1 for p in wp if p.visible)
103             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
104                                    kept=c >= min_crossings))
105         return out
106
107
108     def apply_gate(points, windows: List[Interval], fps: float,
109                    min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
110         """Convenience: return only the windows that pass the crossing gate."""
111         return [(g.start, g.end) for g in
112                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
113
```

21. user (2026-06-10T05:35:48.355Z)

```
1       """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3       WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4       own building blocks (ImageDataset transform + detector + online tracker) to run on
5       any folder of extracted frames and emit a trajectory CSV in the exact shape the
6       indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8           Frame,Visibility,X,Y
9
10      It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11      imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12      (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14      Resumable across reboots
15      ------------------------
16      The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17      the slow, expensive part; the CPU tracker pass that turns detections into a
18      trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19      resume mid-stream). We exploit that split:
20
21        * The detector pass writes its per-window raw outputs incrementally to
22          ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23          records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24          most ``--flush-every`` windows of GPU work instead of the whole pass.
25        * On restart we replay the cached windows, skip the DataLoader ahead to the first
26          un-cached window, and continue. Once every window is cached, the detector pass
27          is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28          the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29          not the GPU hour.
30
31      The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32      to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33      weights, sport, window size, or frame count change.
34
35      Usage (in WSL, from ~/models/WASB-SBDT/src):
36          python wasb_infer.py --frames_dir /path/frames --weights
../pretrained_weights/wasb_badminton_best.pth.tar \
37              --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
every 1000] [--fresh]
38      """
```

```
    import os
    import os.path as osp
    import csv
    import glob
    import json
    import argparse
    import logging
    from collections import defaultdict

    logger = logging.getLogger(__name__)

    CACHE_VERSION = 1
    _DET_FILE = "det_raw.jsonl"
    _MANIFEST_FILE = "manifest.json"


    def _build_cfg(weights: str, sport: str):
        """Compose the WASB eval config via Hydra, overriding model/weights/device."""
        from hydra import compose, initialize
        with initialize(config_path="configs", version_base=None):
            cfg = compose(config_name="eval", overrides=[
                f"dataset={sport}",
                "model=wasb",
                f"detector.model_path={weights}",
                "runner.device=cuda",
                "runner.gpus=[0]",      # this box has a single GPU; default config assumes
more
            ])
        return cfg


    def _frame_id(path: str) -> int:
        """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
        stem = osp.splitext(osp.basename(path))[0]
        digits = "".join(ch for ch in stem if ch.isdigit())
        return int(digits) if digits else -1


    # ------------------------------------------------------------------------ #
    # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
    # ------------------------------------------------------------------------ #
    def _cache_paths(cache_dir: str):
        return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)


    def default_cache_dir(out_csv: str) -> str:
        """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
        d = osp.dirname(osp.abspath(out_csv))
        stem = osp.splitext(osp.basename(out_csv))[0]
        return osp.join(d, stem + "_wasbcache")


    def _atomic_write_text(path: str, text: str) -> None:
        """Write then rename so a crash mid-write can't leave a half-written file."""
        tmp = path + ".tmp"
        with open(tmp, "w") as f:
            f.write(text)
            f.flush()
            os.fsync(f.fileno())
        os.replace(tmp, path)


    def _det_plain(det) -> list:
        """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
```

```
list."""
        xy = det["xy"]
        return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]


    def _dets_to_tracker(plain_list):
        """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
        the tracker does vector math / np.linalg.norm on it)."""
        import numpy as np
        return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
                for p in plain_list]


    def write_manifest(cache_dir: str, manifest: dict) -> None:
        _, mpath = _cache_paths(cache_dir)
        _atomic_write_text(mpath, json.dumps(manifest, indent=2))


    def read_manifest(cache_dir: str):
        _, mpath = _cache_paths(cache_dir)
        if not osp.exists(mpath):
            return None
        try:
            with open(mpath) as f:
                return json.load(f)
        except (json.JSONDecodeError, OSError):
            return None


    def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
                            frames_in: int, frames_total: int) -> bool:
        """A cache is reusable only if the run that produced it matches this run's
inputs."""
        if not manifest or manifest.get("version") != CACHE_VERSION:
            return False
        return (
            manifest.get("frames_dir") == osp.abspath(frames_dir)
            and manifest.get("weights") == weights
            and manifest.get("sport") == sport
            and manifest.get("frames_in") == frames_in
            and manifest.get("frames_total") == frames_total
        )


    def reset_cache(cache_dir: str) -> None:
        """Drop any existing cache so the next run starts clean."""
        det_jsonl, mpath = _cache_paths(cache_dir)
        for p in (det_jsonl, mpath):
            if osp.exists(p):
                os.remove(p)


    def load_and_clean_cache(cache_dir: str):
        """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
        rebuild the per-frame detection accumulation, and rewrite the file to exactly that
        prefix so a trailing half-written line from a crash can't corrupt future appends.

        Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
        [x, y, score, scale] candidates (accumulated across every window the frame is in,
        in window order ? identical to a non-resumed run).
        """
        det_jsonl, _ = _cache_paths(cache_dir)
        det_by_fid = defaultdict(list)
        valid = []
```

```
        if osp.exists(det_jsonl):
            with open(det_jsonl, "r") as f:
                for line in f:
                    s = line.strip()
                    if not s:
                        continue
                    try:
                        obj = json.loads(s)
                    except json.JSONDecodeError:
                        break                       # truncated trailing line (crash mid-
flush)
                    if obj.get("w") != len(valid):  # non-contiguous -> stop at the gap
                        break
                    valid.append(s)
                    for fid, plain in obj["f"]:
                        det_by_fid[int(fid)].extend([list(p) for p in plain])
            # Guarantee a clean append boundary by rewriting only the good prefix.
            _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
        return len(valid), det_by_fid


    def _append_windows(fh, objs) -> None:
        """Append window records as JSONL and durably flush (bounded loss on crash)."""
        for o in objs:
            fh.write(json.dumps(o, separators=(",", ":")) + "\n")
        fh.flush()
        os.fsync(fh.fileno())


    def _atomic_write_trajectory(out_csv: str, rows) -> None:
        os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
        tmp = out_csv + ".tmp"
        with open(tmp, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["Frame", "Visibility", "X", "Y"])
            w.writerows(rows)
            f.flush()
            os.fsync(f.fileno())
        os.replace(tmp, out_csv)


    # ------------------------------------------------------------------------ #
    # Inference
    # ------------------------------------------------------------------------ #
    def run(frames_dir: str, weights: str, out_csv: str, sport: str = "badminton",
            limit: int = 0, batch_size: int = 8, num_workers: int = 4,
            log_every_batches: int = 50, cache_dir: str = None,
            flush_every: int = 1000, fresh: bool = False) -> str:
        import time
        import torch
        from torch.utils.data import DataLoader
        from detectors import build_detector
        from trackers import build_tracker
        from dataloaders import build_img_transforms, build_seq_transforms
        from dataloaders.dataset_loader import ImageDataset
        from utils import Center

        cfg = _build_cfg(weights, sport)
        frames_in = int(cfg["model"]["frames_in"])
        input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
        output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))

        frames = sorted(glob.glob(osp.join(frames_dir, "*.png")) +
glob.glob(osp.join(frames_dir, "*.jpg")))
        if limit:
```

```
            frames = frames[:limit]
        if len(frames) < frames_in:
            raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in
{frames_dir}")

        # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
        samples = []
        for i in range(len(frames) - frames_in + 1):
            window = frames[i:i + frames_in]
            annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
            samples.append({"frames": window, "annos": annos})
        windows_total = len(samples)

        # ---- cache setup / resume decision ---------------------------------- #
        if cache_dir is None:
            cache_dir = default_cache_dir(out_csv)
        os.makedirs(cache_dir, exist_ok=True)
        det_jsonl, _ = _cache_paths(cache_dir)
        manifest = None if fresh else read_manifest(cache_dir)
        resume = manifest_compatible(
            manifest or {}, frames_dir=frames_dir, weights=weights, sport=sport,
            frames_in=frames_in, frames_total=len(frames),
        )
        if resume:
            windows_done, det_by_fid = load_and_clean_cache(cache_dir)
            logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
        else:
            if manifest is not None:
                logger.info("cache incompatible with this run -> starting fresh")
            reset_cache(cache_dir)
            windows_done, det_by_fid = 0, defaultdict(list)

        base_manifest = {
            "version": CACHE_VERSION,
            "frames_dir": osp.abspath(frames_dir),
            "weights": weights,
            "sport": sport,
            "frames_in": frames_in,
            "frames_total": len(frames),
            "windows_total": windows_total,
            "windows_done": windows_done,
        }
        write_manifest(cache_dir, base_manifest)

        # ---- detector pass over the un-cached windows ----------------------- #
        remaining = samples[windows_done:]
        if remaining:
            _, transform_test = build_img_transforms(cfg)
            try:
                _, seq_transform_test = build_seq_transforms(cfg)
            except Exception:
                seq_transform_test = None
            ds = ImageDataset(cfg, remaining, input_wh, output_wh,
                              transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
            loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=num_workers)
            detector = build_detector(cfg)

            t0 = time.time()
            done = windows_done
            win_idx = windows_done
```

```
            log_every = max(1, log_every_batches)
            flush_n = max(1, flush_every)
            pending = []
            logger.info(f"detecting on {len(remaining)} remaining windows "
                        f"(total {windows_total}, batch={batch_size},
workers={num_workers})...")
            det_f = open(det_jsonl, "a")
            try:
                with torch.no_grad():
                    for bi, batch in enumerate(loader):
                        imgs, _hms, trans = batch[0], batch[1], batch[2]
                        img_paths = [list(t) for t in batch[-1]]   # [frame_in][batch] ->
path
                        batch_results, _ = detector.run_tensor(imgs, trans)
                        nb = imgs.shape[0]
                        for ib in range(nb):
                            frames_payload = []
                            for ie in sorted(batch_results[ib].keys()):
                                p = img_paths[ie][ib]
                                fid = _frame_id(p)
                                plain = [_det_plain(d) for d in batch_results[ib][ie]]
                                frames_payload.append([fid, plain])
                                det_by_fid[fid].extend(plain)
                            pending.append({"w": win_idx, "f": frames_payload})
                            win_idx += 1
                        done += nb
                        if len(pending) >= flush_n or done >= windows_total:
                            _append_windows(det_f, pending)
                            pending = []
                            base_manifest["windows_done"] = done
                            write_manifest(cache_dir, base_manifest)
                        if (bi + 1) % log_every == 0 or done >= windows_total:
                            el = time.time() - t0
                            new_done = done - windows_done
                            rate = new_done / el if el > 0 else 0.0
                            eta = (windows_total - done) / rate if rate > 0 else 0.0
                            logger.info(f"{done}/{windows_total} "
                                        f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
                                        f"elapsed {el:.0f}s | ETA {eta:.0f}s")
                    if pending:
                        _append_windows(det_f, pending)
                        base_manifest["windows_done"] = done
                        write_manifest(cache_dir, base_manifest)
            finally:
                det_f.close()
            logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
        else:
            logger.info("detector cache already complete -> tracker-only re-run")

        # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
        tracker = build_tracker(cfg)
        tracker.refresh()
        rows = []
        for fid in sorted(det_by_fid.keys()):
            r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
            rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))

        _atomic_write_trajectory(out_csv, rows)
        visible = sum(1 for _, v, *_ in rows if v)
        logger.info(f"wrote {len(rows)} rows -> {out_csv}")
        logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
        return out_csv


```

```
    def extract_frames(video_path: str, frames_dir: str) -> str:
        """Decode a video to PNG frames named by source frame index (00000000.png ...).

        Idempotent: if the dir already holds the expected number of frames (matching the
        container's reported frame count) we skip re-decoding ? re-extraction after a
        reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
        extracting with ffmpeg on the host and passing --frames_dir.
        """
        import cv2
        os.makedirs(frames_dir, exist_ok=True)
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            raise SystemExit(f"cannot open video: {video_path}")
        expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
        existing = len(glob.glob(osp.join(frames_dir, "*.png")))
        if expected > 0 and existing >= expected:
            cap.release()
            logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
            return frames_dir
        i = 0
        while True:
            ok, frame = cap.read()
            if not ok:
                break
            cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
            i += 1
        cap.release()
        logger.info(f"extracted {i} frames -> {frames_dir}")
        return frames_dir


    def main():
        ap = argparse.ArgumentParser()
        ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
        ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
        ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
        ap.add_argument("--weights", required=True)
        ap.add_argument("--out", required=True)
        ap.add_argument("--sport", default="badminton")
        ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
        ap.add_argument("--batch-size", type=int, default=8)
        ap.add_argument("--num-workers", type=int, default=4)
        ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
        ap.add_argument("--cache-dir", default=None,
                        help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
        ap.add_argument("--flush-every", type=int, default=1000,
                        help="fsync the detector cache every N windows (bounds reboot
loss)")
        ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
        args = ap.parse_args()

        frames_dir = args.frames_dir
        if args.video:
            frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
        if not frames_dir:
            raise SystemExit("provide --frames_dir or --video")
        run(frames_dir, args.weights, args.out, args.sport, args.limit,
            batch_size=args.batch_size, num_workers=args.num_workers,
```

```
405             log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
406             flush_every=args.flush_every, fresh=args.fresh)
407
408
409     if __name__ == "__main__":
410         main()
411
```

22. assistant (2026-06-10T05:36:05.802Z)

Now validators, database, and stitcher.

23. user (2026-06-10T05:36:11.009Z)

```
1       from abc import ABC, abstractmethod
2       from pydantic import BaseModel
3       from typing import List, Dict, Any, Optional
4
5       class ValidationResult(BaseModel):
6           validator_name: str
7           passed: bool
8           message: str
9           details: Dict[str, Any] = {}
10
11      class VideoValidator(ABC):
12          @property
13          @abstractmethod
14          def name(self) -> str:
15              """Unique identifier for this validator."""
16              pass
17
18          @property
19          @abstractmethod
20          def description(self) -> str:
21              """Human-readable explanation of what this validator checks."""
22              pass
23
24          @abstractmethod
25          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
26              """
27              Executes validation.
28              - video_path: Local path to the MP4 file.
29              - config: Dict parsed from config.json.
30              - context: Dynamic context dict shared across validators (stores shared values
like ffprobe metadata).
31              """
32              pass
33
34      class ValidationRegistry:
35          def __init__(self):
36              self._validators: List[VideoValidator] = []
37
38          def register(self, validator: VideoValidator):
39              """Registers a new validator."""
40              self._validators.append(validator)
41
42          def run_all_with_context(self, video_path: str, config: Dict[str, Any]):
43              """Runs all validators and returns ``(results, context)``.
44
45              The per-run ``context`` accumulates shared values (notably
46              ``context['ffprobe_meta']`` produced by FormatValidator). ``run_all``
47              discards it; callers that want media metadata for an observability report
48              use this variant instead of re-probing.
49              """
```

```
            results = []
            context = {} # Shared context dictionary (e.g. for caching ffprobe outputs)
            for val in self._validators:
                try:
                    res = val.validate(video_path, config, context)
                    results.append(res)
                except Exception as e:
                    results.append(ValidationResult(
                        validator_name=val.name,
                        passed=False,
                        message=f"Validator threw an unexpected error: {str(e)}"
                    ))
            return results, context

        def run_all(self, video_path: str, config: Dict[str, Any]) ->
List[ValidationResult]:
            """Runs all registered validators in sequence, accumulating results."""
            results, _ = self.run_all_with_context(video_path, config)
            return results

    # Shared global registry
    registry = ValidationRegistry()

```

24. user (2026-06-10T05:36:11.089Z)

```
    import sqlite3
    import json
    import os
    import logging
    from typing import List, Dict, Any, Optional

    logger = logging.getLogger(__name__)

    class Database:
        def __init__(self, db_path: str):
            self.db_path = db_path
            self._init_db()

        def _get_connection(self):
            conn = sqlite3.connect(self.db_path)
            conn.row_factory = sqlite3.Row
            # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
            conn.execute("PRAGMA foreign_keys = ON")
            return conn

        def _init_db(self):
            """Initializes tables for videos, play_segments, and events."""
            with self._get_connection() as conn:
                cursor = conn.cursor()

                # Videos Table
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS videos (
                        id TEXT PRIMARY KEY,
                        filepath TEXT NOT NULL,
                        processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                        status TEXT NOT NULL,
                        validation_results TEXT
                    )
                """)

                # Play Segments Table (Extensible metadata JSON)
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS play_segments (
```

```
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        video_id TEXT NOT NULL,
                        start_time REAL NOT NULL,
                        end_time REAL NOT NULL,
                        duration REAL NOT NULL,
                        server TEXT,
                        receiver TEXT,
                        metadata TEXT,
                        FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
                    )
                """)

                # Events Table
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS events (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        segment_id INTEGER NOT NULL,
                        timestamp REAL NOT NULL,
                        type TEXT NOT NULL,
                        player TEXT,
                        details TEXT,
                        FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
CASCADE
                    )
                """)

                # Versioned migrations live here. PRAGMA user_version is the schema
                # contract ? bump it for every change and add an ordered `if version < N`
                # block below. Tests assert the expected version, so accidental drift fails
CI.
                version = cursor.execute("PRAGMA user_version").fetchone()[0]

                if version < 1:
                    # v1: raw candidate detections (pre-confirmation), detector-agnostic.
                    # Common fields are columns; detector-specific metrics go in `stats`
JSON,
                    # so a new segmenter reuses this table by setting its own
`detector`/`stats`.
                    cursor.execute("""
                        CREATE TABLE IF NOT EXISTS candidate_segments (
                            id INTEGER PRIMARY KEY AUTOINCREMENT,
                            video_id TEXT NOT NULL,
                            detector TEXT NOT NULL,
                            chunk_index INTEGER,
                            start_time REAL NOT NULL,
                            end_time REAL NOT NULL,
                            duration REAL NOT NULL,
                            confidence REAL,
                            stats TEXT,
                            detection_params TEXT,
                            confirmed_segment_id INTEGER,
                            human_label TEXT,
                            notes TEXT,
                            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                            FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
                            FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
                        )
                    """)
                    cursor.execute("""
                        CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
                            ON candidate_segments(video_id, detector)
                    """)
                    cursor.execute("PRAGMA user_version = 1")
                # future: if version < 2: cursor.execute("ALTER TABLE ..."); PRAGMA
```

```
user_version = 2

                conn.commit()

        def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
            try:
                with self._get_connection() as conn:
                    conn.cursor().execute(
                        "INSERT OR REPLACE INTO videos (id, filepath, status,
validation_results) VALUES (?, ?, ?, ?)",
                        (video_id, filepath, status, json.dumps(validation_results))
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to add video: {e}")
                return False

        def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
            try:
                with self._get_connection() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
                        (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add play segment: {e}")
                return None

        def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
            try:
                with self._get_connection() as conn:
                    conn.cursor().execute(
                        "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
                        (segment_id, timestamp, event_type, player, json.dumps(details or
{}))
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to add event: {e}")
                return False

        def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
                                  chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
                                  stats: Optional[Dict[str, Any]] = None,
                                  detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
            """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
            detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
            try:
                with self._get_connection() as conn:
```

```
                    cursor = conn.cursor()
                    cursor.execute(
                        """INSERT INTO candidate_segments
                           (video_id, detector, chunk_index, start_time, end_time, duration,
                            confidence, stats, detection_params)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
                        (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
                         confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add candidate segment: {e}")
                return None

        def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
            """Record that a candidate detection was confirmed as a given play_segment."""
            try:
                with self._get_connection() as conn:
                    conn.cursor().execute(
                        "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
                        (segment_id, candidate_id)
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
                return False

        def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
                                     only_unlabeled: bool = False) -> List[Dict[str, Any]]:
            """Fetch candidate detections for the annotation / fine-tuning workflow.
            `stats` and `detection_params` are deserialized from JSON."""
            query = "SELECT * FROM candidate_segments WHERE video_id = ?"
            params: List[Any] = [video_id]

            if detector:
                query += " AND detector = ?"
                params.append(detector)
            if only_unlabeled:
                query += " AND human_label IS NULL"

            query += " ORDER BY start_time"

            candidates = []
            with self._get_connection() as conn:
                rows = conn.cursor().execute(query, params).fetchall()
                for row in rows:
                    d = dict(row)
                    d["stats"] = json.loads(d["stats"] or "{}")
                    d["detection_params"] = json.loads(d["detection_params"] or "{}")
                    candidates.append(d)
            return candidates

        def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
            with self._get_connection() as conn:
                row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
                if row:
                    d = dict(row)
                    d["validation_results"] = json.loads(d["validation_results"] or "[]")
```

```
                    return d
            return None

        def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
            """
            Dynamically queries play segments based on filters:
            - duration_min: float
            - server: string
            - receiver: string
            - event_type: string (e.g. smash, foul)
            """
            query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
            params = [video_id]

            if filters.get("duration_min"):
                query += " AND r.duration >= ?"
                params.append(filters["duration_min"])

            if filters.get("server"):
                query += " AND r.server = ?"
                params.append(filters["server"])

            if filters.get("receiver"):
                query += " AND r.receiver = ?"
                params.append(filters["receiver"])

            if filters.get("event_type"):
                # Check if there is an event of a specific type inside this segment
                query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
                params.append(filters["event_type"])

            segments_list = []
            with self._get_connection() as conn:
                cursor = conn.cursor()
                rows = cursor.execute(query, params).fetchall()

                for row in rows:
                    r_dict = dict(row)
                    r_id = r_dict["id"]
                    r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")

                    # Fetch associated events
                    event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
                    r_dict["events"] = []
                    for er in event_rows:
                        e_dict = dict(er)
                        e_dict["details"] = json.loads(e_dict["details"] or "{}")
                        r_dict["events"].append(e_dict)

                    segments_list.append(r_dict)

            return segments_list

        def clear_video_data(self, video_id: str):
            with self._get_connection() as conn:
                conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
                conn.commit()

```

25. user (2026-06-10T05:36:27.807Z)

```
1       import subprocess
```

```
2       import json
3       import os
4       from backend.pipeline.validators.base import VideoValidator, ValidationResult
5       from typing import Dict, Any
6
7       class FormatValidator(VideoValidator):
8           @property
9           def name(self) -> str:
10              return "format_check"
11
12          @property
13          def description(self) -> str:
14              return "Verifies the video is a valid, uncorrupted MP4 container and extracts
metadata."
15
16          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
17              if not os.path.exists(video_path):
18                  return ValidationResult(
19                      validator_name=self.name,
20                      passed=False,
21                      message=f"File not found: {video_path}"
22                  )
23
24              cmd = [
25                  "ffprobe",
26                  "-v", "error",
27                  "-select_streams", "v:0",
28                  "-show_format",
29                  "-show_streams",
30                  "-of", "json",
31                  video_path
32              ]
33
34              try:
35                  result = subprocess.run(cmd, capture_output=True, text=True, check=True)
36                  info = json.loads(result.stdout)
37
38                  if not info.get("streams") or not info.get("format"):
39                      return ValidationResult(
40                          validator_name=self.name,
41                          passed=False,
42                          message="Failed to retrieve valid video streams from the file."
43                      )
44
45                  # Store the metadata in context for subsequent validators to utilize
46                  context["ffprobe_meta"] = info
47
48                  stream = info["streams"][0]
49                  fmt = info["format"]
50
51                  details = {
52                      "format_name": fmt.get("format_name"),
53                      "duration": float(fmt.get("duration", 0)),
54                      "size_bytes": int(fmt.get("size", 0)),
55                      "codec_name": stream.get("codec_name"),
56                  }
57
58                  # Check if container name includes mp4
59                  if "mp4" not in fmt.get("format_name", "").lower():
60                      return ValidationResult(
61                          validator_name=self.name,
62                          passed=False,
63                          message=f"Invalid format: Found '{fmt.get('format_name')}', but MP4
is required.",
```

```
                        details=details
                    )

                return ValidationResult(
                    validator_name=self.name,
                    passed=True,
                    message="Video format is valid MP4 container.",
                    details=details
                )

            except subprocess.CalledProcessError as e:
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message="Video stream is corrupted or unreadable by ffprobe.",
                    details={"error": e.stderr}
                )
            except Exception as e:
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message=f"Unexpected error running ffprobe: {str(e)}"
                )

```

26. user (2026-06-10T05:36:28.284Z)

```
    from backend.pipeline.validators.base import VideoValidator, ValidationResult
    from typing import Dict, Any

    class ResolutionFPSValidator(VideoValidator):
        @property
        def name(self) -> str:
            return "resolution_fps_check"

        @property
        def description(self) -> str:
            return "Checks if the video meets minimum resolution (e.g. 720p) and frame rate
(e.g. 30fps) requirements."

        def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
            meta = context.get("ffprobe_meta")
            if not meta or not meta.get("streams"):
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message="Video metadata is missing. Run format_check validator first."
                )

            stream = meta["streams"][0]
            width = int(stream.get("width", 0))
            height = int(stream.get("height", 0))

            # Parse frame rate (e.g. "30/1" or "30000/1001")
            fps_str = stream.get("r_frame_rate", "0/1")
            try:
                num, den = map(int, fps_str.split("/"))
                fps = num / den if den != 0 else 0
            except ValueError:
                fps = 0.0

            details = {
                "width": width,
                "height": height,
```

```
                "fps": round(fps, 2)
            }

            # Check bounds from config
            val_cfg = config.get("validation", {})
            min_w = val_cfg.get("min_resolution_width", 1280)
            min_h = val_cfg.get("min_resolution_height", 720)
            min_fps = val_cfg.get("min_fps", 29.9)

            if width < min_w or height < min_h:
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message=f"Low resolution: Found {width}x{height}, but minimum is
{min_w}x{min_h}.",
                    details=details
                )

            if fps < min_fps:
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message=f"Low frame rate: Found {round(fps, 2)} FPS, but minimum is
{min_fps} FPS.",
                    details=details
                )

            return ValidationResult(
                validator_name=self.name,
                passed=True,
                message="Video resolution and frame rate meet minimum standards.",
                details=details
            )

```

27. user (2026-06-10T05:36:28.653Z)

```
from backend.pipeline.validators.base import registry, VideoValidator, ValidationResult
from backend.pipeline.validators.format import FormatValidator
from backend.pipeline.validators.resolution_fps import ResolutionFPSValidator
from backend.pipeline.validators.pts_continuity import PTSContinuityValidator
from backend.pipeline.validators.scene_cut import SceneCutDensityValidator
from backend.pipeline.validators.blur import BlurValidator
from backend.pipeline.validators.relevance import RelevanceValidator

# Register all default validators in sequence (dependencies first)
registry.register(FormatValidator())
registry.register(ResolutionFPSValidator())
registry.register(PTSContinuityValidator())
registry.register(SceneCutDensityValidator())
registry.register(BlurValidator())
registry.register(RelevanceValidator())

__all__ = ["registry", "VideoValidator", "ValidationResult"]

```

28. user (2026-06-10T05:36:36.851Z)

```
import subprocess
from backend.pipeline.validators.base import VideoValidator, ValidationResult
from typing import Dict, Any

class PTSContinuityValidator(VideoValidator):
    @property
    def name(self) -> str:
```

```
8               return "pts_continuity_check"
9
10          @property
11          def description(self) -> str:
12              return "Analyzes Presentation Time Stamps (PTS) of keyframes to check for
timeline gaps, jumps, or editing splices."
13
14          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
15              # Command to extract packet PTS and flags (extremely fast, works on H.264 &
H.265)
16              cmd = [
17                  "ffprobe",
18                  "-v", "error",
19                  "-select_streams", "v:0",
20                  "-show_entries", "packet=pts_time,flags",
21                  "-of", "csv=p=0",
22                  video_path
23              ]
24
25              try:
26                  result = subprocess.run(cmd, capture_output=True, text=True, check=True)
27                  output = result.stdout.strip()
28
29                  if not output:
30                      return ValidationResult(
31                          validator_name=self.name,
32                          passed=False,
33                          message="No packet timestamps could be extracted from the video
stream."
34                      )
35
36                  timestamps = []
37                  for line in output.split("\n"):
38                      line = line.strip()
39                      if line:
40                          parts = line.split(",")
41                          if len(parts) == 2:
42                              pts_str, flags = parts
43                              if "K" in flags: # Check if keyframe
44                                  try:
45                                      timestamps.append(float(pts_str))
46                                  except ValueError:
47                                      continue
48
49                  if len(timestamps) < 2:
50                      return ValidationResult(
51                          validator_name=self.name,
52                          passed=True,
53                          message="Video is too short to perform a PTS continuity check.",
54                          details={"keyframes_count": len(timestamps)}
55                      )
56
57                  # Analyze gaps
58                  max_gap = 0.0
59                  backward_jumps = 0
60                  gaps = []
61
62                  for i in range(1, len(timestamps)):
63                      gap = timestamps[i] - timestamps[i-1]
64                      gaps.append(gap)
65                      if gap < 0:
66                          backward_jumps += 1
67                      if gap > max_gap:
68                          max_gap = gap
```

```
69
70                  # An continuous GoPro recording usually has keyframe intervals (GOP size) of
1.0 to 4.0 seconds.
71                  # If there's a keyframe gap larger than 8.0 seconds or a negative gap
(backward jump), it's highly likely spliced.
72                  details = {
73                      "keyframes_count": len(timestamps),
74                      "max_keyframe_gap_seconds": round(max_gap, 2),
75                      "backward_jumps_detected": backward_jumps,
76                      "first_keyframe": timestamps[0],
77                      "last_keyframe": timestamps[-1]
78                  }
79
80                  if backward_jumps > 0:
81                      return ValidationResult(
82                          validator_name=self.name,
83                          passed=False,
84                          message="Video integrity compromise: Backward timeline jumps
(splicing) detected.",
85                          details=details
86                      )
87
88                  # Threshold for sudden gap (e.g. 10.0 seconds)
89                  if max_gap > 10.0:
90                      return ValidationResult(
91                          validator_name=self.name,
92                          passed=False,
93                          message=f"Video integrity compromise: Detected a timestamp gap of
{round(max_gap, 2)}s, indicating edited or missing content.",
94                          details=details
95                      )
96
97                  return ValidationResult(
98                      validator_name=self.name,
99                      passed=True,
100                     message="PTS timeline is continuous with no abnormal gaps or backward
jumps.",
101                     details=details
102                 )
103
104             except subprocess.CalledProcessError as e:
105                 return ValidationResult(
106                     validator_name=self.name,
107                     passed=False,
108                     message="Failed to execute ffprobe PTS analysis.",
109                     details={"error": e.stderr}
110                 )
111             except Exception as e:
112                 return ValidationResult(
113                     validator_name=self.name,
114                     passed=False,
115                     message=f"Unexpected error in PTS validation: {str(e)}"
116                 )
117
```

29. user (2026-06-10T05:36:37.242Z)

```
1       import cv2
2       import numpy as np
3       from backend.pipeline.validators.base import VideoValidator, ValidationResult
4       from typing import Dict, Any
5
6       class SceneCutDensityValidator(VideoValidator):
7           @property
8           def name(self) -> str:
```

```
9               return "scene_cut_check"
10
11          @property
12          def description(self) -> str:
13              return "Analyzes visual frames to detect abrupt scene cuts, which indicate an
edited or compiled video."
14
15          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
16              cap = cv2.VideoCapture(video_path)
17              if not cap.isOpened():
18                  return ValidationResult(
19                      validator_name=self.name,
20                      passed=False,
21                      message="Failed to open video file using OpenCV."
22                  )
23
24              fps = cap.get(cv2.CAP_PROP_FPS)
25              total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
26              duration = total_frames / fps if fps > 0 else 0
27
28              if duration <= 0:
29                  cap.release()
30                  return ValidationResult(
31                      validator_name=self.name,
32                      passed=False,
33                      message="Invalid video duration."
34                  )
35
36              # Sample at 2 frames per second to keep it fast
37              sample_interval = max(1, int(fps / 2))
38
39              cuts_detected = 0
40              prev_hist = None
41
42              frame_idx = 0
43              sampled_count = 0
44
45              # Adjust histogram comparison thresholds
46              # Using correlation: 1.0 is perfect match, < 0.6 indicates a sudden massive
scene change
47              cut_threshold = 0.6
48
49              while True:
50                  cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
51                  ret, frame = cap.read()
52                  if not ret:
53                      break
54
55                  # Calculate color histogram
56                  hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
57                  hist = cv2.calcHist([hsv], [0, 1], None, [50, 60], [0, 180, 0, 256])
58                  cv2.normalize(hist, hist, alpha=0, beta=1, norm_type=cv2.NORM_MINMAX)
59
60                  if prev_hist is not None:
61                      # Compare correlation
62                      score = cv2.compareHist(prev_hist, hist, cv2.HISTCMP_CORREL)
63                      if score < cut_threshold:
64                          cuts_detected += 1
65
66                  prev_hist = hist
67                  sampled_count += 1
68                  frame_idx += sample_interval
69
70                  # Cap check at 100 frames max (e.g. first 50 seconds or evenly spread) to
```

```
keep validation instant
71                  if frame_idx >= total_frames or sampled_count > 100:
72                      break
73
74              cap.release()
75
76              # Compute cuts per minute
77              sampled_duration = (sampled_count * sample_interval) / fps if fps > 0 else 1
78              cuts_per_minute = (cuts_detected / sampled_duration) * 60 if sampled_duration >
0 else 0
79
80              details = {
81                  "cuts_detected": cuts_detected,
82                  "sampled_frames": sampled_count,
83                  "cuts_per_minute": round(cuts_per_minute, 3)
84              }
85
86              val_cfg = config.get("validation", {})
87              max_cuts_rate = val_cfg.get("max_scene_cuts_per_minute", 0.5)
88
89              if cuts_per_minute > max_cuts_rate:
90                  return ValidationResult(
91                      validator_name=self.name,
92                      passed=False,
93                      message=f"Video splicing compromise: Detected {cuts_detected} cuts
({round(cuts_per_minute, 2)} cuts/min), which exceeds the continuous raw file limit of
{max_cuts_rate} cuts/min.",
94                      details=details
95                  )
96
97              return ValidationResult(
98                  validator_name=self.name,
99                  passed=True,
100                 message="Scene continuity check passed. No abnormal visual cuts detected.",
101                 details=details
102             )
103
```

30. user (2026-06-10T05:36:42.209Z)

```
1       import cv2
2       from backend.pipeline.validators.base import VideoValidator, ValidationResult
3       from typing import Dict, Any
4
5       class BlurValidator(VideoValidator):
6           @property
7           def name(self) -> str:
8               return "blur_check"
9
10          @property
11          def description(self) -> str:
12              return "Samples frames across the video and checks if the footage is excessively
blurry or out of focus."
13
14          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
15              cap = cv2.VideoCapture(video_path)
16              if not cap.isOpened():
17                  return ValidationResult(
18                      validator_name=self.name,
19                      passed=False,
20                      message="Failed to open video file for blur analysis."
21                  )
22
23              total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
```

```
            if total_frames <= 0:
                cap.release()
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message="Invalid frame count."
                )

            # Sample 15 frames evenly distributed across the video
            num_samples = 15
            step = max(1, total_frames // num_samples)

            variances = []

            for i in range(num_samples):
                frame_idx = i * step
                cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
                ret, frame = cap.read()
                if not ret:
                    continue

                gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
                # Calculate the Laplacian variance (standard focus metric)
                laplacian_var = cv2.Laplacian(gray, cv2.CV_64F).var()
                variances.append(laplacian_var)

            cap.release()

            if not variances:
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message="Could not read frames to perform blur verification."
                )

            avg_variance = sum(variances) / len(variances)

            details = {
                "avg_laplacian_variance": round(avg_variance, 2),
                "min_laplacian_variance": round(min(variances), 2),
                "max_laplacian_variance": round(max(variances), 2),
                "sampled_frames_count": len(variances)
            }

            val_cfg = config.get("validation", {})
            min_blur_threshold = val_cfg.get("min_blur_threshold", 100.0)

            if avg_variance < min_blur_threshold:
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message=f"Video quality rejected: Focus check failed. Average sharpness
score of {round(avg_variance, 1)} is below the minimum required threshold of
{min_blur_threshold}.",
                    details=details
                )

            return ValidationResult(
                validator_name=self.name,
                passed=True,
                message="Video sharpness meets focus requirements.",
                details=details
            )

```

31. user (2026-06-10T05:36:42.391Z)

```
1       import cv2
2       import numpy as np
3       from backend.pipeline.validators.base import VideoValidator, ValidationResult
4       from typing import Dict, Any
5
6       class RelevanceValidator(VideoValidator):
7           @property
8           def name(self) -> str:
9               return "relevance_check"
10
11          @property
12          def description(self) -> str:
13              return "Checks if the video content is relevant to badminton by scanning for
court color palettes and geometric features."
14
15          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
16              cap = cv2.VideoCapture(video_path)
17              if not cap.isOpened():
18                  return ValidationResult(
19                      validator_name=self.name,
20                      passed=False,
21                      message="Failed to open video file for relevance validation."
22                  )
23
24              total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
25              if total_frames <= 0:
26                  cap.release()
27                  return ValidationResult(
28                      validator_name=self.name,
29                      passed=False,
30                      message="Invalid frame count."
31                  )
32
33              # Sample 10 frames across the video
34              num_samples = 10
35              step = max(1, total_frames // num_samples)
36
37              green_ratios = []
38
39              # Get sport profile config
40              sport_name = config.get("sport", "badminton")
41              from backend.sports import sport_registry
42              try:
43                  sport_profile = sport_registry.get(sport_name)
44                  sport_rel_cfg = sport_profile.get_relevance_config()
45              except KeyError:
46                  sport_rel_cfg = {}
47
48              val_cfg = config.get("validation", {})
49              rel_cfg = val_cfg.get("relevance", {})
50
51              # Read bounds from sport profile with fallback to user config/defaults
52              hsv_lower = np.array(rel_cfg.get("court_green_lower",
sport_rel_cfg.get("court_green_lower", [35, 40, 40])))
53              hsv_upper = np.array(rel_cfg.get("court_green_upper",
sport_rel_cfg.get("court_green_upper", [85, 255, 255])))
54              min_court_ratio = rel_cfg.get("court_pixels_min_ratio",
sport_rel_cfg.get("court_pixels_min_ratio", 0.05))
55
56              for i in range(num_samples):
57                  frame_idx = i * step
58                  cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
```

```
59                  ret, frame = cap.read()
60                  if not ret:
61                      continue
62
63                  # Downscale frame for speed
64                  h, w = frame.shape[:2]
65                  small_frame = cv2.resize(frame, (320, 240))
66
67                  # Convert to HSV color space
68                  hsv = cv2.cvtColor(small_frame, cv2.COLOR_BGR2HSV)
69
70                  # Mask court green pixels
71                  mask = cv2.inRange(hsv, hsv_lower, hsv_upper)
72                  green_pixels = cv2.countNonZero(mask)
73                  total_pixels = mask.shape[0] * mask.shape[1]
74
75                  ratio = green_pixels / total_pixels
76                  green_ratios.append(ratio)
77
78              cap.release()
79
80              if not green_ratios:
81                  return ValidationResult(
82                      validator_name=self.name,
83                      passed=False,
84                      message="Failed to extract frames for relevance checking."
85                  )
86
87              avg_green_ratio = sum(green_ratios) / len(green_ratios)
88
89              details = {
90                  "avg_green_court_ratio": round(avg_green_ratio, 4),
91                  "max_green_court_ratio": round(max(green_ratios), 4),
92                  "min_green_court_ratio": round(min(green_ratios), 4),
93                  "green_threshold": min_court_ratio
94              }
95
96              # If a certain amount of court color is visible, consider it relevant
97              if avg_green_ratio < min_court_ratio:
98                  return ValidationResult(
99                      validator_name=self.name,
100                     passed=False,
101                     message=f"Video rejected: Relevance check failed. The scene does not
contain a recognizable {sport_name} court (court area found: {round(avg_green_ratio * 100, 2)}%
vs min required {round(min_court_ratio * 100, 2)}%).",
102                     details=details
103                 )
104
105             return ValidationResult(
106                 validator_name=self.name,
107                 passed=True,
108                 message=f"Video identified as relevant to {sport_name} ({sport_name} court
structures found).",
109                 details=details
110             )
111
```

32. assistant (2026-06-10T05:36:58.842Z)

Now the stitcher, audio detector, and utils.

33. user (2026-06-10T05:36:59.791Z)

```
1       import os
2       import subprocess
```

```
3       import tempfile
4       import logging
5       from typing import List, Tuple, Union
6
7       logger = logging.getLogger(__name__)
8
9       class VideoCompiler:
10          def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5):
11              self.output_dir = output_dir
12              self.end_padding = end_padding
13              self.begin_padding = begin_padding
14              os.makedirs(output_dir, exist_ok=True)
15
16          @staticmethod
17          def _parse_time(val: Union[int, float, str]) -> float:
18              """Normalizes a timestamp value to float seconds.
19              Handles: float/int (pass-through), plain numeric strings ("12.4"),
20              and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
21              This mirrors the parse_time logic used in gemini.py to ensure
22              the stitcher can always consume whatever format the indexer produced."""
23              if isinstance(val, (int, float)):
24                  return float(val)
25              if isinstance(val, str):
26                  val = val.strip()
27                  if ":" in val:
28                      # Handle MM:SS, HH:MM:SS, etc.
29                      parts = val.split(":")
30                      parts.reverse()
31                      total = 0.0
32                      for i, p in enumerate(parts):
33                          try:
34                              total += float(p) * (60 ** i)
35                          except ValueError:
36                              pass
37                      return total
38                  try:
39                      return float(val)
40                  except ValueError:
41                      logger.warning(f"Could not parse timestamp value '{val}' as a number.
Defaulting to 0.0.")
42                      return 0.0
43              logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
44              return 0.0
45
46          def _get_video_duration(self, video_path: str) -> float:
47              """Probes the video file to get its total duration in seconds.
48              Returns 0.0 if it cannot be determined."""
49              try:
50                  result = subprocess.run(
51                      [
52                          "ffprobe", "-v", "error",
53                          "-show_entries", "format=duration",
54                          "-of", "default=noprint_wrappers=1:nokey=1",
55                          video_path
56                      ],
57                      capture_output=True, text=True, check=True
58                  )
59                  return float(result.stdout.strip())
60              except Exception as e:
61                  logger.warning(f"Could not determine video duration via ffprobe: {e}")
62                  return 0.0
63
64          def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
```

```
float]], output_filename: str) -> str:
            """
            Extracts selected segments from a raw video and stitches them together
            into a seamless highlights file, applying end_padding to prevent abrupt endings.
            """
            if not segments:
                raise ValueError("No highlight segments provided to compile.")

            if not os.path.exists(raw_video_path):
                raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")

            output_path = os.path.join(self.output_dir, output_filename)

            # Probe video duration so we can detect out-of-range segments
            video_duration = self._get_video_duration(raw_video_path)
            if video_duration > 0:
                logger.info(f"Source video duration: {video_duration:.2f}s")
            else:
                logger.warning("Video duration unknown. Cannot validate segment boundaries
against video length.")

            # We will create temporary clips and stitch them using FFmpeg concat demuxer
            temp_clips = []
            skipped_segments = []

            # Create a temp directory within output_dir
            with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
                logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
                for idx, (raw_start, raw_end) in enumerate(segments):
                    # Normalize timestamps to float seconds ? handles float, int,
                    # plain numeric strings, and colon-separated formats like "MM:SS"
                    start = self._parse_time(raw_start)
                    end = self._parse_time(raw_end)
                    segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s -
{end:.2f}s]"

                    # Validate the segment times
                    if start < 0:
                        logger.warning({segment_label}: start_time is negative
({start:.2f}s). Clamping to 0.)
                        start = 0.0

                    if start >= end:
                        logger.error({segment_label}: start_time ({start:.2f}s) >= end_time
({end:.2f}s). Skipping invalid segment.)
                        skipped_segments.append((idx, start, end, "start >= end"))
                        continue

                    # Check if segment extends beyond video duration
                    if video_duration > 0:
                        if start >= video_duration:
                            logger.error(f"{segment_label}: start_time ({start:.2f}s) is
beyond the video duration ({video_duration:.2f}s). "
                                         f"This segment cannot be extracted ? the video ran
out. Skipping.")
                            skipped_segments.append((idx, start, end, "start beyond video
end"))
                            continue

                        if end > video_duration:
                            original_end = end
                            end = video_duration
                            logger.warning(f"{segment_label}: end_time ({original_end:.2f}s)
```

```
exceeds video duration ({video_duration:.2f}s). "
                                        f"Clamping end_time to {video_duration:.2f}s. The
segment may be truncated.")

                    clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")

                    # Precise sub-second trim using fast re-encoding to preserve stream
headers
                    padded_start = max(0.0, start - self.begin_padding)
                    padded_end = end + self.end_padding

                    # Clamp padded_end to video duration if known
                    if video_duration > 0 and padded_end > video_duration:
                        padded_end = video_duration
                        logger.info(f"{segment_label}: Padded end ({end +
self.end_padding:.2f}s) exceeds video duration. "
                                    f"Clamping to {video_duration:.2f}s (end padding
partially applied).")

                    trim_duration = padded_end - padded_start
                    logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")

                    cmd = [
                        "ffmpeg", "-y",
                        "-ss", str(round(padded_start, 3)),
                        "-to", str(round(padded_end, 3)),
                        "-i", raw_video_path,
                        "-c:v", "libx264",
                        "-preset", "superfast",
                        "-crf", "22",
                        "-c:a", "aac",
                        "-b:a", "128k",
                        clip_path
                    ]

                    try:
                        result = subprocess.run(cmd, capture_output=True, check=True)
                        # Verify the output clip was actually created and has content
                        if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
                            temp_clips.append(clip_path)
                        else:
                            logger.error({segment_label}: FFmpeg exited successfully but
output clip is missing or empty. Skipping.)
                            skipped_segments.append((idx, start, end, "empty output clip"))
                    except subprocess.CalledProcessError as e:
                        stderr_msg = e.stderr.decode(errors='replace').strip()
                        logger.error({segment_label}: FFmpeg trim failed.)
                        print(f"  Command: {' '.join(cmd)}")
                        print(f"  FFmpeg stderr: {stderr_msg[-500:]}")  # Last 500 chars to
avoid flooding
                        skipped_segments.append((idx, start, end, f"ffmpeg error:
{stderr_msg[-200:]}"))
                        continue

                # Report summary of skipped segments
                if skipped_segments:
                    logger.info(f"\nSUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
                    for seg_idx, seg_start, seg_end, reason in skipped_segments:
                        logger.info(f"  - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
                    logger.info("")

                if not temp_clips:
```

```
174                     raise RuntimeError(
175                         f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
176                         f"All {len(segments)} segments were skipped. "
177                         f"Check the errors above for details. "
178                         f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
179                     )
180 
181                 logger.info(Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch...)
182 
183                 # Create a list file for FFmpeg concat demuxer
184                 list_file_path = os.path.join(tmp_dir, "clips.txt")
185                 with open(list_file_path, "w") as f:
186                     for clip in temp_clips:
187                         # Escape single quotes in path
188                         safe_path = clip.replace("'", "'\\''")
189                         f.write(f"file '{safe_path}'\n")
190 
191                 logger.info(Stitching clips together...)
192                 # Stitch them together without re-encoding (since they were encoded with
identical settings)
193                 concat_cmd = [
194                     "ffmpeg", "-y",
195                     "-f", "concat",
196                     "-safe", "0",
197                     "-i", list_file_path,
198                     "-c", "copy",
199                     output_path
200                 ]
201 
202                 try:
203                     result = subprocess.run(concat_cmd, capture_output=True, check=True)
204 
205                     # Final verification
206                     if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
207                         raise RuntimeError(
208                             f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
209                             f"but output file is missing or empty: {output_path}"
210                         )
211 
212                     output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
213                     logger.info(Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB))
214                     return output_path
215                 except subprocess.CalledProcessError as e:
216                     stderr_msg = e.stderr.decode(errors='replace').strip()
217                     raise RuntimeError(
218                         f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
219                         f"  Command: {' '.join(concat_cmd)}\n"
220                         f"  FFmpeg stderr: {stderr_msg[-500:]}"
221                     )
222 
```

34. user (2026-06-10T05:37:00.146Z)

```
1       import subprocess
2       import os
3       import shutil
4       import logging
5 
6       logger = logging.getLogger(__name__)
7 
8       def get_video_duration(video_path: str) -> float:
9           """Returns the exact duration of a video file in seconds using ffprobe.
```

```
10          Returns 0.0 on failure."""
11          try:
12              duration_output = subprocess.check_output([
13                  "ffprobe", "-v", "error", "-show_entries", "format=duration",
14                  "-of", "default=noprint_wrappers=1:nokey=1", video_path
15              ]).decode().strip()
16              return float(duration_output)
17          except Exception as e:
18              logger.error(f"Could not detect video duration for {video_path}: {e}")
19              return 0.0
20
21      def extract_video_chunk(video_path: str, start_time: float, duration: float,
22                              output_path: str, reencode: bool = False,
23                              scale_width: int = None) -> bool:
24          """Extracts a chunk from a video file.
25
26          Args:
27              reencode: If True, re-encodes with libx264 (slower but frame-accurate).
28                        If False, uses stream-copy (fast but cuts at nearest keyframe).
29                        Use True for short clips where keyframe drift matters.
30              scale_width: If set (requires reencode), downscale to this width (height auto,
31                        even). Useful for high-res sources headed to an API that analyses at
32                        low res anyway ? shrinks uploads with no analysis loss.
33          """
34          cmd = [
35              "ffmpeg", "-y",
36              "-ss", str(start_time),
37              "-t", str(duration),
38              "-i", video_path,
39          ]
40          if reencode:
41              if scale_width:
42                  cmd += ["-vf", f"scale={int(scale_width)}:-2"]
43              cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
44          else:
45              cmd += ["-c", "copy"]
46          cmd.append(output_path)
47          try:
48              subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
49              return True
50          except Exception as e:
51              logger.error(f"Failed to extract chunk {start_time}s - {start_time+duration}s:
{e}")
52              return False
53
```

35. user (2026-06-10T05:37:19.123Z)

```
  File "backend/pipeline/stitcher/compiler.py", line 100
    logger.warning({segment_label}: start_time is negative ({start:.2f}s). Clamping to 0.)
                                                                                  ^
SyntaxError: invalid decimal literal
SYNTAX_FAIL
```

36. assistant (2026-06-10T05:37:26.611Z)

```
Critical confirmed ? compiler.py doesn't parse. Let me check when this broke and whether main.py
is importable.
```

37. user (2026-06-10T05:37:28.311Z)

```
7fbbeb0 feat(logging): standardize to logging in hot paths + update docs
8615534 refactor: adopt approved layered directory structure
```

```
7fbbeb0874ce2c2f7a9947c303d822ec71996e6b Sun Jun 7 14:03:29 2026 +0000 feat(logging):
standardize to logging in hot paths + update docs
```

38. user (2026-06-10T05:37:34.771Z)

```
    from backend.pipeline.stitcher.compiler import VideoCompiler
  File "C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\stitcher\compiler.py", line 100
    logger.warning({segment_label}: start_time is negative ({start:.2f}s). Clamping to 0.)
                                                                    ^
SyntaxError: invalid decimal literal
```

39. user (2026-06-10T05:37:44.813Z)

```
* docs/next-steps-multisport
  master
ON_MASTER
__init__.py
__pycache__
test_annotation_providers.py
test_automated_provider.py
test_base.py
test_calibrate_local.py
test_calibrate_wasb.py
test_calibration.py
test_config.py
test_database.py
test_distill_local.py
test_eval_annotations.py
test_eval_batch.py
test_eval_classifier.py
test_eval_harness.py
test_eval_metrics.py
test_eval_partial_labels.py
test_eval_regression.py
test_eval_training.py
test_export_wasb_segments.py
test_gemini.py
test_gemini_refine.py
test_geometry.py
test_hashing.py
test_hit_detector.py
test_provider_gemini.py
test_providers.py
test_rally_gate.py
test_rally_seq_proto.py
test_rally_slicer.py
test_reporting.py
test_reporting_harvest.py
test_reporting_spec_faq.py
test_sport_profiles.py
test_tracknet_hybrid.py
test_tracknet_runner.py
test_validators_base.py
test_video.py
test_wasb_cache.py
test_yolo_hybrid.py
```

40. assistant (2026-06-10T05:37:51.168Z)

```
The broken commit is on master. Let me sweep the whole backend for any other parse failures,
then run the test suite.
```

41. user (2026-06-10T05:37:54.457Z)

```
BAD FILES: 2
('scripts\\generate_khelacharya_flyer.py', '  File "scripts\\generate_khelacharya_flyer.py",
line 39')
('backend\\pipeline\\stitcher\\compiler.py', '  File "backend\\pipeline\\stitcher\\compiler.py",
line 100')
```

42. user (2026-06-10T05:37:59.714Z)

```
NAVY_LIGHT = HexColor("#1E293B")
TEAL = HexColor("#0D9488")
TEAL_DARK = HexColor("#0F766E")
TEAL_LIGHT = HexColor("#CCFBF1")
SLATE = HexColor("#334155")
SLATE_LIGHT = HexColor("#64748B")
SLATE_VLIGHT = HexColor("#F1F5F9")
WHITE = white

[Full script body would be inserted here in a real call ? truncated for this simulation response
for length reasons. In actual use the complete working script from the local file would be
provided.]
```

43. user (2026-06-10T05:38:16.177Z)

```
============================================================================
 Full test suite:  C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
 obsreport (optional reporting dep): available
============================================================================
.......................................................................... [ 28%]
.......................................................................... [ 56%]
.......................................................................... [ 85%]
.....................................                                    [100%]
254 passed in 13.77s
============================================================================
 RESULT: PASS - all tests green
============================================================================
```

44. assistant (2026-06-10T05:38:26.182Z)

```
Test suite is green (254 passed) while the main entrypoint is import-broken ? confirms the
coverage blind spot. Now providers, sports, annotations.
```

45. user (2026-06-10T05:38:27.133Z)

```
1       import os
2       import time
3       import json
4       import logging
5       from typing import Tuple
6       from google import genai
7       from google.genai import types
8       from backend.providers.base import AIVideoProvider
9
10      logger = logging.getLogger(__name__)
11
12      # The Gemini File API caps uploads at 2GB; stay well under it. Callers must chunk so
13      # each uploaded clip is below this. Oversize uploads are refused (not silently
attempted).
14      MAX_UPLOAD_MB = 500
15
16      class GeminiProvider(AIVideoProvider):
17          def __init__(self, api_key: str, model_name: str):
18              super().__init__(api_key, model_name)
19              # Initialize client lazily to avoid connection/auth issues during class
```

```
instantiation
            self._client = None

        @property
        def client(self):
            if self._client is None:
                self._client = genai.Client(api_key=self.api_key)
            return self._client

        def analyze_video(self, video_path: str, prompt: str,
                          chunk_duration: float = 0.0,
                          label: str = "") -> Tuple[list, dict]:
            log_prefix = f"Gemini {label}" if label else "Gemini"

            metrics = {
                "upload_time": 0.0,
                "process_time": 0.0,
                "gen_time": 0.0
            }

            # 0. Refuse oversize uploads (Gemini File API limit is 2GB; we cap lower for
safety/speed)
            try:
                size_mb = os.path.getsize(video_path) / (1024 * 1024)
            except OSError:
                size_mb = 0.0
            if size_mb > MAX_UPLOAD_MB:
                logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB >
{MAX_UPLOAD_MB}MB cap ? "
                             f"chunk smaller. Skipping upload.")
                return [], metrics

            # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
connections)
            logger.info(f"[{log_prefix}] Uploading {video_path}...")
            t_upload_start = time.time()
            video_file = None
            for attempt in range(3):
                try:
                    video_file = self.client.files.upload(file=video_path)
                    metrics["upload_time"] = time.time() - t_upload_start
                    logger.info(f"[{log_prefix}] Upload complete in
{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
                    break
                except Exception as e:
                    logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed:
{e}")
                    if attempt < 2:
                        time.sleep(2 * (attempt + 1))
            if video_file is None:
                logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
                return [], metrics

            # 2. Wait for processing on Google servers
            t_process_start = time.time()
            try:
                while video_file.state.name == "PROCESSING":
                    logger.info(f"[{log_prefix}] Processing on Google servers. Elapsed:
{time.time()-t_process_start:.1f}s. Retrying in 10s...")
                    time.sleep(10)
                    video_file = self.client.files.get(name=video_file.name)
                if video_file.state.name == "FAILED":
                    logger.error(f"[{log_prefix}] Video processing failed:
{video_file.error.message}")
                    try:
```

```
77                          self.client.files.delete(name=video_file.name)
78                      except Exception:
79                          pass
80                      return [], metrics
81              except Exception as e:
82                  logger.error(f"[{log_prefix}] Status check failed: {e}")
83                  try:
84                      self.client.files.delete(name=video_file.name)
85                  except Exception:
86                      pass
87                  return [], metrics
88
89              metrics["process_time"] = time.time() - t_process_start
90              logger.info(f"[{log_prefix}] Processing finished in
{metrics['process_time']:.1f}s.")
91
92              # 3. Call Gemini (retry transient 503/UNAVAILABLE model-overload + network
blips)
93              logger.info(f"[{log_prefix}] Submitting multimodal request to
{self.model_name}...")
94              t_gen_start = time.time()
95              raw_text = None
96              for attempt in range(3):
97                  try:
98                      response = self.client.models.generate_content(
99                          model=self.model_name,
100                         contents=[video_file, prompt],
101                         config=types.GenerateContentConfig(
102                             response_mime_type="application/json",
103                             temperature=0.0
104                         )
105                     )
106                     raw_text = response.text
107                     metrics["gen_time"] = time.time() - t_gen_start
108                     logger.info(f"[{log_prefix}] Response received in
{metrics['gen_time']:.1f}s:\n{raw_text}")
109                     break
110                 except Exception as e:
111                     logger.warning(f"[{log_prefix}] generate attempt {attempt + 1}/3 failed:
{e}")
112                     if attempt < 2:
113                         time.sleep(3 * (attempt + 1))
114             try:
115                 self.client.files.delete(name=video_file.name)
116             except Exception:
117                 pass
118
119             if not raw_text:
120                 return [], metrics
121
122             # 4. Parse JSON
123             try:
124                 segments = json.loads(raw_text)
125                 if not isinstance(segments, list):
126                     logger.error(f"[{log_prefix}] Response is not a JSON array.")
127                     return [], metrics
128                 return segments, metrics
129             except Exception as e:
130                 logger.error(f"[{log_prefix}] JSON parse failed: {e}")
131                 return [], metrics
132
```

46. user (2026-06-10T05:38:28.079Z)

```
1       from abc import ABC, abstractmethod
```

```
2       from typing import Tuple
3
4       class AIVideoProvider(ABC):
5           def __init__(self, api_key: str, model_name: str):
6               self.api_key = api_key
7               self.model_name = model_name
8
9           @abstractmethod
10          def analyze_video(self, video_path: str, prompt: str,
11                            chunk_duration: float = 0.0,
12                            label: str = "") -> Tuple[list, dict]:
13              """Sends a video file to the AI model along with a prompt, and returns a tuple
of
14              (segments_list, metrics_dict).
15
16              Args:
17                  video_path: Path to the video file to analyze.
18                  prompt: Prompt instructions.
19                  chunk_duration: Duration of this chunk in seconds, or 0.0 if not chunked.
20                  label: Diagnostic label for logging.
21
22              Returns:
23                  Tuple containing:
24                    - A list of detected segment dicts (e.g. [{"start_time": ..., "end_time":
...}])
25                    - A dictionary of performance metrics (e.g. {"upload_time": ...,
"process_time": ..., "gen_time": ...})
26              """
27              pass
28
```

47. user (2026-06-10T05:38:28.262Z)

```
1       from typing import Dict, Type
2       from backend.providers.base import AIVideoProvider
3       from backend.providers.gemini import GeminiProvider
4
5       class ProviderRegistry:
6           def __init__(self):
7               self._registry: Dict[str, Type[AIVideoProvider]] = {}
8               # Register Gemini provider
9               self.register("gemini", GeminiProvider)
10
11          def register(self, name: str, provider_cls: Type[AIVideoProvider]):
12              self._registry[name] = provider_cls
13
14          def get(self, name: str, api_key: str, model_name: str) -> AIVideoProvider:
15              if name not in self._registry:
16                  raise KeyError(f"AI Provider '{name}' is not registered in the
ProviderRegistry. Available: {list(self._registry.keys())}")
17              return self._registry[name](api_key=api_key, model_name=model_name)
18
19          def list_available(self) -> Dict[str, str]:
20              return {name: cls.__doc__ or f"{name.capitalize()} provider" for name, cls in
self._registry.items()}
21
22      provider_registry = ProviderRegistry()
23
```

48. user (2026-06-10T05:38:39.156Z)

```
1       from typing import Dict, Type
2       from backend.sports.base import SportProfile
3       from backend.sports.badminton import BadmintonSportProfile
4
```

```
5       class SportRegistry:
6           def __init__(self):
7               self._registry: Dict[str, Type[SportProfile]] = {}
8               # Automatically register Badminton
9               self.register(BadmintonSportProfile)
10
11          def register(self, profile_cls: Type[SportProfile]):
12              inst = profile_cls()
13              self._registry[inst.name] = profile_cls
14
15          def get(self, name: str) -> SportProfile:
16              if name not in self._registry:
17                  raise KeyError(f"Sport '{name}' is not registered in the SportRegistry.
Available: {list(self._registry.keys())}")
18              return self._registry[name]()
19
20          def list_available(self) -> Dict[str, str]:
21              return {name: cls.__doc__ or f"{name.capitalize()} sport profile" for name, cls
in self._registry.items()}
22
23      sport_registry = SportRegistry()
24
```

49. user (2026-06-10T05:38:39.627Z)

```
1       from abc import ABC, abstractmethod
2
3       class SportProfile(ABC):
4           @property
5           @abstractmethod
6           def name(self) -> str:
7               """The identifier of the sport (e.g., 'badminton')."""
8               pass
9
10          @abstractmethod
11          def get_prompt(self, chunk_duration: float = 0.0) -> str:
12              """Full system prompt for the AI model, including output schema."""
13              pass
14
15          @abstractmethod
16          def get_relevance_config(self) -> dict:
17              """Sport-specific visual validation params (court colors, thresholds, etc.)."""
18              pass
19
```

50. user (2026-06-10T05:38:39.963Z)

```
1       from backend.sports.base import SportProfile
2
3       class BadmintonSportProfile(SportProfile):
4           @property
5           def name(self) -> str:
6               return "badminton"
7
8           def get_prompt(self, chunk_duration: float = 0.0) -> str:
9               base_prompt = """System Role & Objective
10      You are a highly precise computer vision data extraction engine specializing in
badminton match analysis. Your exclusive objective is to parse a full-length match video and
generate a flawless JSON array containing exact rally boundaries and shot metrics for automated
highlight generation.
11
12      Core Directives
13      Detect every single active rally in the video. Do not miss any live gameplay.
14      Extract high-precision timestamps as TOTAL DECIMAL SECONDS from the start of the video
for exact frame-accurate cuts.
```

```
    Estimate the number of shots exchanged (net crossings) and provide a confidence score
for that count, acknowledging frame-sampling limitations.

    CRITICAL TIMESTAMP FORMAT RULES:
    - All start_time and end_time values MUST be expressed as TOTAL SECONDS from the
beginning of the video, as a decimal number.
    - Example: A rally starting at 1 minute 14 seconds into the video MUST be written as
74.0, NOT as "1:14" or 114.
    - Example: A rally starting at 2 minutes 30.5 seconds MUST be written as 150.5, NOT as
"2:30.5" or 230.5.
    - NEVER use MM:SS or HH:MM:SS colon-separated format. NEVER concatenate minutes and
seconds into a single number without converting.
    - The downstream pipeline will BREAK if timestamps are not in total decimal seconds.

    Positive Constraints: Rally Boundaries
    Start Condition: The exact fractional second the server's racket makes contact with the
shuttlecock.
    End Condition: The rally concludes ONLY when both of the following physical criteria are
met sequentially: First, the shuttlecock hits the court surface, strikes the net and drops, or
lands visibly out of bounds. Second, a 0.3 to 0.5-second buffer has elapsed where both players
have visibly ceased all kinetic momentum (no lunging, sliding, or racket follow-through).

    Negative Constraints: Anomaly Rejection (CRITICAL)
    Replay Rejection: You must strictly ignore and filter out all slow-motion instant
replays. Replays are not live rallies.
    Dead-Time Rejection: Ignore all Hawkeye/line-call challenges, interval coaching, towel
breaks, court mopping, changing shuttles, and player celebrations.
    Pre-serve Rituals: Do not start the timestamp during the players' setup or stance. Wait
for physical racket-to-shuttle contact.

    Shot Tracking & Data Confidence
    Shot Definition: Count every successful traverse of the shuttlecock across the net
(serves, clears, smashes, drops, drives). Do not count the final grounding or pre-net faults.
    Confidence Scoring: Because the shuttlecock moves at extreme velocities that may fall
between sampled frames, provide a shot_count_confidence metric (Low, Medium, High). Base your
estimate on player biomechanics, positioning, and racket swings if the shuttle itself is
blurred.

    Strict Output Schema
    Output strictly in the following JSON array format. Do not include markdown formatting,
introductory text, conversational filler, or explanations. Any deviation from this schema will
break the downstream pipeline.
    Remember: start_time and end_time are TOTAL SECONDS (decimal), not MM:SS.

    [
    {
    "start_time": 74.0,
    "end_time": 81.3,
    "shuttle_exchange_count": 11,
    "shot_count_confidence": "High"
    },
    {
    "start_time": 150.5,
    "end_time": 174.2,
    "shuttle_exchange_count": 24,
    "shot_count_confidence": "Medium"
    }
    ]
    """
            if chunk_duration > 0.0:
                duration_context = (
                    f"IMPORTANT VIDEO CONTEXT: This video segment is exactly
{chunk_duration:.1f} seconds long. "
                    f"All timestamps MUST be between 0 and {chunk_duration:.1f}. "
                    f"Any timestamp beyond {chunk_duration:.1f}s is INVALID.\n\n"
```

```
                )
                return duration_context + base_prompt
            return base_prompt

        def get_relevance_config(self) -> dict:
            return {
                "court_green_lower": [35, 40, 40],
                "court_green_upper": [85, 255, 255],
                "court_pixels_min_ratio": 0.05
            }

```

51. user (2026-06-10T05:38:47.089Z)

```
    """Annotation provider registry ? pluggable sources of ground-truth rally labels.

    An `AnnotationProvider` turns a video reference into a list of `(start, end)` rally
    intervals ? the **same shape** that `backend.eval.annotations.load_annotations`
    produces and `backend.eval.metrics` consumes. That symmetry is the whole point:
    golden-set ground truth can come from a CSV file, a human annotation vendor, or an
    automated oracle behind one interface, so the regression flow
    (`backend.eval.regression`) never needs to know where labels came from.

    Mirrors the existing registry pattern in `backend/providers/base.py` and
    `backend/indexer/base.py`. See `docs/GOLDEN_SET_VENDORING.md` (and ADR-006) for the
    design rationale.
    """
    from abc import ABC, abstractmethod
    from typing import Dict, List, Optional, Tuple, Type

    # An (start, end) interval in seconds ? identical to backend.eval.metrics.Interval.
    Interval = Tuple[float, float]

    # Job lifecycle statuses returned by poll().
    QUEUED = "queued"
    RUNNING = "running"
    DONE = "done"
    FAILED = "failed"
    STATUSES = (QUEUED, RUNNING, DONE, FAILED)


    class AnnotationProvider(ABC):
        """Abstract source of ground-truth rally intervals for a video.

        The three-method contract (submit ? poll ? fetch) accommodates both
        synchronous providers (a file already on disk) and asynchronous ones (a human
        vendor that takes days). Synchronous providers can use the `annotate()`
        convenience, which runs the full cycle in one call.
        """

        @property
        @abstractmethod
        def name(self) -> str:
            """Unique registry key for this provider (e.g. 'file', 'vendor', 'auto')."""

        @abstractmethod
        def submit(self, video_ref: str, guideline: Optional[str] = None,
                   sport: str = "badminton") -> str:
            """Submit a video for annotation; return an opaque job_id.

            `video_ref` is provider-defined (a file path, a video_id/MD5, a URL).
            `guideline` is the annotation spec (what counts as a rally start/end).
            """

        @abstractmethod
```

```
        def poll(self, job_id: str) -> str:
            """Return one of STATUSES for the given job."""

        @abstractmethod
        def fetch(self, job_id: str) -> List[Interval]:
            """Return the completed annotations as sorted `(start, end)` intervals.

            Should raise if the job is not DONE / the labels are unavailable, rather
            than returning an empty list (which would silently skew downstream scores).
            """

        def annotate(self, video_ref: str, guideline: Optional[str] = None,
                     sport: str = "badminton") -> List[Interval]:
            """Convenience: submit + fetch for a provider that completes synchronously.

            Async providers (e.g. a human vendor with a multi-day SLA) should be driven
            via submit/poll/fetch explicitly instead ? calling this on one will raise
            once the job isn't immediately DONE.
            """
            job_id = self.submit(video_ref, guideline=guideline, sport=sport)
            status = self.poll(job_id)
            if status != DONE:
                raise RuntimeError(
                    f"{self.name}.annotate(): job '{job_id}' is '{status}', not DONE. "
                    f"Use submit/poll/fetch for asynchronous providers."
                )
            return self.fetch(job_id)


    class AnnotationProviderRegistry:
        """Registry of annotation providers, keyed by each provider's `name`."""

        def __init__(self):
            self._providers: Dict[str, Type[AnnotationProvider]] = {}

        def register(self, provider_cls: Type[AnnotationProvider]) ->
Type[AnnotationProvider]:
            """Register a provider class under its `name`. Returns the class (decorator-
friendly)."""
            try:
                key = provider_cls().name
            except Exception:
                # Providers needing constructor args fall back to the class name.
                key = getattr(provider_cls, "__name__", "unknown")
            self._providers[key] = provider_cls
            return provider_cls

        def get(self, name: str, **kwargs) -> AnnotationProvider:
            """Instantiate the named provider, passing through any constructor kwargs."""
            cls = self._providers.get(name)
            if cls is None:
                raise KeyError(
                    f"Unknown annotation provider '{name}'. Available:
{self.list_available()}"
                )
            return cls(**kwargs)

        def list_available(self) -> List[str]:
            return list(self._providers.keys())


    annotation_registry = AnnotationProviderRegistry()

```

52. user (2026-06-10T05:38:47.567Z)

```
    """FileProvider ? ground-truth rally labels from pre-made CSV files.

    This formalizes today's manual / ShuttleSet annotation flow behind the
    `AnnotationProvider` interface: the "annotation job" is simply a CSV that already
    exists on disk (the same `start,end` format `run_eval.py --scaffold` writes). The
    job_id IS the resolved CSV path, so submit/poll/fetch all operate on that path.

    It's the synchronous, zero-cost provider used as the default golden-set source and
    as the stand-in in tests/CI before a human-vendor or automated provider is wired in.
    """
    import os
    from typing import List, Optional

    from backend.annotations.base import (
        AnnotationProvider,
        annotation_registry,
        Interval,
        DONE,
        FAILED,
    )
    from backend.eval.annotations import load_annotations


    class FileProvider(AnnotationProvider):
        def __init__(self, csv_dir: Optional[str] = None, ext: str = ".csv"):
            """
            Args:
                csv_dir: if set, a bare `video_ref` (not itself a path) is resolved to
                         `<csv_dir>/<video_ref><ext>` ? handy for keying by video_id/MD5.
                ext: file extension used for that directory-based resolution.
            """
            self.csv_dir = csv_dir
            self.ext = ext

        @property
        def name(self) -> str:
            return "file"

        def _resolve(self, video_ref: str) -> str:
            """Map a video_ref to a CSV path.

            An existing path is used as-is; otherwise it's treated as a video_id and
            looked up under csv_dir (if configured).
            """
            if os.path.isfile(video_ref):
                return video_ref
            if self.csv_dir:
                return os.path.join(self.csv_dir, f"{video_ref}{self.ext}")
            return video_ref

        def submit(self, video_ref: str, guideline: Optional[str] = None,
                   sport: str = "badminton") -> str:
            # Nothing to do ? the label file is expected to already exist. The job_id
            # is the resolved CSV path. (guideline/sport are irrelevant for a file.)
            return self._resolve(video_ref)

        def poll(self, job_id: str) -> str:
            return DONE if os.path.isfile(job_id) else FAILED

        def fetch(self, job_id: str) -> List[Interval]:
            if not os.path.isfile(job_id):
                raise FileNotFoundError(f"annotation file not found: {job_id}")
            return load_annotations(job_id)
```

```
66      annotation_registry.register(FileProvider)
67
```

53. user (2026-06-10T05:38:48.156Z)

```
1       """AutomatedProvider ? the fast-tier "oracle" annotation source (GS-4, interface stub).
2
3       The fast/CI tier of the golden-set flow needs an *automated* labeler that turns a
4       video into rally `(start, end)` intervals quickly and cheaply, as a smoke alarm
5       between authoritative human-vendor runs (the slow tier).
6
7       This slice ships the **interface only**, deliberately model-agnostic: an
8       `AutomatedProvider` that delegates labeling to an injected `labeler` callable, plus
9       a deterministic fake for tests/CI. No concrete model is wired yet ? that's a
10      tracked backlog item (docs/BACKLOG.md), to be chosen alongside the production LLM
11      step so the lineages differ (see the oracle rule below).
12
13      ?? **Oracle rule.** The labeler MUST be a *different model lineage* than whatever
14      production uses. If production becomes the offline ActionFormer segmenter and the
15      oracle is also ActionFormer, they share blind spots and the regression test is
16      blind ? green while both are wrong. `lineage` records the oracle's model family so
17      callers can guard against it matching production (`warn_if_same_lineage`).
18      """
19      import logging
20      from typing import Callable, Dict, List, Optional
21
22      from backend.annotations.base import (
23          AnnotationProvider,
24          annotation_registry,
25          Interval,
26          DONE,
27          FAILED,
28      )
29
30      logger = logging.getLogger(__name__)
31
32      # A labeler turns (video_ref, guideline, sport) into rally intervals.
33      Labeler = Callable[[str, Optional[str], str], List[Interval]]
34
35
36      def _not_configured(video_ref: str, guideline: Optional[str], sport: str) ->
List[Interval]:
37          """Default labeler: refuses to run until a real model is injected.
38
39          Keeps `AutomatedProvider()` constructible (so the registry can key it as 'auto')
40          while making accidental use of an unconfigured oracle fail loudly rather than
41          silently returning no rallies.
42          """
43          raise NotImplementedError(
44              "AutomatedProvider has no labeler configured. Inject one via "
45              "annotation_registry.get('auto', labeler=<fn>, lineage='<model-family>'). "
46              "Choosing the concrete oracle model is a tracked item ? see docs/BACKLOG.md
(GS-4)."
47          )
48
49
50      def warn_if_same_lineage(oracle_lineage: Optional[str], production_lineage:
Optional[str]) -> bool:
51          """Warn (and return True) if the oracle shares production's model lineage.
52
53          Encodes the oracle rule in code, not just docs: a same-lineage oracle can't be
54          trusted to catch production's failures. Callers wiring CI should pass both.
55          """
56          if oracle_lineage and production_lineage and oracle_lineage == production_lineage:
57              logger.warning(
```

```
                "Oracle lineage '%s' matches production's ? the regression test may be "
                "blind to shared failure modes. Use a different model lineage.",
                oracle_lineage,
            )
            return True
        return False


    class AutomatedProvider(AnnotationProvider):
        def __init__(self, labeler: Optional[Labeler] = None, lineage: str = "unspecified"):
            """
            Args:
                labeler: callable producing rally intervals for a video. Defaults to a
                         stub that raises until configured.
                lineage: the oracle's model family (e.g. 'gemini', 'qwen-vl') ? recorded
                         so callers can enforce the oracle rule via warn_if_same_lineage.
            """
            self._labeler = labeler or _not_configured
            self.lineage = lineage
            # In-process synchronous job store: job_id -> labeled intervals.
            self._jobs: Dict[str, List[Interval]] = {}

        @property
        def name(self) -> str:
            return "auto"

        def submit(self, video_ref: str, guideline: Optional[str] = None,
                   sport: str = "badminton") -> str:
            # Synchronous stub: run the labeler now and stash the result. A real async
            # oracle (e.g. a queued API job) would kick off here and resolve in poll().
            intervals = self._labeler(video_ref, guideline, sport)
            intervals = sorted((float(s), float(e)) for s, e in intervals)
            job_id = video_ref
            self._jobs[job_id] = intervals
            return job_id

        def poll(self, job_id: str) -> str:
            return DONE if job_id in self._jobs else FAILED

        def fetch(self, job_id: str) -> List[Interval]:
            if job_id not in self._jobs:
                raise KeyError(f"AutomatedProvider: unknown job '{job_id}' (submit it
first)")
            return self._jobs[job_id]


    annotation_registry.register(AutomatedProvider)

```

54. user (2026-06-10T05:38:55.347Z)

```
    """Per-video observability reports for the indexer.

    `obsreport` is a SOFT dependency: if it is not installed, this module degrades to a
    no-op and the pipeline runs exactly as before. The single entrypoint
    `emit_indexer_report(...)` is designed to be called from a `finally` block and
    NEVER raises ? emitting a report must not be able to break a processing run.
    """
    from __future__ import annotations

    import os
    from typing import Any, Dict, List, Optional

    try:  # soft dependency ? never hard-fail the pipeline on a reporting import
        import obsreport  # noqa: F401
```

```
        from . import harvest

        OBSREPORT_AVAILABLE = True
    except Exception:  # pragma: no cover - exercised only when obsreport is absent
        OBSREPORT_AVAILABLE = False


    def report_enabled(config: Optional[Dict[str, Any]]) -> bool:
        return bool((config or {}).get("report", {}).get("enabled", True))


    def emit_indexer_report(
        *,
        db,
        video_id: str,
        video_path: Optional[str],
        config: Optional[Dict[str, Any]],
        val_results: List[Any],
        val_context: Optional[Dict[str, Any]] = None,
        segmenter_requested: Optional[str] = None,
        segmenter_actual: Optional[str] = None,
        was_reingest: bool = False,
        segmentation_ran: bool = True,
    ) -> Optional[Dict[str, str]]:
        """Harvest + write the report. Returns the written file paths, or None if
        reporting is unavailable/disabled/failed. Never raises."""
        config = config or {}
        # Accept either a plain dict or a typed config object (e.g.
backend.config.AppConfig).
        # Normalize to a plain nested dict so harvest's dict access works uniformly.
        if hasattr(config, "model_dump"):
            try:
                config = config.model_dump(mode="json")
            except Exception:
                config = {}
        if not OBSREPORT_AVAILABLE or not report_enabled(config):
            return None
        try:
            play_segments = db.query_play_segments(video_id, {}) if db is not None else []
            candidates = db.query_candidate_segments(video_id) if db is not None else []
            rec = harvest.record_run(
                video_id=video_id,
                video_path=video_path,
                config=config,
                val_results=val_results,
                val_context=val_context,
                play_segments=play_segments,
                candidates=candidates,
                segmenter_requested=segmenter_requested,
                segmenter_actual=segmenter_actual,
                was_reingest=was_reingest,
                segmentation_ran=segmentation_ran,
            )
            output_dir = (config.get("report") or {}).get("output_dir")
            res = rec.write(output_dir=output_dir)
            return res.paths or None
        except Exception:  # pragma: no cover - defensive; reporting must never break a run
            return None

```

55. user (2026-06-10T05:38:55.771Z)

```
    """Build an observability report for one indexer run.

    Pure-ish harvest: given the validation results/context, the persisted segments,
```

```
    and run metadata, populate an `obsreport.RunRecorder` with the raw signals. The
    *footgun rules* (which turn signals into flags) live declaratively in spec.yaml,
    so this module only records facts faithfully ? it does not decide warnings.

    Imported only after `backend.reporting` confirms obsreport is installed, so it may
    import obsreport freely.
    """
    from __future__ import annotations

    import os
    from pathlib import Path
    from typing import Any, Dict, List, Optional

    try:
        import obsreport  # soft dep; only needed for RunRecorder / load_spec at runtime
    except ImportError:
        obsreport = None  # type: ignore[assignment]

    _AI_SEGMENTERS = {"yolo_hybrid", "tracknet_hybrid", "wasb_hybrid", "gemini"}
    _SPEC_PATH = Path(__file__).resolve().parent / "spec.yaml"


    def load_spec() -> "obsreport.ReportSpec":
        return obsreport.load_spec(_SPEC_PATH)


    # --- media metadata ----------------------------------------------------
    def video_meta_from_context(val_context: Optional[Dict[str, Any]], video_path:
Optional[str]) -> Dict[str, Any]:
        """Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source
        distinguishes a real probe from the absence of one (the fallback footgun)."""
        meta: Dict[str, Any] = {"fps_source": "unknown"}
        if video_path and os.path.exists(video_path):
            try:
                meta["size_mb"] = round(os.path.getsize(video_path) / (1024 * 1024), 1)
            except OSError:
                pass

        ffmeta = (val_context or {}).get("ffprobe_meta") or {}
        streams = ffmeta.get("streams") or []
        fmt = ffmeta.get("format") or {}
        if streams:
            s = streams[0]
            w, h = s.get("width"), s.get("height")
            meta["width"], meta["height"] = w, h
            if w and h:
                meta["resolution"] = f"{w}x{h}"
            fps_str = s.get("r_frame_rate", "0/1")
            try:
                num, den = (int(x) for x in fps_str.split("/"))
                if den:
                    meta["fps"] = round(num / den, 2)
                    meta["fps_source"] = "probed"
            except (ValueError, ZeroDivisionError):
                pass
            if s.get("codec_name"):
                meta["container"] = fmt.get("format_name") or s.get("codec_name")
        if fmt.get("duration"):
            try:
                meta["duration_s"] = round(float(fmt["duration"]), 1)
            except (ValueError, TypeError):
                pass
        if fmt.get("format_name"):
            meta["container"] = fmt.get("format_name")

```

```
68          # Audio readiness signal (for Input Quality UX extension per review item 2).
69          # Guidelines make "record with audio on, mic unobstructed" a first-class requirement
70          # (ADR-007) because shuttle thwacks are a strong rally cue for recall.
71          # We only record presence here (cheap, from ffprobe already in context); the actual
72          # thwack onset probe (pipeline/audio/hit_detector) remains a diagnostic /
calibration
73          # tool for now.
74          audio_streams = [s for s in streams if (s or {}).get("codec_type") == "audio"]
75          meta["audio_present"] = len(audio_streams) > 0
76          return meta
77
78
79      # --- stage builders ----------------------------------------------------------
80      def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:
81          with rec.stage("validation") as st:
82              total = len(val_results)
83              passed = sum(1 for r in val_results if getattr(r, "passed", False))
84              st.add_metric("checks_passed", passed)
85              st.add_metric("checks_total", total)
86              failures = [r for r in val_results if not getattr(r, "passed", True)]
87              for r in failures:
88                  st.messages.append(f"FAILED {getattr(r, 'validator_name', '?')}: {getattr(r,
'message', '')}")
89              st.status = "error" if failures else ("ok" if total else "not_run")
90
91
92      def _segmentation_stage(
93          rec: "obsreport.RunRecorder",
94          play_segments: List[Dict[str, Any]],
95          candidates: List[Dict[str, Any]],
96          segmenter_requested: Optional[str],
97          segmenter_actual: Optional[str],
98          config_default: Optional[str],
99          was_reingest: bool,
100         ran: bool,
101     ) -> None:
102         with rec.stage("segmentation") as st:
103             if not ran:
104                 st.status = "not_run"
105             seg_count = len(play_segments)
106             total_play = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
107             detector_tag = None
108             if play_segments:
109                 detector_tag = (play_segments[0].get("metadata") or {}).get("detector")
110             st.add_metric("candidate_windows", len(candidates))
111             st.add_metric("segment_count", seg_count, audience="both")
112             st.add_metric("total_play_s", total_play, unit="s")
113             # "diverges" only when a segmenter actually RAN, the user did NOT explicitly
114             # pick it, and it differs from config's default (the genuine surprise case).
115             # An explicit --segmenter override or a halted run is not a divergence.
116             matches_default = True
117             if ran and segmenter_actual and config_default:
118                 matches_default = (segmenter_actual == config_default)
119             st.details.update({
120                 "segmenter_requested": segmenter_requested,
121                 "segmenter_actual": segmenter_actual,
122                 "config_default": config_default,
123                 "segmenter_explicitly_requested": segmenter_requested is not None,
124                 "matches_config_default": matches_default,
125                 "was_reingest": was_reingest,
126                 "detector": detector_tag,
127             })
128             # Surface metadata trustworthiness at the data level (not just in a warning):
129             # the motion baseline fabricates per-segment metadata; everything else measures
it.
```

```
            if ran and segmenter_actual:
                st.details["metadata_source"] = (
                    "synthetic/heuristic ? NOT measured" if segmenter_actual == "motion"
                    else "measured (detector/AI)"
                )


    def _ai_stage(rec: "obsreport.RunRecorder", config: Dict[str, Any], segmenter_actual:
Optional[str],
                  play_segments: List[Dict[str, Any]], ran: bool) -> None:
        """Only meaningful for AI-based segmenters. Records whether the provider key was
        present ? the signal the silent_provider_noop rule keys on."""
        ai_based = segmenter_actual in _AI_SEGMENTERS
        provider = (config.get("ai_provider") or "gemini")
        key_env = f"{provider.upper()}_API_KEY"
        with rec.stage("ai_handoff") as st:
            if not ran or not ai_based:
                st.status = "not_run" if not ran else "skipped"
            st.details.update({
                "ai_based": ai_based,
                "provider": provider if ai_based else None,
                "model": config.get("ai_model") if ai_based else None,
                "api_key_present": bool(os.environ.get(key_env)) if ai_based else None,
            })
            if ai_based:
                st.add_metric("confirmed_segments", len(play_segments))


    def _highlights(rec: "obsreport.RunRecorder", play_segments: List[Dict[str, Any]],
                    video_meta: Dict[str, Any]) -> None:
        total = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
        coverage = None
        dur = video_meta.get("duration_s")
        if dur:
            coverage = round(100.0 * total / dur, 1)
        notes = []
        if not play_segments:
            notes.append("No highlights were produced for this video.")
        rec.set_highlights(segment_count=len(play_segments), total_highlight_s=total,
                           coverage_pct=coverage, notes=notes)


    # --- top-level ----------------------------------------------------------
    def record_run(
        *,
        video_id: str,
        video_path: Optional[str],
        config: Dict[str, Any],
        val_results: List[Any],
        val_context: Optional[Dict[str, Any]],
        play_segments: List[Dict[str, Any]],
        candidates: List[Dict[str, Any]],
        segmenter_requested: Optional[str] = None,
        segmenter_actual: Optional[str] = None,
        was_reingest: bool = False,
        segmentation_ran: bool = True,
        tool_version: Optional[str] = None,
        spec: Optional["obsreport.ReportSpec"] = None,
        **recorder_kwargs: Any,
    ) -> "obsreport.RunRecorder":
        """Assemble a finalized recorder for an indexer run (does NOT write)."""
        rec = obsreport.RunRecorder(
            "badminton-highlight-indexer",
            tool_version=tool_version,
            video_id=video_id,
```

```
194             video_id_kind="file_md5",
195             video_path=video_path,
196             spec=spec if spec is not None else load_spec(),
197             **recorder_kwargs,
198         )
199         vm = video_meta_from_context(val_context, video_path)
200         # If we never probed an fps but segmentation still ran, the segmenter must have
201         # used its internal fallback (30fps / 1280px) ? surface that as the footgun it is,
202         # AND show the assumed numeric value so the report reads "30 (fallback)" not "?
(fallback)".
203         if vm.get("fps_source") == "unknown" and segmentation_ran:
204             vm["fps_source"] = "fallback"
205             if vm.get("fps") is None:
206                 vm["fps"] = 30.0  # matches the runners' fallback (_probe_fps_width -> 30.0
/ 1280)
207         rec.set_video_meta(**vm)
208
209         _validation_stage(rec, val_results)
210         config_default = (config.get("indexing") or {}).get("default_segmenter")
211         _segmentation_stage(rec, play_segments, candidates, segmenter_requested,
212                             segmenter_actual, config_default, was_reingest,
segmentation_ran)
213         _ai_stage(rec, config, segmenter_actual, play_segments, segmentation_ran)
214         _highlights(rec, play_segments, vm)
215
216         rec.finalize()
217         return rec
218
```

56. user (2026-06-10T05:39:08.302Z)

```
1       # Observability report spec for the badminton-highlight-indexer.
2       # The report ENGINE (config language, renderers) lives in the shared `obsreport`
3       # library; this file is the indexer-specific layout + footgun rules. Every
4       # actionable rule cites a README/FAQ section so a reader can self-debug.
5       name: indexer
6       title: Badminton Highlight Indexer ? Video Report
7
8       # Customer here = whoever receives the highlight reel.
9       customer_verdict:
10        pass: "? Your highlights are ready."
11        pass_with_warnings: "? Your highlights are ready. (A couple of minor notes above.)"
12        fail: "?? We couldn't finish generating highlights for this video. Please contact
support."
13
14      sections:
15        - { id: run,        title: "Run",                kind: identity,   audience: both }
16        - { id: video,      title: "Video",              kind: video_meta, audience: both }
17        - { id: highlights, title: "Highlights",         kind: highlights, audience: customer
}
18        - { id: cust_notes, title: "Things to know",     kind: flags,      audience: customer
}
19        - { id: stages,     title: "Processing stages",  kind: stages,     audience:
technical }
20        - { id: findings,   title: "Findings & warnings", kind: flags,     audience:
technical }
21        - { id: verdict,    title: "Verdict",            kind: verdict,    audience: both }
22
23      rules:
24        # The classic silent failure: 0 segments because the AI provider no-op'd on a
25        # missing API key ? NOT because the video has no rallies.
26        - code: silent_provider_noop
27          severity: error
28          when:
29            - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: false }
```

```
          - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }
        message: >-
          0 segments AND no AI provider API key was set. The segmenter (see Processing
          stages > segmentation > segmenter_actual) hands its candidate windows to the AI
          provider, which silently did nothing without a key ? so this is a MISSING
          CREDENTIAL, not 'the video has no rallies'. Fix: put GEMINI_API_KEY=<key> in a
          .env file in the project root and re-run (see README#Q7).
        faq_ref: "README#Q7"
        customer_message: >-
          We couldn't generate highlights for this video due to a configuration issue
          on our side. Please contact support.

      # API key WAS present but an AI segmenter still produced nothing ? likely a
      # provider error / quota / WSL healthcheck issue rather than a truly empty video.
      - code: ai_empty_vs_no_rallies
        severity: warn
        when:
          - { path: "stages.ai_handoff.details.ai_based", op: eq, value: true }
          - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: true }
          - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }
        message: >-
          An AI-based segmenter returned 0 segments even though the API key is set.
          Empty can mean a provider error, exhausted quota, or (for WASB/TrackNet) a
          failed WSL healthcheck ? these all return [] silently. Re-run with -u for
          unbuffered logs and check the provider/WSL output before concluding 'no rallies'.
        faq_ref: "README#Q7"

      # fps was assumed rather than probed -> velocity thresholds + timing may be wrong.
      - code: fps_fallback_used
        severity: warn
        when:
          - { path: "video_meta.fps_source", op: eq, value: fallback }
        message: >-
          fps was NOT probed from the file ? a fallback default (shown as the fps value)
          was used. Because rally timing and motion thresholds scale with fps, boundaries
          and motion sensitivity may be miscalibrated. This usually means validation didn't
          run (it probes the file) or ffprobe couldn't read r_frame_rate. Confirm the real
          fps with ffprobe and re-run.
        faq_ref: "README#Q14"

      # Audio stream missing or disabled. Per VIDEO_RECORDING_GUIDELINES (and ADR-007)
      # the shuttle "thwack" is a first-class rally cue for recall; recording with audio
      # on (mic unobstructed, no aggressive noise suppression) is now a hard requirement
      # for good results with the shuttle-tracking substrates.
      - code: audio_disabled_or_missing
        severity: warn
        when:
          - { path: "video_meta.audio_present", op: eq, value: false }
        message: >-
          No audio stream was detected in the input (or it could not be read). The shuttle
          "thwack" impact sound is a strong cue that improves rally recall (especially for
          the WASB/TrackNet detector paths). Record with audio enabled, keep the camera
          mic unobstructed, and avoid heavy wind/noise-reduction filters that flatten the
          transient. See VIDEO_RECORDING_GUIDELINES.md (the "Audio (REQUIRED)" row) and
          AUDIO_RALLY_DETECTION_PLAN.md.
        faq_ref: "VIDEO_RECORDING_GUIDELINES"
        customer_message: >-
          For best highlight results, record with audio on and the mic unobstructed.
          The shuttle impact sound helps the detector find rallies. Re-record or
          re-process after enabling audio (see VIDEO_RECORDING_GUIDELINES.md).

      # The motion baseline fabricates its per-segment metadata/events ? they are not
      # measured. Warn so nobody trusts them as ground truth.
      - code: synthetic_motion_metadata
        severity: warn
```

```
95          when:
96            - { path: "stages.segmentation.details.segmenter_actual", op: eq, value: motion }
97          message: >-
98            The 'motion' segmenter was used: its per-segment metadata (intensity, speed,
99            events, server/receiver) are PLACEHOLDER values (unmeasured) ? fabricated by the
100           heuristic, NOT derived from the video ? so a PASS with segments here does not mean
101           the metadata is trustworthy. Use yolo_hybrid/gemini for real shot metadata.
102         faq_ref: "README#Q10"
103
104       # The segmenter that ran differs from config's default -> you may be looking at a
105       # different detector than expected (config/setup/CLI defaults are known to diverge).
106       - code: segmenter_divergence
107         severity: info
108         when:
109           - { path: "stages.segmentation.details.matches_config_default", op: eq, value:
false }
110           - { path: "stages.segmentation.details.segmenter_explicitly_requested", op: eq,
value: false }
111         message: >-
112           The segmenter that ran differs from config.json's default_segmenter, and no
113           explicit --segmenter was given ? so a fallback default kicked in. The indexer's
114           defaults are known to diverge across config.json / setup.py / the CLI; confirm
115           you ran the detector you meant.
116         faq_ref: "README#Q7"
117
118       # Re-processing an already-indexed video replaced its prior segments (INSERT OR
119       # REPLACE cascades). Note it so a changed result isn't mistaken for instability.
120       - code: reingest_replaced_prior
121         severity: info
122         when:
123           - { path: "stages.segmentation.details.was_reingest", op: eq, value: true }
124         message: >-
125           This video was already in the database; re-processing REPLACED its previous
126           segments (videos are keyed by file MD5; re-ingest cascades a delete). Expected
127           when you intend to re-run; just know the earlier results are gone.
128         faq_ref: "README#Q11"
129
130       # A validation check failed -> segmentation was halted. Point the reader at the
131       # report's Validation section + the per-check FAQ.
132       - code: validation_failed
133         severity: error
134         when:
135           - { path: "stages.validation.status", op: eq, value: error }
136         message: >-
137           One or more validation checks failed, so indexing was halted. See the
138           Processing stages > validation block above: each failed check states the
139           reason (relevance -> README#Q2, fps/keyframes -> README#Q5).
140         faq_ref: "README#Q13"
141         customer_message: >-
142           This video couldn't be processed because it didn't pass our quality checks
143           (for example: too blurry, too low resolution, or not a badminton court).
144
```

57. user (2026-06-10T05:39:08.804Z)

```
1       """Canonical file-identity hashing for the indexer.
2
3       `video_id` is the MD5 of the *whole* video file. Historically each entrypoint
4       (`backend/main.py`, `run_eval.py`, `run_phase3_eval.py`, `run_process_and_stitch.py`)
5       defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
6       The hash values agreed (same bytes hashed), but the duplicated code was a footgun:
7       any future change to one copy would silently diverge the DB key.
8
9       This module is the single source of truth. Import `compute_video_id` everywhere a
10      video needs to be keyed.
```

```
    """

    from __future__ import annotations

    import hashlib

    # 64 KB matches the value used by the live pipeline (backend/main.py) before this
    # was centralized. The chunk size does not affect the resulting digest ? it only
    # bounds memory use while streaming large files.
    _CHUNK_SIZE = 65536


    def compute_video_id(filepath: str) -> str:
        """Return the MD5 hex digest of the whole file ? the canonical video_id.

        Streams the file in fixed-size chunks so multi-gigabyte recordings never load
        into memory at once.
        """
        hasher = hashlib.md5()
        with open(filepath, "rb") as f:
            for chunk in iter(lambda: f.read(_CHUNK_SIZE), b""):
                hasher.update(chunk)
        return hasher.hexdigest()

```

58. user (2026-06-10T05:39:09.098Z)

```
    import math
    from typing import Tuple

    def get_center(bbox: Tuple[float, float, float, float]) -> Tuple[float, float]:
        """Calculate the center (x, y) of a bounding box.
        bbox format: (x1, y1, x2, y2)
        """
        x1, y1, x2, y2 = bbox
        return ((x1 + x2) / 2.0, (y1 + y2) / 2.0)

    def get_distance(p1: Tuple[float, float], p2: Tuple[float, float]) -> float:
        """Calculate the Euclidean distance between two points (x, y)."""
        return math.sqrt((p2[0] - p1[0])**2 + (p2[1] - p1[1])**2)

    def get_velocity(bbox1: Tuple[float, float, float, float], bbox2: Tuple[float, float,
float, float], fps: float) -> float:
        """Calculate velocity (pixels per second) of a bounding box moving between two
consecutive frames.
        """
        c1 = get_center(bbox1)
        c2 = get_center(bbox2)
        dist = get_distance(c1, c2)
        return dist * fps

    def get_velocity_normalised(bbox1: Tuple[float, float, float, float], bbox2:
Tuple[float, float, float, float],
                                fps: float, frame_width: float) -> float:
        """Calculate velocity as a fraction of frame width per second.

        This makes the threshold resolution-independent:
        - 0.15 means the center moved 15% of the frame width in one second.
        - Works identically on 720p, 1080p, and 4K footage.

        Args:
            bbox1: Previous bounding box (x1, y1, x2, y2).
            bbox2: Current bounding box (x1, y1, x2, y2).
            fps: Effective frames per second (after frame-skipping).
            frame_width: Width of the video frame in pixels.
```

```
        """
        raw_velocity = get_velocity(bbox1, bbox2, fps)
        if frame_width <= 0:
            return raw_velocity  # Fallback to raw pixels if width unknown
        return raw_velocity / frame_width

    def calculate_iou(boxA: Tuple[float, float, float, float], boxB: Tuple[float, float,
float, float]) -> float:
        """Calculate the Intersection over Union (IoU) of two bounding boxes.
        bbox format: (x1, y1, x2, y2)
        """
        xA = max(boxA[0], boxB[0])
        yA = max(boxA[1], boxB[1])
        xB = min(boxA[2], boxB[2])
        yB = min(boxA[3], boxB[3])

        interArea = max(0, xB - xA + 1) * max(0, yB - yA + 1)
        if interArea == 0:
            return 0.0

        boxAArea = (boxA[2] - boxA[0] + 1) * (boxA[3] - boxA[1] + 1)
        boxBArea = (boxB[2] - boxB[0] + 1) * (boxB[3] - boxB[1] + 1)

        iou = interArea / float(boxAArea + boxBArea - interArea)
        return iou

```

59. user (2026-06-10T05:39:20.682Z)

```
    """Audio shuttle-hit ("thwack") detection ? local, dependency-light (numpy + ffmpeg).

    Stages 0?1 of docs/AUDIO_RALLY_DETECTION_PLAN.md: extract the audio track, build a
    spectral-flux onset envelope (with pre-emphasis to boost the broadband HF transient
    a racket?shuttle contact makes), threshold it adaptively, and return (timestamp,
    strength) hit candidates. Pure numpy + stdlib `wave` (NO scipy); ffmpeg (subprocess)
    only for extraction. Consumed by the E1 go/no-go probe; later feeds rally fusion
    (ADR-007) if E1 passes. WASB is never modified ? this is a separate signal.
    """
    from __future__ import annotations

    import os
    import wave
    import tempfile
    import subprocess
    from dataclasses import dataclass
    from typing import List, Tuple

    import numpy as np


    @dataclass
    class HitEvent:
        t: float         # seconds
        strength: float  # onset-envelope peak value (arbitrary units)


    def extract_audio(video_path: str, out_wav: str = None, sr: int = 16000) -> str:
        """ffmpeg ? mono 16-bit PCM WAV at `sr`. Returns the wav path (temp if not given).
        Audio-only decode, so it's fast even for multi-GB video."""
        if out_wav is None:
            fd, out_wav = tempfile.mkstemp(suffix=".wav")
            os.close(fd)
        cmd = ["ffmpeg", "-y", "-i", video_path, "-vn", "-ac", "1", "-ar", str(sr),
               "-c:a", "pcm_s16le", out_wav]
        subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
```

```
stderr=subprocess.DEVNULL)
        return out_wav


    def load_wav(path: str) -> Tuple[np.ndarray, int]:
        """Read a PCM WAV ? (float32 samples in [-1,1], sample_rate). Mono-downmixes."""
        with wave.open(path, "rb") as w:
            sr, n, sw, ch = w.getframerate(), w.getnframes(), w.getsampwidth(),
w.getnchannels()
            raw = w.readframes(n)
        if sw == 2:
            a = np.frombuffer(raw, dtype=np.int16).astype(np.float32) / 32768.0
        elif sw == 4:
            a = np.frombuffer(raw, dtype=np.int32).astype(np.float32) / 2147483648.0
        elif sw == 1:
            a = np.frombuffer(raw, dtype=np.uint8).astype(np.float32) / 128.0 - 1.0
        else:
            raise ValueError(f"unsupported sample width {sw}")
        if ch > 1:
            a = a.reshape(-1, ch).mean(axis=1)
        return a, sr


    def _preemphasis(x: np.ndarray, coef: float = 0.97) -> np.ndarray:
        if len(x) < 2:
            return x.copy()
        return np.append(x[0], x[1:] - coef * x[:-1])


    def onset_envelope(samples: np.ndarray, sr: int, frame: int = 1024,
                       hop: int = 256, band: Tuple[float, float] = None) ->
Tuple[np.ndarray, float]:
        """Spectral-flux onset strength per hop (pure-numpy STFT). Returns (env,
hop_seconds).

        `band=(lo_hz, hi_hz)` restricts the flux to a frequency band ? a shuttle "thwack"
        is HF-rich while voices/footsteps live lower, so band-limiting (e.g. 3-9 kHz)
        suppresses non-shuttle transients. Needs sr high enough for hi_hz (Nyquist =
sr/2)."""
        x = _preemphasis(np.asarray(samples, dtype=np.float32))
        hop_sec = hop / float(sr)
        if len(x) < frame:
            return np.zeros(0, dtype=np.float32), hop_sec
        win = np.hanning(frame).astype(np.float32)
        n_frames = 1 + (len(x) - frame) // hop
        idx = np.arange(frame)[None, :] + hop * np.arange(n_frames)[:, None]
        frames = x[idx] * win
        mag = np.abs(np.fft.rfft(frames, axis=1))
        if band is not None:
            freqs = np.fft.rfftfreq(frame, d=1.0 / sr)
            keep = (freqs >= band[0]) & (freqs <= band[1])
            if keep.any():
                mag = mag[:, keep]                                  # band-limited flux
        diff = np.diff(mag, axis=0)                                 # frame-to-frame change
        flux = np.sqrt((np.maximum(diff, 0.0) ** 2).sum(axis=1))    # positive (onset) flux
        env = np.concatenate([[0.0], flux]).astype(np.float32)      # align to n_frames
        return env, hop_sec


    def _moving_stats(env: np.ndarray, win: int) -> Tuple[np.ndarray, np.ndarray]:
        """Sliding-window mean and std (reflect-padded, via cumsum). win forced odd."""
        if win % 2 == 0:
            win += 1
        pad = win // 2
        p = np.pad(env.astype(np.float64), pad, mode="reflect")
```

```
97          c1 = np.cumsum(np.insert(p, 0, 0.0))
98          c2 = np.cumsum(np.insert(p * p, 0, 0.0))
99          s1 = c1[win:] - c1[:-win]
100         s2 = c2[win:] - c2[:-win]
101         mean = s1 / win
102         var = np.maximum(s2 / win - mean * mean, 0.0)
103         return mean[:len(env)], np.sqrt(var)[:len(env)]
104
105
106     def detect_hits(samples: np.ndarray, sr: int, k: float = 2.5, win_frames: int = 41,
107                     min_gap_sec: float = 0.08, band: Tuple[float, float] = None) ->
List[HitEvent]:
108         """Detect impact transients: local maxima of the onset envelope that exceed an
109         adaptive threshold (moving mean + k·std), keeping the strongest within
`min_gap_sec`.
110         `k` is the sensitivity knob (lower = more hits); `band` band-limits the onset flux
111         to the shuttle-click frequencies (e.g. (3000, 9000))."""
112         env, hop_sec = onset_envelope(samples, sr, band=band)
113         if len(env) < 3:
114             return []
115         mean, std = _moving_stats(env, win_frames)
116         thr = mean + k * std
117         is_peak = (env[1:-1] >= env[:-2]) & (env[1:-1] > env[2:]) & (env[1:-1] > thr[1:-1])
118         peak_idx = np.nonzero(is_peak)[0] + 1
119         if peak_idx.size == 0:
120             return []
121         # enforce min inter-peak gap: greedily keep strongest first
122         min_gap = max(1, int(round(min_gap_sec / hop_sec)))
123         order = peak_idx[np.argsort(-env[peak_idx])]
124         taken = np.zeros(len(env), dtype=bool)
125         kept = []
126         for i in order:
127             lo, hi = max(0, i - min_gap), min(len(env), i + min_gap + 1)
128             if taken[lo:hi].any():
129                 continue
130             taken[i] = True
131             kept.append(i)
132         kept.sort()
133         return [HitEvent(t=float(i * hop_sec), strength=float(env[i])) for i in kept]
134
135
136     def detect_hits_in_video(video_path: str, sr: int = 16000, **kw) -> List[HitEvent]:
137         """Convenience: extract audio from a video and detect hits. Cleans up the temp
wav."""
138         wav = extract_audio(video_path, sr=sr)
139         try:
140             samples, real_sr = load_wav(wav)
141             return detect_hits(samples, real_sr, **kw)
142         finally:
143             try:
144                 os.remove(wav)
145             except OSError:
146                 pass
147
```

60. user (2026-06-10T05:39:30.316Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\config.json:38:
"yolo_parallel_chunks": false,
C:\Users\avidu\Projects\badminton-highlight-indexer\config.json:39:        "yolo_chunk_workers":
2,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:59:
yolo_parallel_chunks: bool = False
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:60:
yolo_chunk_workers: int = 2
```

```
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py:342:        prefetch_workers =
idx_cfg.get("yolo_prefetch_workers", 2)
```

61. user (2026-06-10T05:39:32.093Z)

```
No matches found
```

62. user (2026-06-10T05:39:32.714Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\motion.py:33:
return Failure(message=f"Could not open video: {video_path}", is_recoverable=False)
```

63. user (2026-06-10T05:39:43.697Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:78:
default_highlights_name: str = "highlights.mp4"
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:88:    use_cache:
bool = False
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:89:
min_shot_count: int = 1
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:60:
# A: Check for debug_duration to trim the video first
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:61:
debug_duration = self.config.get("indexing", {}).get("debug_duration")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:68:
if debug_duration:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:69:
logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration} seconds for fast
processing...")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:81:
"-t", str(debug_duration),
```

64. user (2026-06-10T05:39:44.585Z)

```
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py:326:        chunks_dir =
os.path.join("output", "chunks_yolo")
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py:480:        gemini_dir =
os.path.join("output", f"chunks_{provider_name}_handoff")
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\wasb_hybrid.py:114:        out_dir = os.path.join("output",
"wasb")
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\wasb_hybrid.py:155:        handoff_dir =
os.path.join("output", f"chunks_{provider_name}_handoff")
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\tracknet_hybrid.py:127:        tn_out_dir =
os.path.join("output", "tracknet")
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\tracknet_hybrid.py:175:        handoff_dir =
os.path.join("output", f"chunks_{provider_name}_handoff")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:70:
temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:119:
chunks_dir = os.path.join("output", "chunks")
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\eval\regression.py:30:DEFAULT_BASELINE_PATH = os.path.join("output",
"regression_baseline.json")
```

65. assistant (2026-06-10T05:39:55.601Z)

Let me empirically verify the nested-config drop behavior ? this is load-bearing for a high-

severity finding.

66. user (2026-06-10T05:40:00.602Z)

```
indexing keys: ['ai_handoff_padding', 'ai_max_workers', 'chunk_duration', 'chunk_overlap',
'dead_time_merge_gap', 'default_segmenter', 'detector_model', 'detector_score_threshold',
'log_yolo_candidates', 'min_action_window_duration', 'min_segment_duration', 'motion_threshold',
'store_yolo_candidates', 'tracknet', 'use_gpu', 'yolo_chunk_workers', 'yolo_frame_skip',
'yolo_parallel_chunks', 'yolo_tracker', 'yolo_velocity_threshold']
wasb in indexing dump: False
debug_duration in dump: False
yolo_prefetch_workers in dump: False
top-level extra kept: {'kept': True}
hasattr indexing.wasb: False
```

67. user (2026-06-10T05:40:12.388Z)

```
74:    # Own provider/client per call: the genai client isn't reliably thread-safe, and a
75:    # shared one across workers throws socket errors. A fresh lazy client is cheap.
121:    with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
186:    with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
```

68. user (2026-06-10T05:40:13.630Z)

```
backend/api/:
static

backend/tools/:
__init__.py
__pycache__
export.py
rally_slicer.py
```

69. user (2026-06-10T05:40:36.113Z)

```
33:from backend.config import load_config
90:    cfg = load_config()  # typed AppConfig (same object the live pipeline feeds segmenters)
```

70. user (2026-06-10T05:41:02.894Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py:206:        raise
HTTPException(status_code=500, detail=f"Highlight stitching failed: {str(e)}")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:86:
begin_padding: float = 0.5
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:87:    end_padding:
float = 0.5
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:106:    stitching:
StitchingConfig = Field(default_factory=StitchingConfig)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\static\app.js:347:
showLoader("Compiling Highlight Reels", "Triggering localized high-speed FFmpeg stitching
demuxers...");
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py:10:
def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float = 0.5):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py:12:
self.end_padding = end_padding
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py:13:
self.begin_padding = begin_padding
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py:67:
into a seamless highlights file, applying end_padding to prevent abrupt endings.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py:90:
logger.info(f"Trimming {len(segments)} segments (begin_padding={self.begin_padding}s,
end_padding={self.end_padding}s)...")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py:125:
padded_start = max(0.0, start - self.begin_padding)
```

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py:126:
padded_end = end + self.end_padding
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py:131:
logger.info(f"{segment_label}: Padded end ({end + self.end_padding:.2f}s) exceeds video
duration. "
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\gemini_refine.py:40:
"""Union of overlapping intervals, optionally stitching across a small gap.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:379:
stitch_cfg = self.config.get("stitching", {})
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:380:
begin_pad = stitch_cfg.get("begin_padding", 0.5)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:381:
end_pad = stitch_cfg.get("end_padding", 0.5)
```

71. user (2026-06-10T05:44:49.664Z)

```
Structured output provided successfully
```

72. assistant (2026-06-10T05:44:50.606Z)

```
You've hit your session limit · resets 3:50pm (Asia/Calcutta)
```